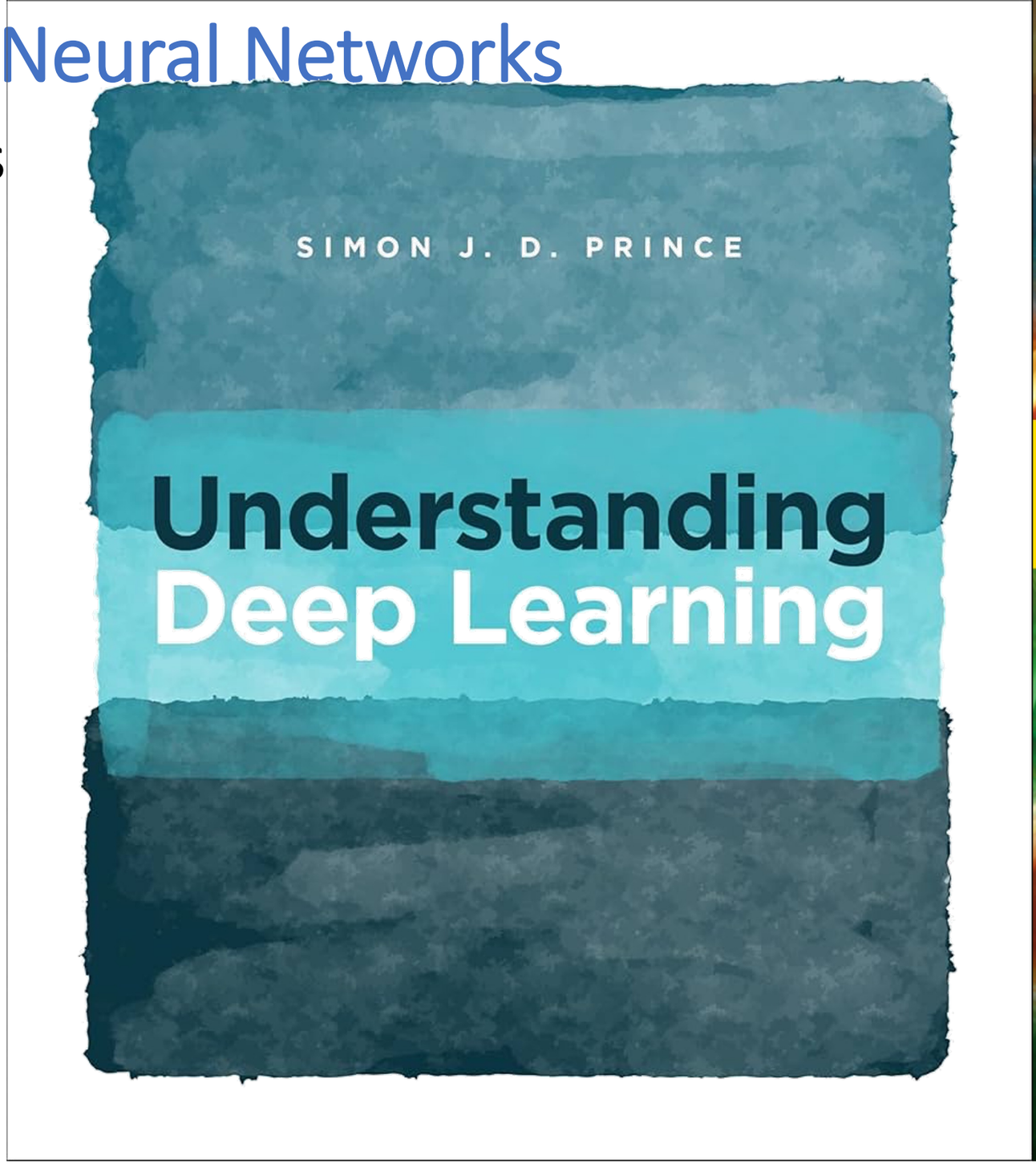
CS-663
Digital Image Processing (DIP)

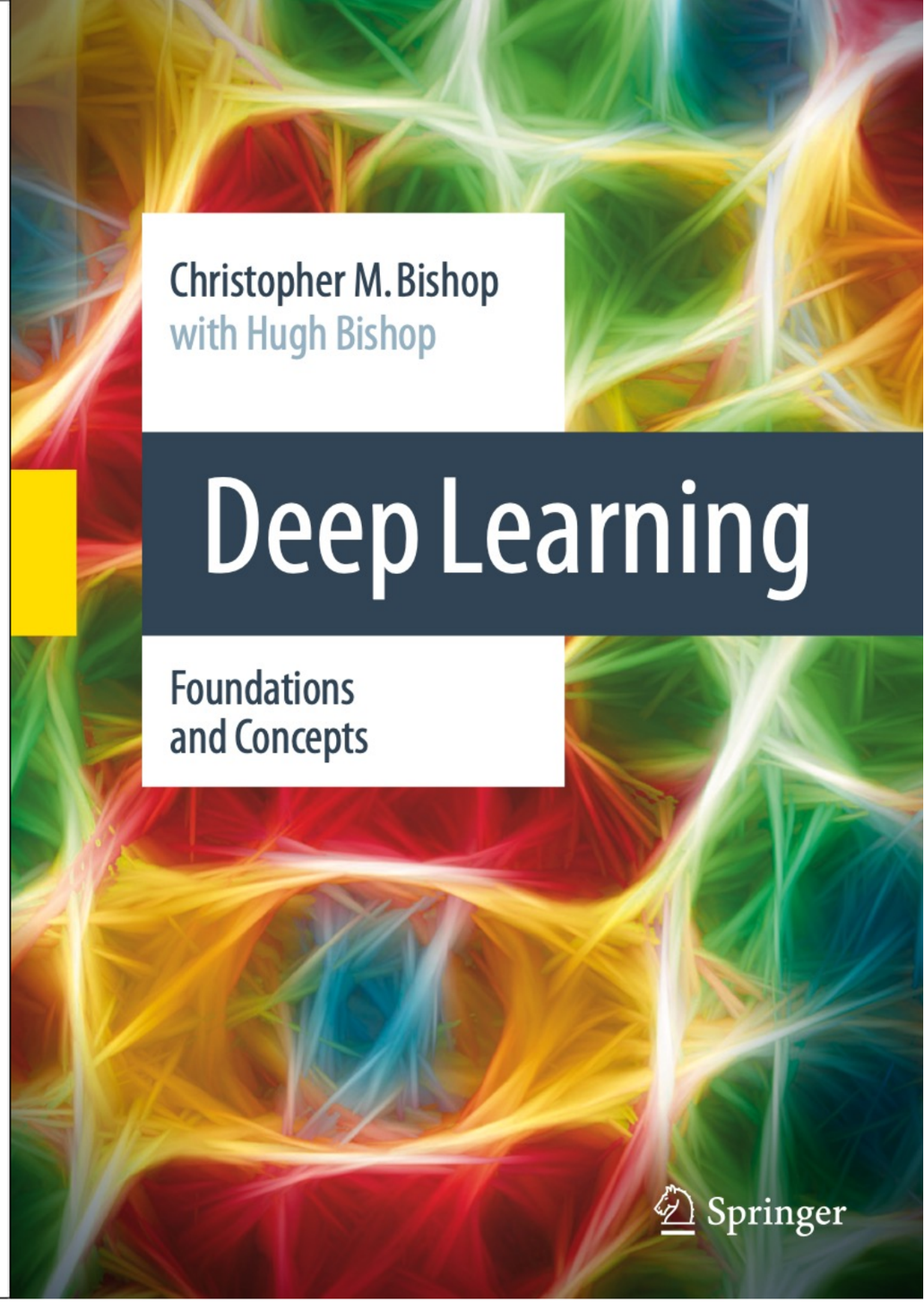
Deep Neural Networks

Suyash P. Awate

Deep Neural Networks

- Books





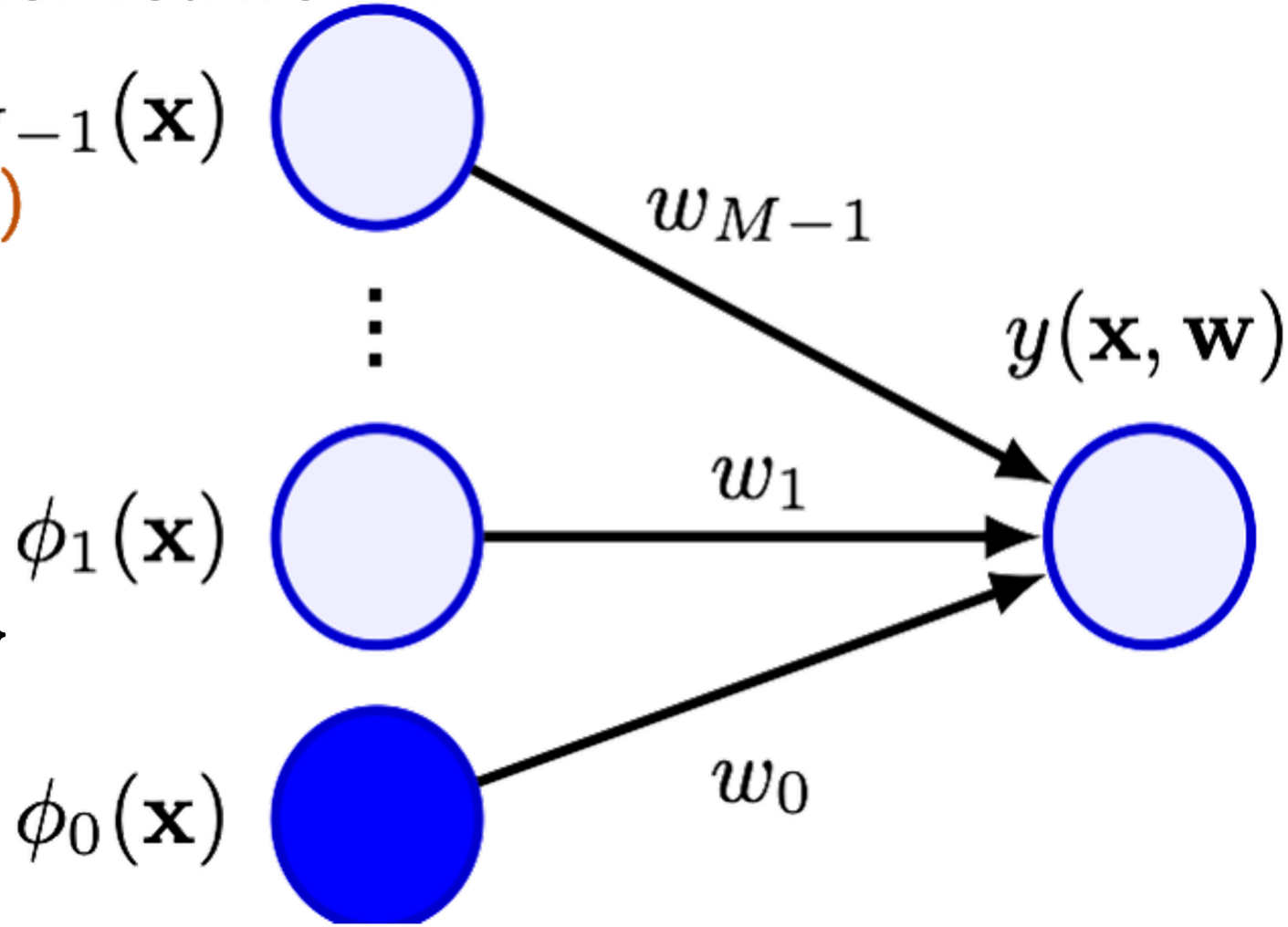
Single-Layer Networks: Regression

- **Model**: Predict (scalar) **y** as a linear function of (vector) **x** …
- … or as a linear function of fixed nonlinear functions **ϕ_j(.) derived from x**

$$y(\mathbf{x}, \mathbf{w}) = w_0 + w_1 x_1 + \ldots + w_D x_D$$

$$y(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^{M-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^{\mathrm{T}} \boldsymbol{\phi}(\mathbf{x})$$

$$y(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x})$$



- NN input nodes: { x_j } or { $\phi_j(x)$ }
- NN output node: y(x,w)
- NN parameters: w

Single-Layer Networks: Regression

- **Target**/reference data **t** is assumed to be noisy, and modeled as:

$$p(t|\mathbf{x}, \mathbf{w}, \sigma^2) = \mathcal{N}(t|y(\mathbf{x}, \mathbf{w}), \sigma^2)$$

- **Training data**: set of inputs { x_n } & set of target/reference values { t_n }
- **Likelihood** function
  P ({t_n,X_n}|w,σ^2) ∝

$$p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \sigma^2) = \prod_{n=1}^{N} \mathcal{N}(t_n|\mathbf{w}^{\mathrm{T}}\boldsymbol{\phi}(\mathbf{x}_n), \sigma^2)$$

- Maximizing log-likelihood function

$$\ln p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \sigma^2) = \sum_{n=1}^{N} \ln \mathcal{N}(t_n|\mathbf{w}^{\mathrm{T}}\boldsymbol{\phi}(\mathbf{x}_n), \sigma^2)$$

$$= -\frac{N}{2}\ln\sigma^2 - \frac{N}{2}\ln(2\pi) - \frac{1}{\sigma^2}E_D(\mathbf{w})$$

- Minimizing loss function

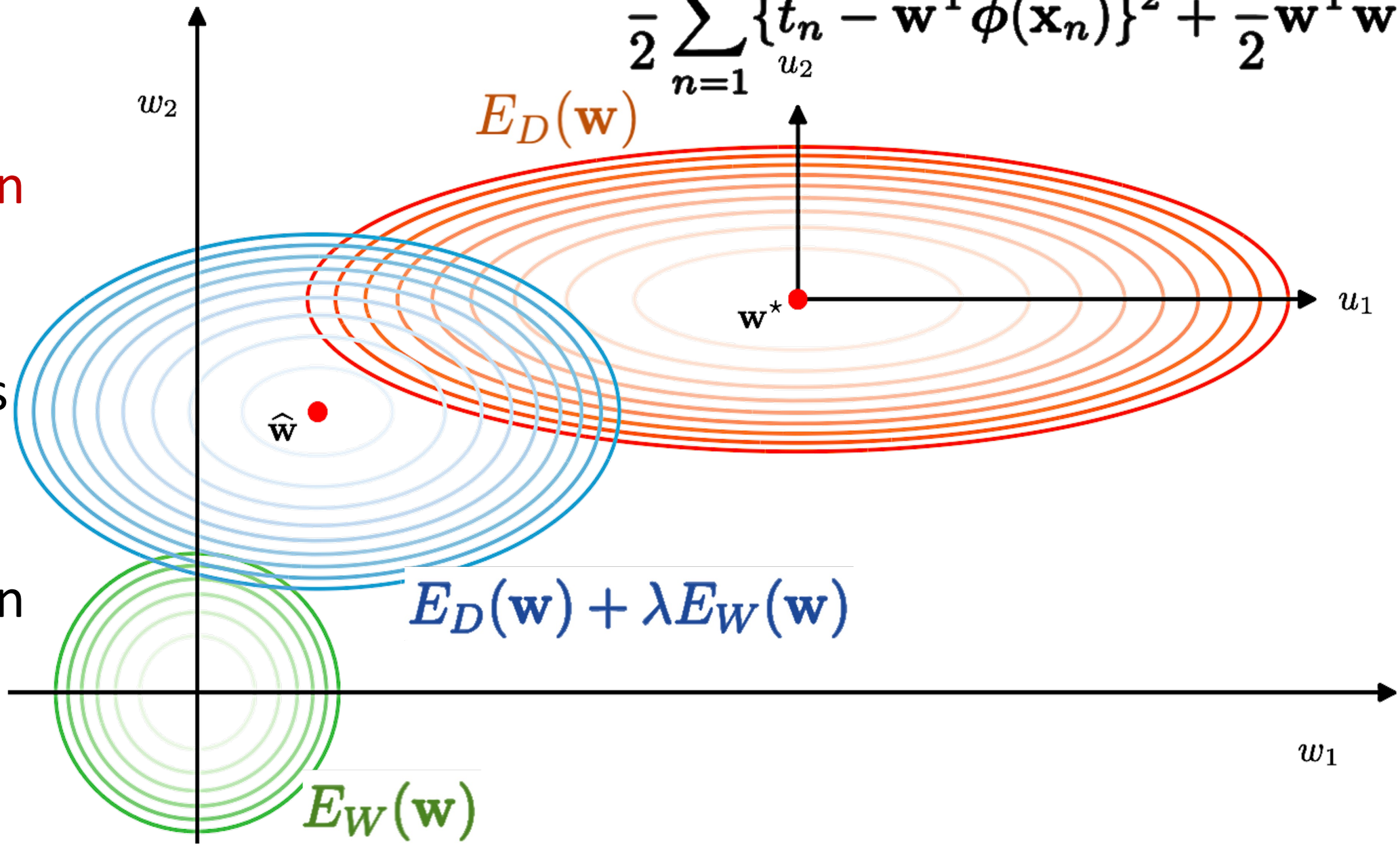
$$E_D(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\{t_n - \mathbf{w}^{\mathrm{T}}\boldsymbol{\phi}(\mathbf{x}_n)\}^2$$

Single-Layer Networks: Regression

- Minimizing loss function including regularization
- Optimal w estimates shift towards origin
- Amount of regularization λ is typically a free parameter

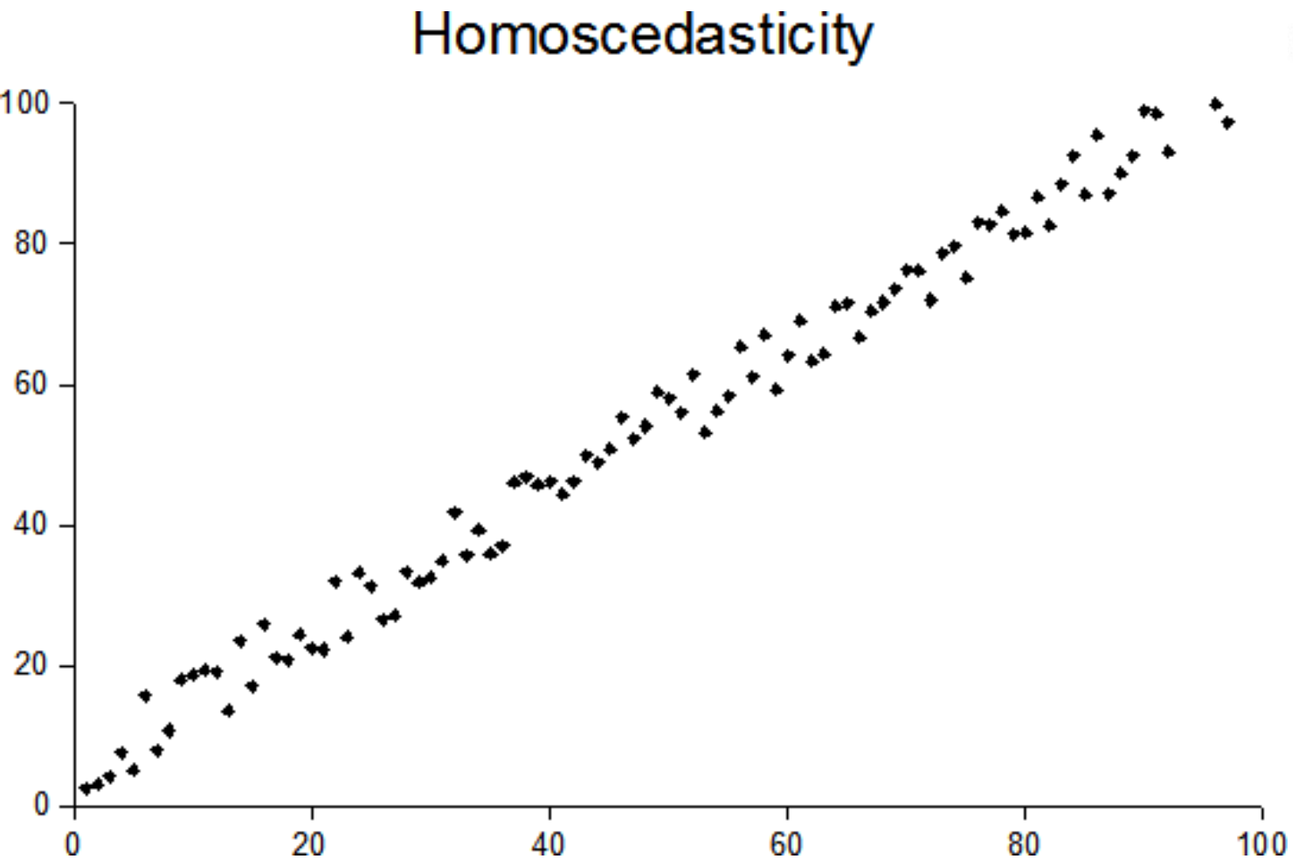
$$E_D(\mathbf{w}) + \lambda E_W(\mathbf{w})$$

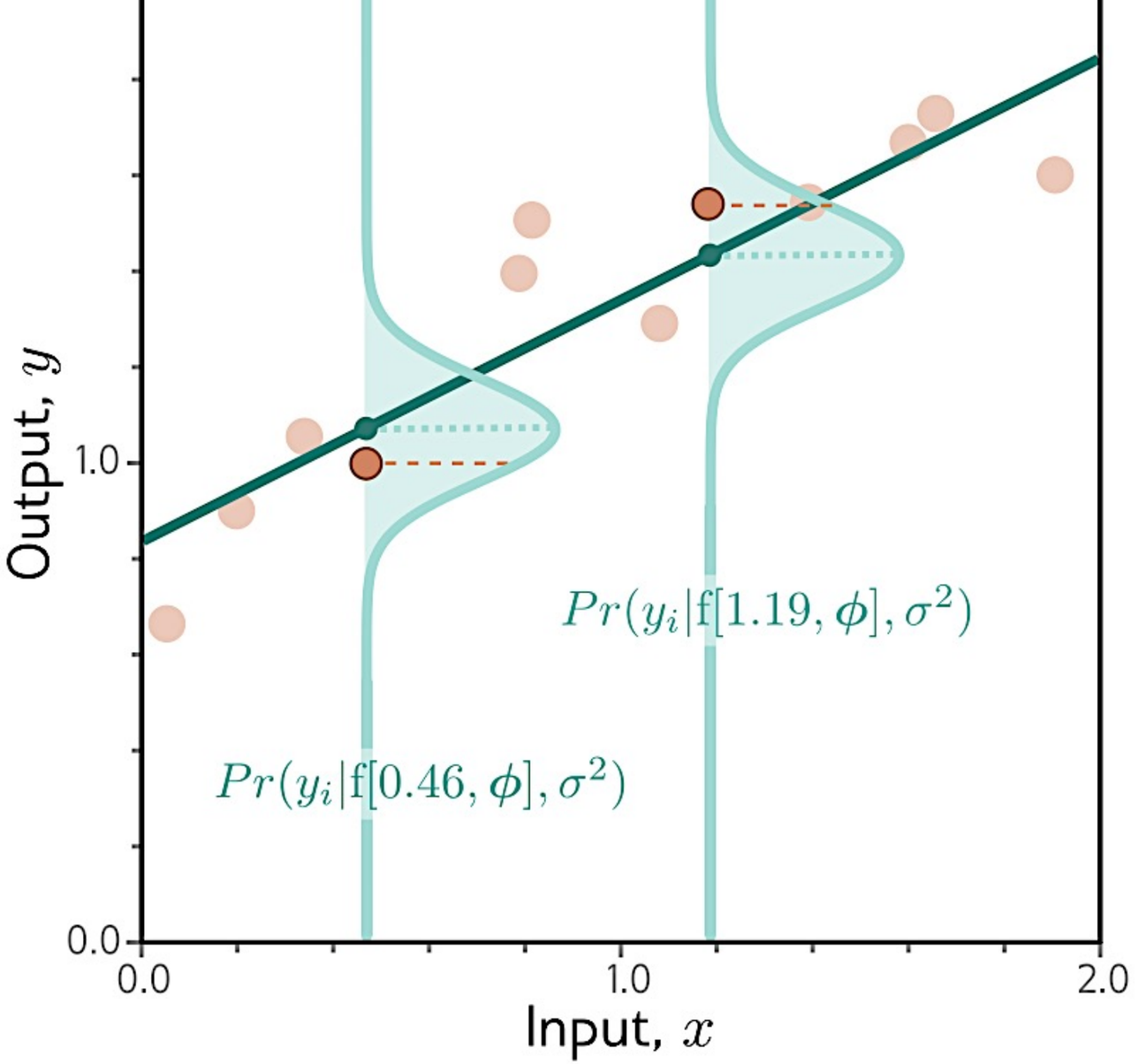
$$\frac{1}{2}\sum_{n=1}^{N}\{t_n - \mathbf{w}^{\mathrm{T}}\boldsymbol{\phi}(\mathbf{x}_n)\}^2 + \frac{\lambda}{2}\mathbf{w}^{\mathrm{T}}\mathbf{w}$$



Regression: Homoscedastic

- Notation change:
  weights are ϕ (not w)
- f(.) can represent
  some (single-layer) network





Regression: Homoscedastic

- Notation change: weights are ϕ (not w)

$$\hat{\boldsymbol{\phi}} = \underset{\boldsymbol{\phi}}{\operatorname{argmin}} \left[-\sum_{i=1}^{I} \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(y_i - \mathbf{f}[\mathbf{x}_i, \boldsymbol{\phi}])^2}{2\sigma^2} \right] \right] \right]$$

1) Constant **known** variance

$$= \underset{\boldsymbol{\phi}}{\operatorname{argmin}} \left[-\sum_{i=1}^{I} \left(\log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \right] - \frac{(y_i - \mathbf{f}[\mathbf{x}_i, \boldsymbol{\phi}])^2}{2\sigma^2} \right) \right]$$

$$= \underset{\boldsymbol{\phi}}{\operatorname{argmin}} \left[-\sum_{i=1}^{I} -\frac{(y_i - \mathbf{f}[\mathbf{x}_i, \boldsymbol{\phi}])^2}{2\sigma^2} \right]$$

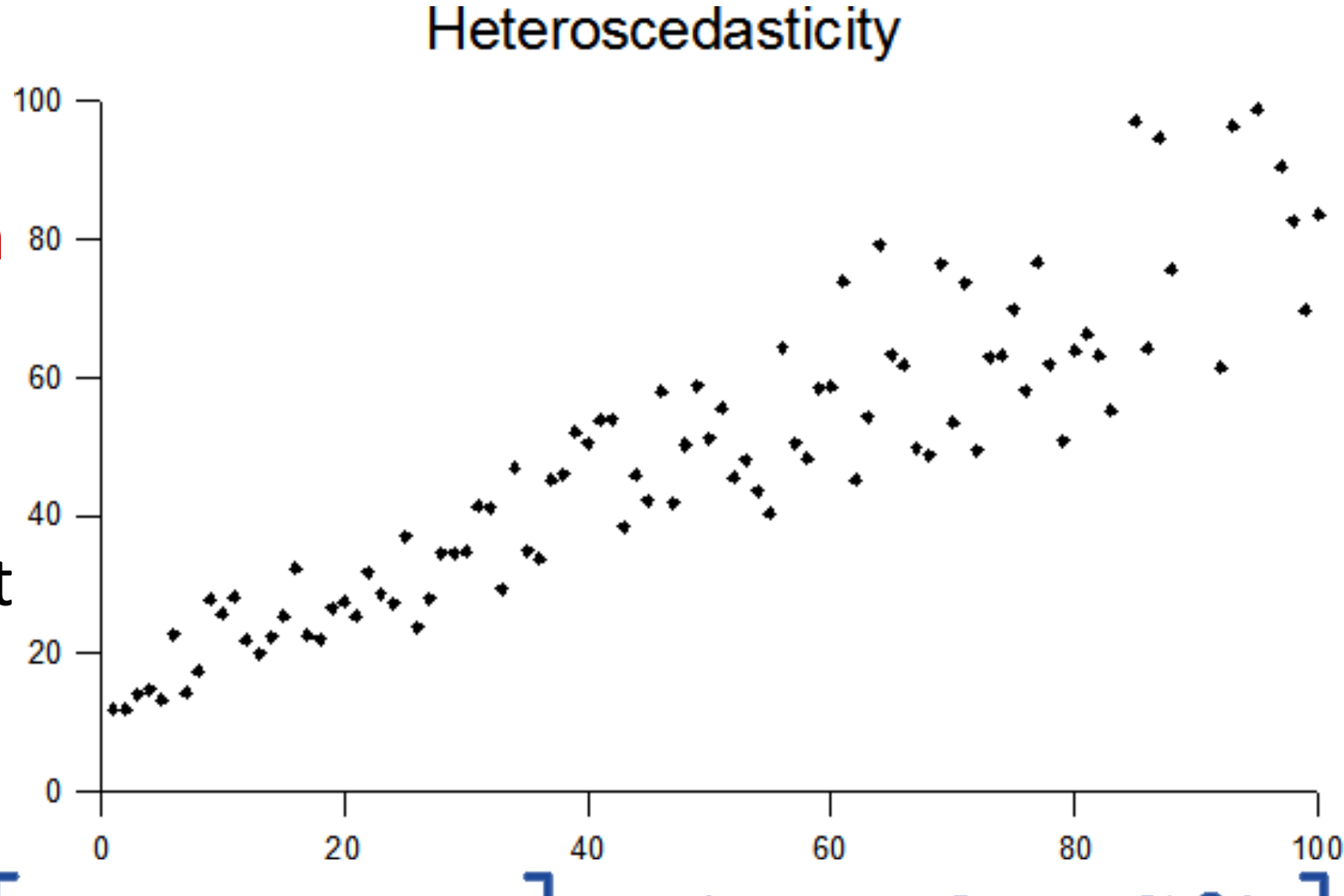
2) Constant **unknown** variance

$$= \underset{\boldsymbol{\phi}}{\operatorname{argmin}} \left[\sum_{i=1}^{I} (y_i - \mathbf{f}[\mathbf{x}_i, \boldsymbol{\phi}])^2 \right],$$

$$\hat{\boldsymbol{\phi}}, \hat{\sigma}^2 = \underset{\boldsymbol{\phi}, \sigma^2}{\operatorname{argmin}} \left[-\sum_{i=1}^{I} \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(y_i - \mathbf{f}[\mathbf{x}_i, \boldsymbol{\phi}])^2}{2\sigma^2} \right] \right] \right]$$

Regression: Heteroscedastic

- Notation change: weights are ϕ (not w)
- Network $f_1(.)$ predicts mean
- Network $f_2(.)$ predicts standard deviation, which isn't constant, but is input-data dependent



$$\mu = \mathrm{f}_1[\mathbf{x}, \boldsymbol{\phi}]$$

$$\sigma^2 = \mathrm{f}_2[\mathbf{x}, \boldsymbol{\phi}]^2$$

$$\hat{\boldsymbol{\phi}} = \underset{\boldsymbol{\phi}}{\operatorname{argmin}} \left[-\sum_{i=1}^{I} \left(\log \left[\frac{1}{\sqrt{2\pi \mathrm{f}_2[\mathbf{x}_i, \boldsymbol{\phi}]^2}} \right] - \frac{(y_i - \mathrm{f}_1[\mathbf{x}_i, \boldsymbol{\phi}])^2}{2\mathrm{f}_2[\mathbf{x}_i, \boldsymbol{\phi}]^2} \right) \right]$$

Single-Layer Networks: Regression

- Limitations of
  linear combinations of
  fixed basis functions $\phi_j(.)$

$$y(\mathbf{x}, \mathbf{w}) = f\left(\sum_{j=1}^{M} w_j \phi_j(\mathbf{x}) + w_0\right)$$

1) For regression, f(.) can be identity
2) For classification (which is also a form of regression),
   f(.) = "activation" function,
   e.g., sigmoid in logistic regression

$$\sigma(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{1+e^x}$$

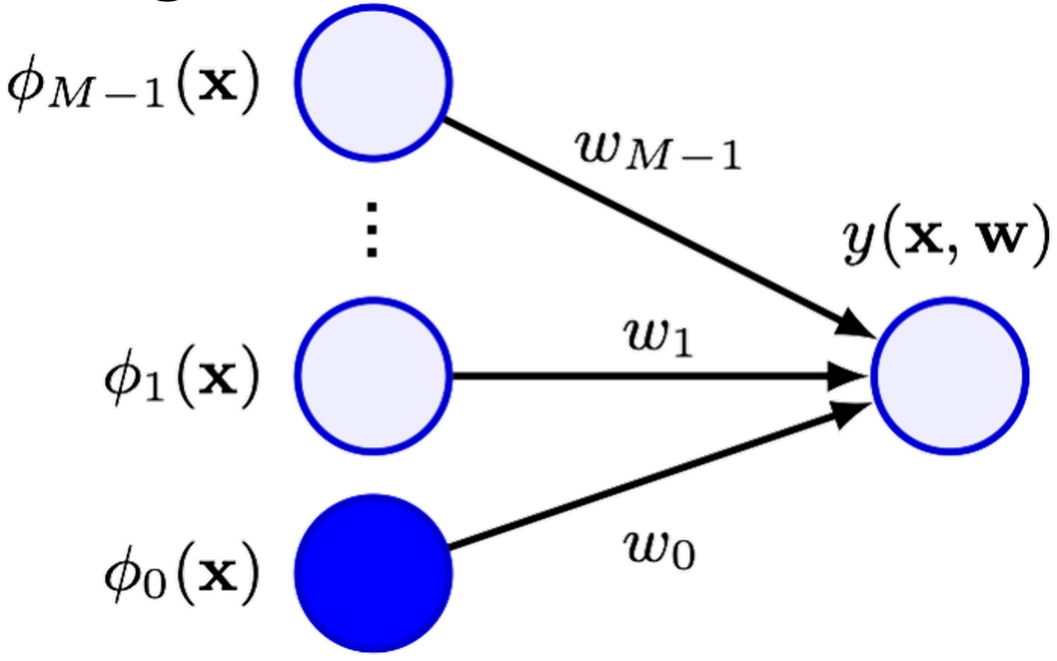
- For good-quality regression, we need a set of basis functions that:
  a) Has enough expressive power so that linear/log-linear model works well: indicating a large-enough set
  b) Protects us against overfitting: indicating a small-enough set
- So, let's learn a set of basis functions from data, optimized for task

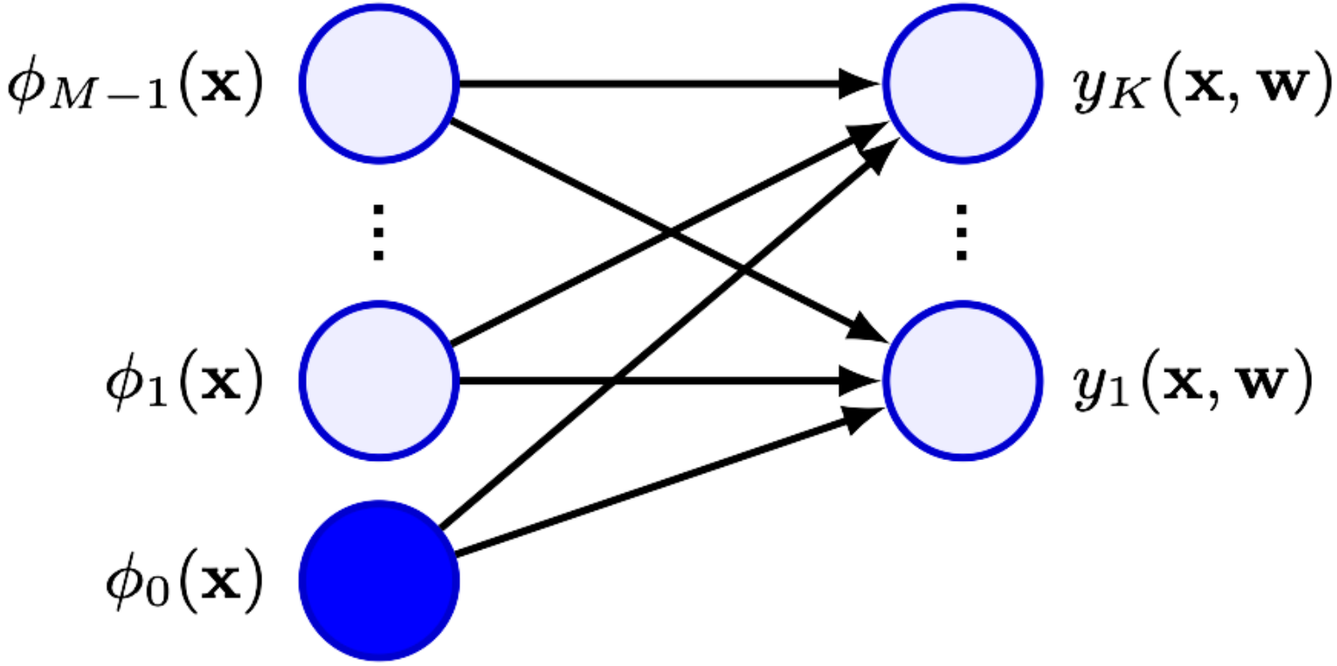
Single-Layer Networks: Classification

- **Multi-class classification**
- Activation function is softmax(.), actually soft(arg)max(.)
- Multi-class classification → (output K class probabilities)
- Regression↓

$$a_k = \mathbf{w}_k^{\mathrm{T}} \boldsymbol{\phi}$$

$$p(\mathcal{C}_k|\boldsymbol{\phi}) = y_k(\boldsymbol{\phi}) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}$$





Single-Layer Networks: Classification

- Multi-class classification
- Likelihood function ← categorical distribution
- Target vector t_n is one-hot vector indicating ground truth
- NN output is softmax vector y_n indicating predicted probabilities

$$p(\mathbf{T}|\mathbf{w}_1,\ldots,\mathbf{w}_K) = \prod_{n=1}^{N}\prod_{k=1}^{K} p(\mathcal{C}_k|\boldsymbol{\phi}_n)^{t_{nk}} = \prod_{n=1}^{N}\prod_{k=1}^{K} y_{nk}^{t_{nk}}$$

- Loss function := negative log-likelihood
  - In information theory, this is "cross entropy" between probability mass functions p(.) and q(.)

$$H(p,q) = -\sum_{x\in\mathcal{X}} p(x)\,\log q(x)$$

$$-\sum_{n=1}^{N}\sum_{k=1}^{K} t_{nk}\ln y_{nk}$$

Two-Layer Networks

- First layer

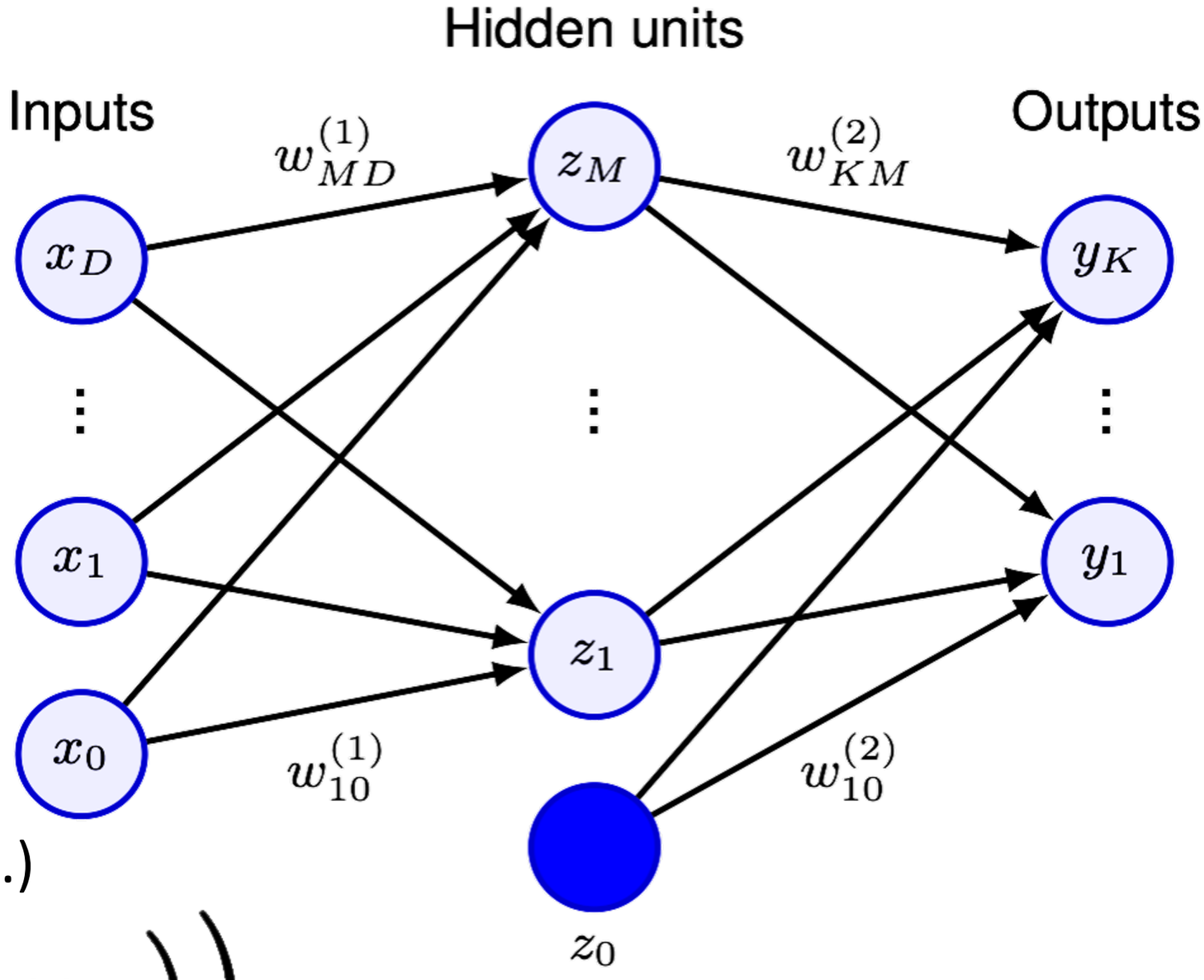
$$x_0 = 1$$

$$a_j = \sum_{i=0}^{D} w_{ji}^{(1)} x_i$$

  - Nonlinear activation function h(.)

- Second layer
  - Nonlinear activation function f(.)

$$y_k(\mathbf{x}, \mathbf{w}) = f\left(\sum_{j=0}^{M} w_{kj}^{(2)} h\left(\sum_{i=0}^{D} w_{ji}^{(1)} x_i\right)\right)$$



$$\mathbf{y}(\mathbf{x}, \mathbf{w}) = f\left(\mathbf{W}^{(2)} h\left(\mathbf{W}^{(1)} \mathbf{x}\right)\right)$$

Two-Layer Networks

- Two-layer feed-forward networks are “universal approximators”
- When using nonlinear (non-polynomial) activation functions
- Can model/ approximate any function defined over a continuous subset of R^D to arbitrary accuracy
- Many theoretical & empirical studies in 1980s

ELSEVIER

Neural Networks

Volume 2, Issue 5, 1989, Pages 359-366

Original contribution

Multilayer feedforward networks are universal approximators

Kurt Hornik

Technische Universität Wien Austria

Maxwell Stinchcombe, Halbert White [1]

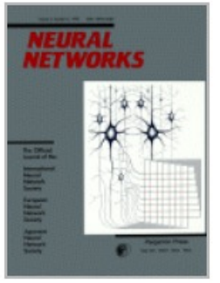
University of California, San Diego USA

Two-Layer Networks

ELSEVIER

Neural Networks

Volume 6, Issue 6, 1993, Pages 861-867



Original Contribution

Multilayer feedforward networks with a nonpolynomial activation function can approximate any function

Moshe Leshno [1], Vladimir Ya. Lin [2], Allan Pinkus [2], Shimon Schocken [3]

[1] The Hebrew University, Israel

[2] Technion, Israel

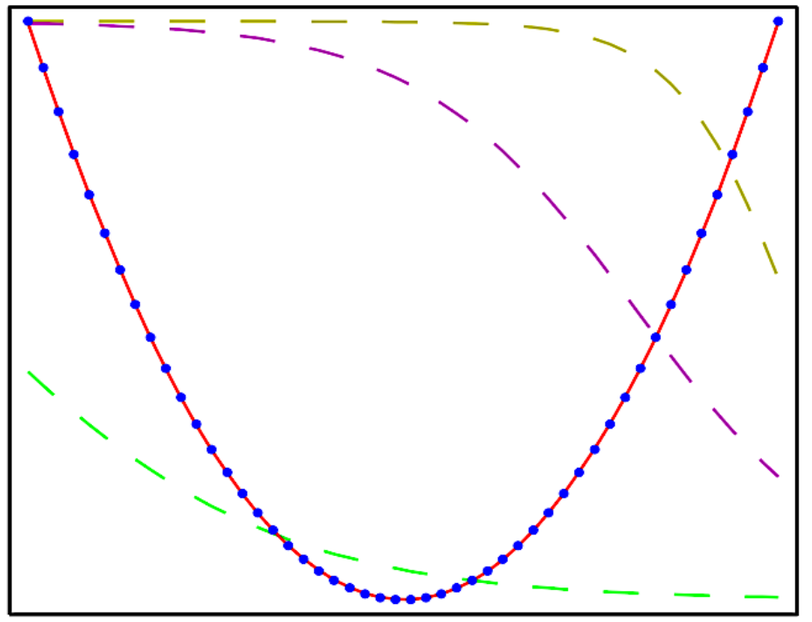
[3] New York University, USA

Two-Layer Networks

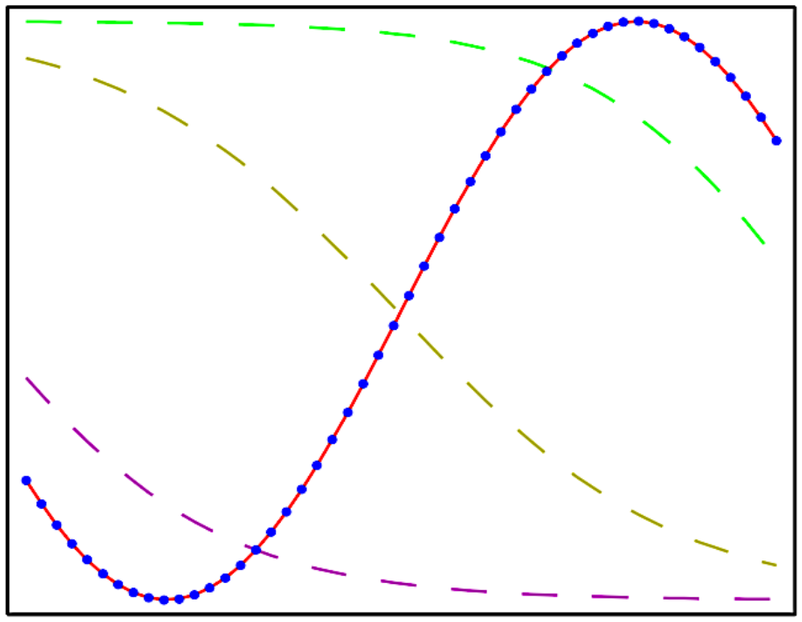
- Example
- 1 hidden layer with 3 units
- tanh activation function

$$\tanh x = \frac{\sinh x}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{e^{2x} - 1}{e^{2x} + 1}$$

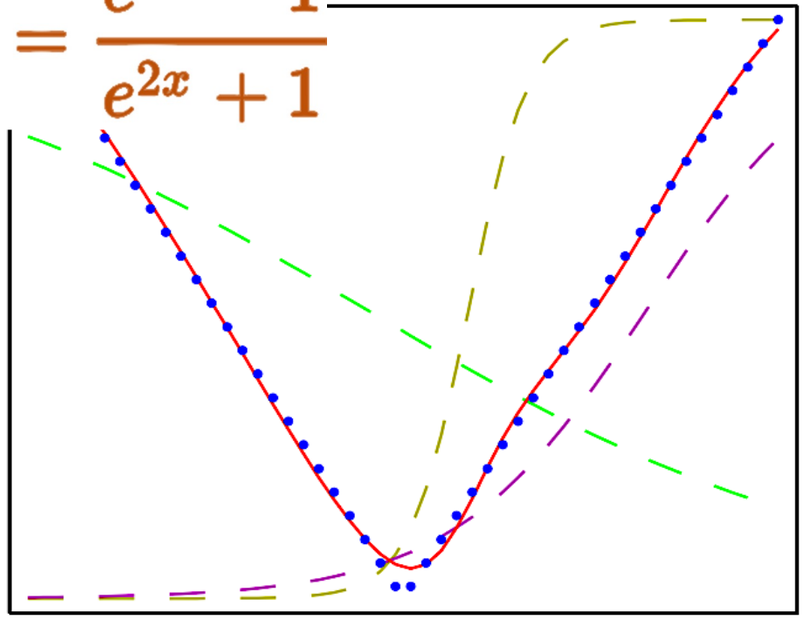
- Blue = data
- Red = fitted curve
- Dashed lines = outputs of hidden units



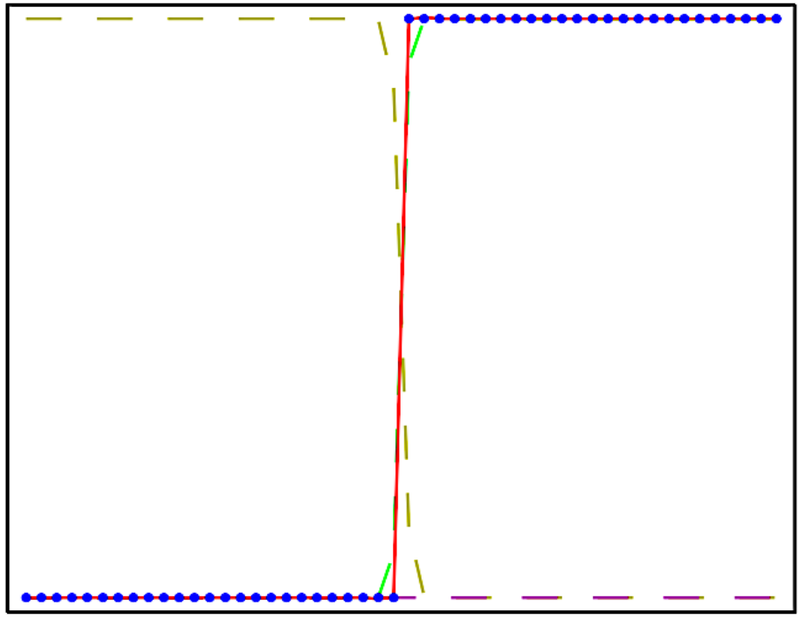
(a)



(b)



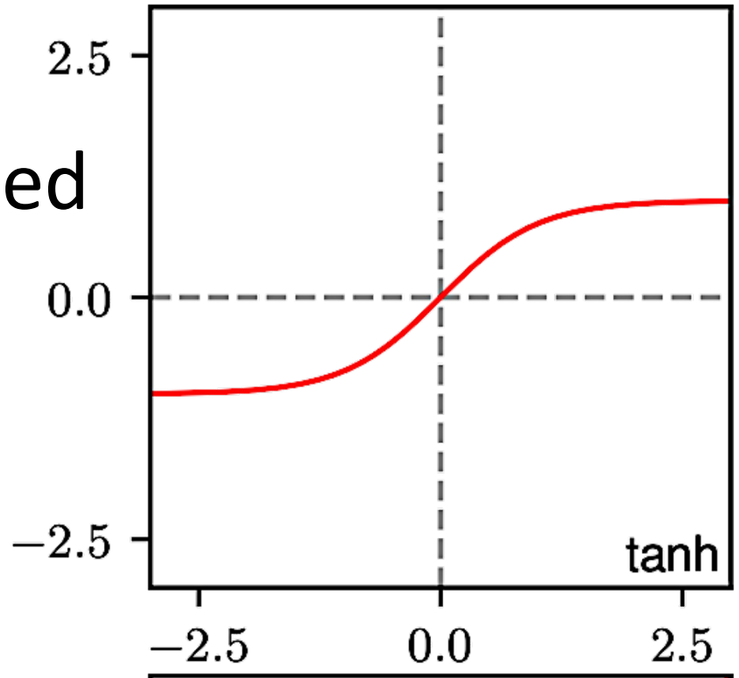
(c)

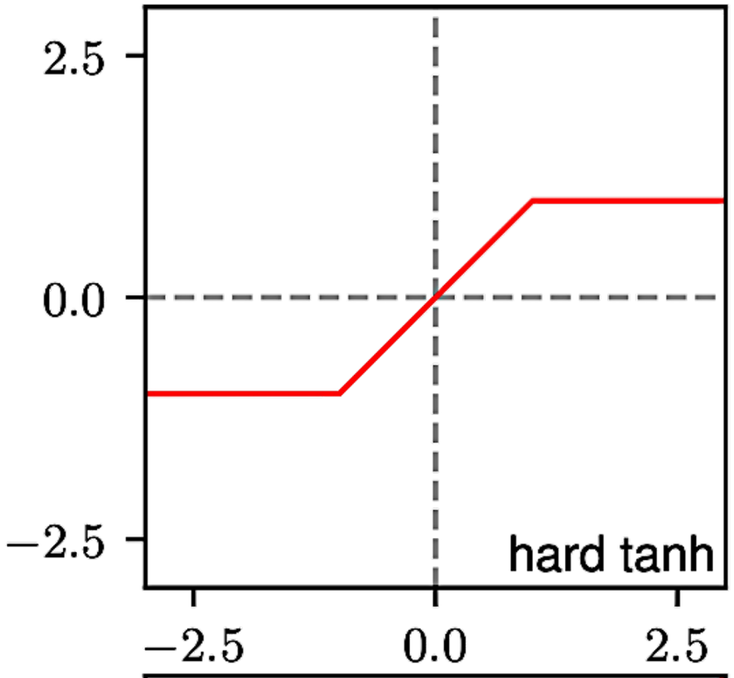


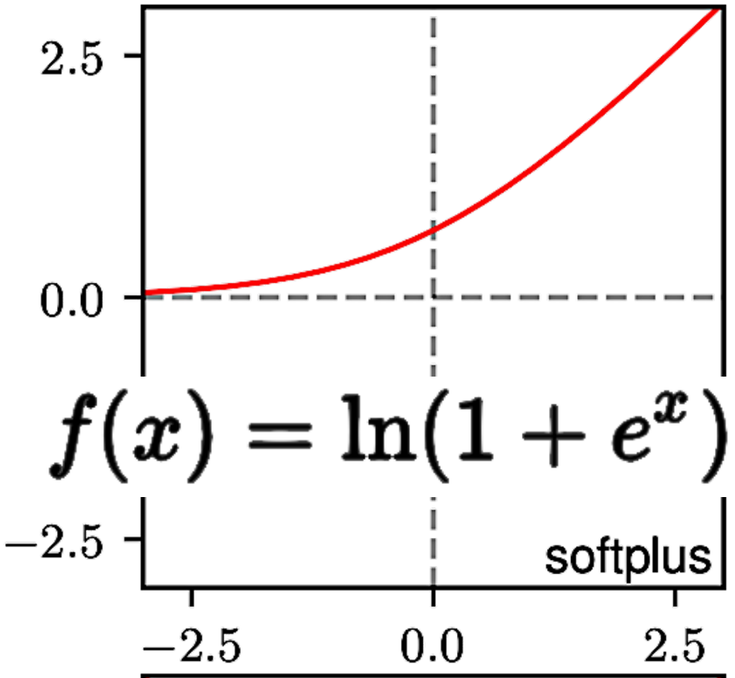
(d)

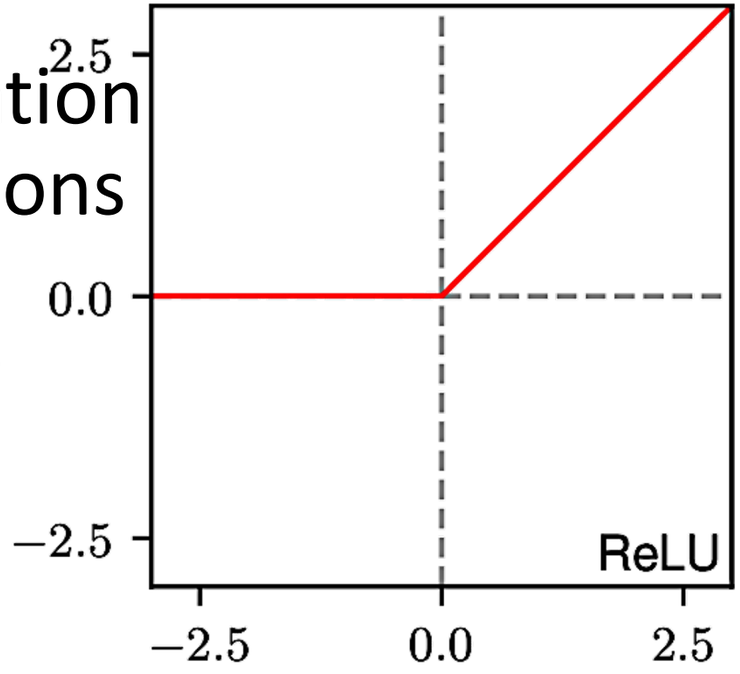
Activation Functions

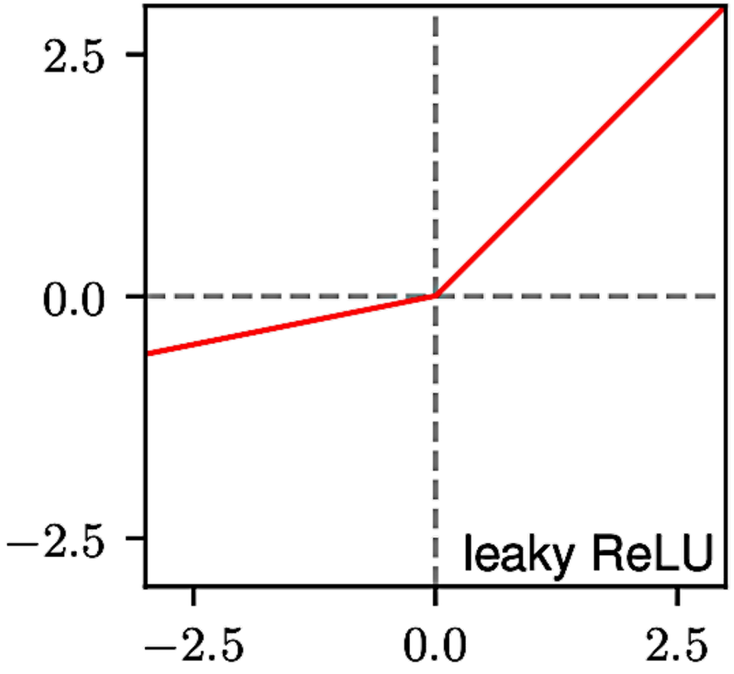
- ReLU: rectified linear unit
- What if our activation functions are linear ?

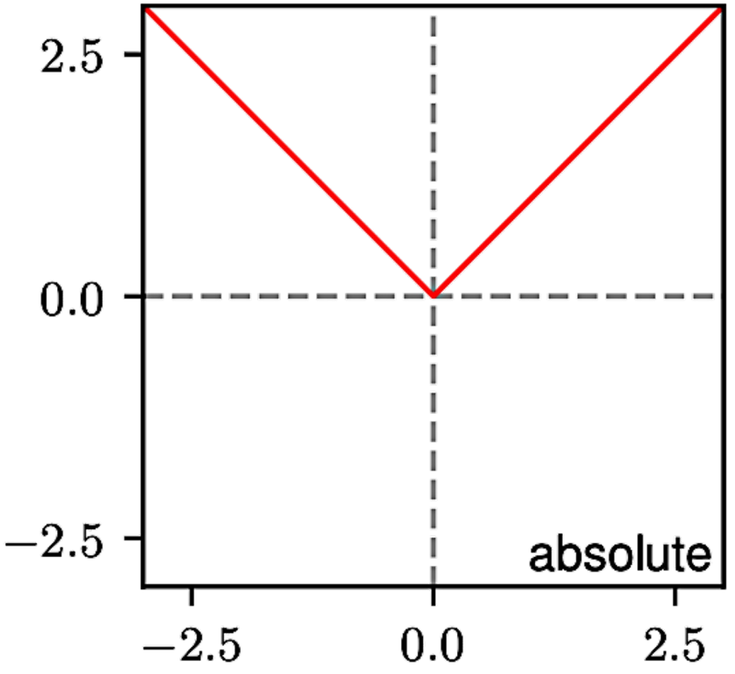






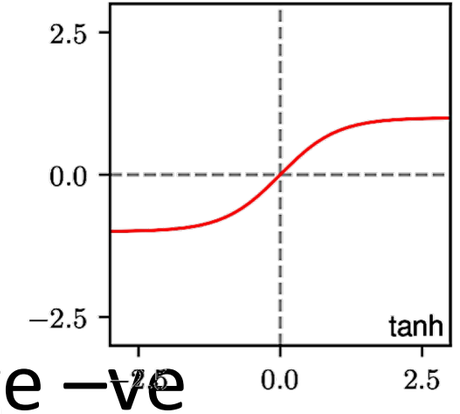


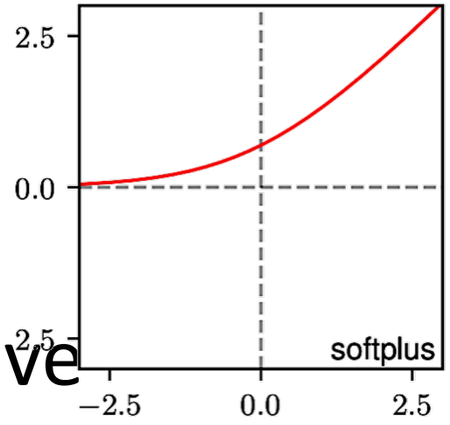


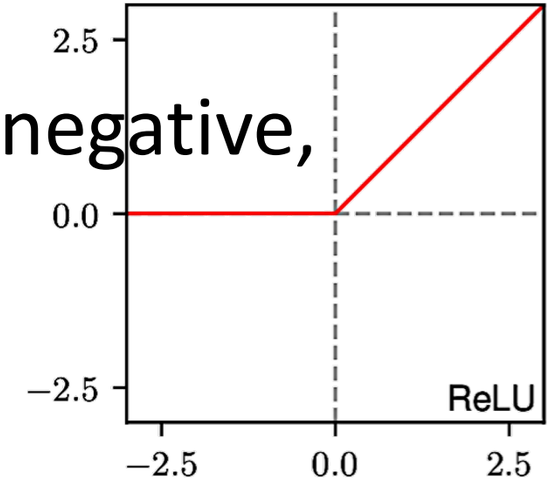


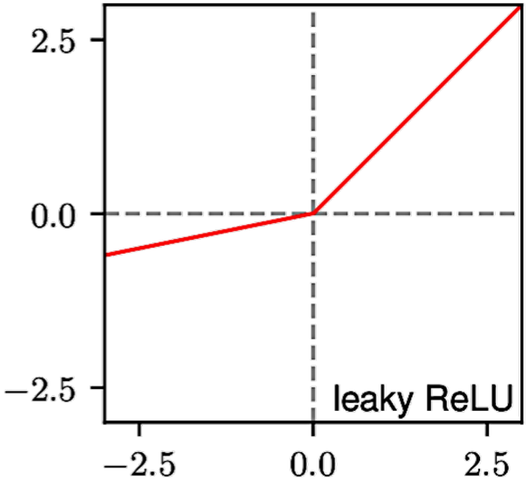
Activation Functions

- For some activation functions, e.g., sigmoid and tanh, gradients go to zero exponentially when activation-function inputs are either large +ve or large –ve
  - "Vanishing gradients"/derivatives problem
  - More about this later
- Softplus/ReLu is better (in that sense) because its derivative remains non-zero when its inputs are large positive
  - ReLu (& its derivative) is fast to evaluate → breakthrough in empirical analyses
  - Allowed learning of larger DNNs
- ReLu's problem is that, when inputs are negative, gradients are zero (no feedback signal)
  - Leaky ReLu avoids this limitation of ReLu



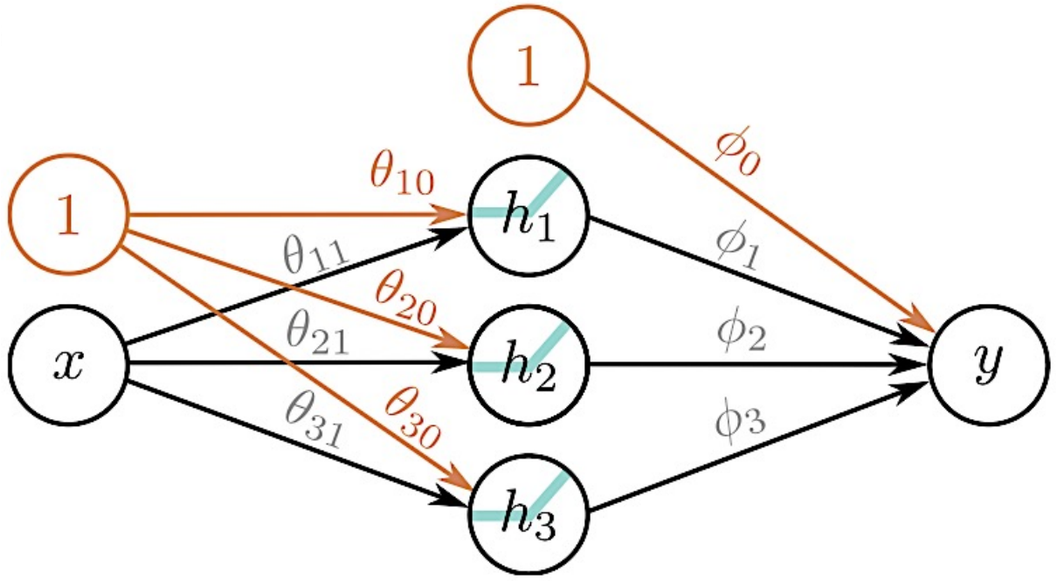






Shallow Neural Networks

- Consider a 2-layer network modeling a function: Real → Real
- Scalar input x
- Scalar output y
- Layer 1 modeled by weights & offsets $\theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}$
- Layer 2 modeled by weights & offsets $\phi_0, \phi_1, \phi_2, \phi_3$
- Activation function a(.) is ReLU



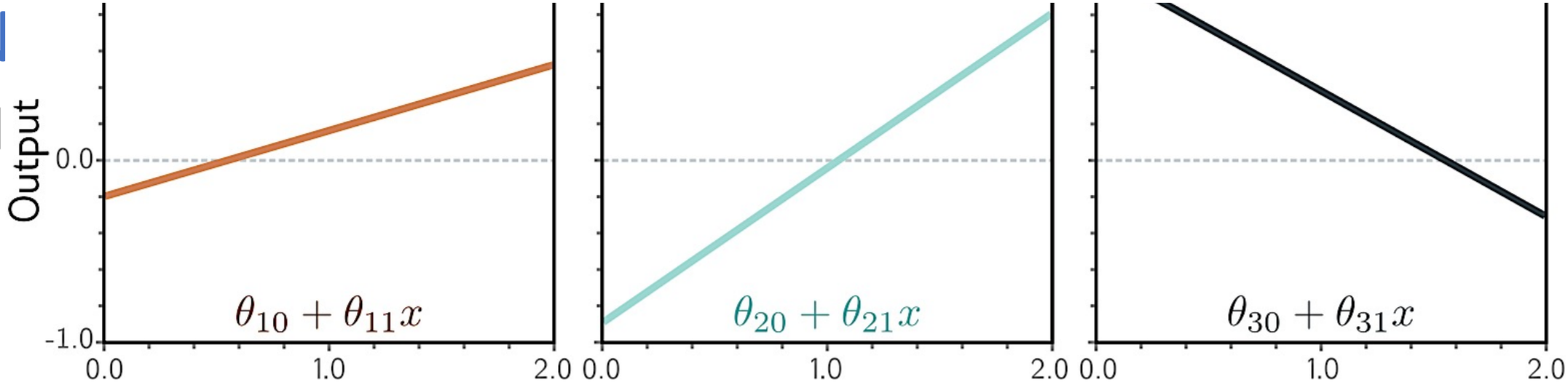
$$\mathbf{a}[z] = \mathbf{ReLU}[z] = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}$$

- Output

$$y = \mathbf{f}[x, \boldsymbol{\phi}]$$
$$= \phi_0 + \phi_1 \mathbf{a}[\theta_{10} + \theta_{11} x] + \phi_2 \mathbf{a}[\theta_{20} + \theta_{21} x] + \phi_3 \mathbf{a}[\theta_{30} + \theta_{31} x]$$

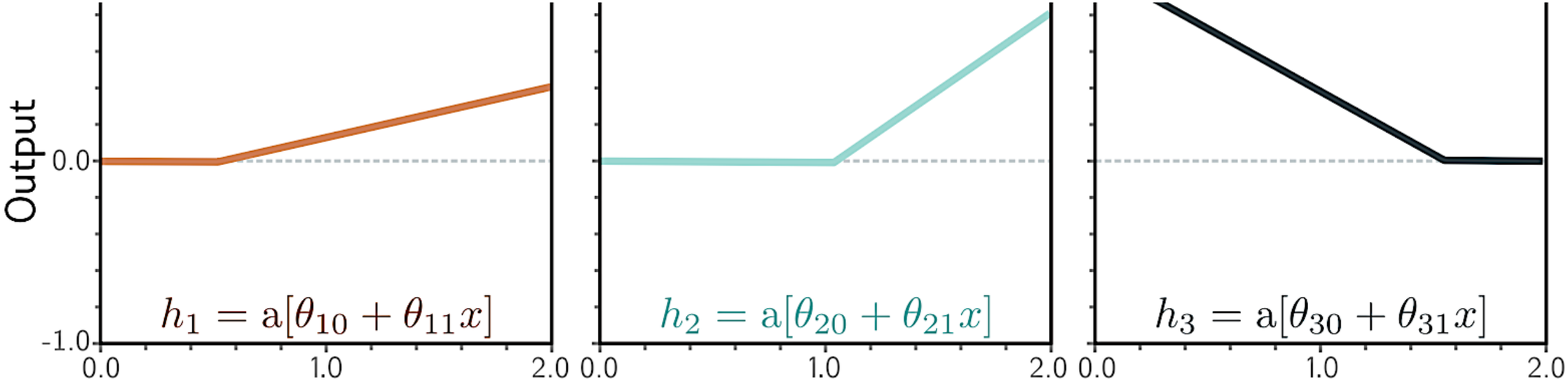
Shallow N

- Horizontal axis = function inputs



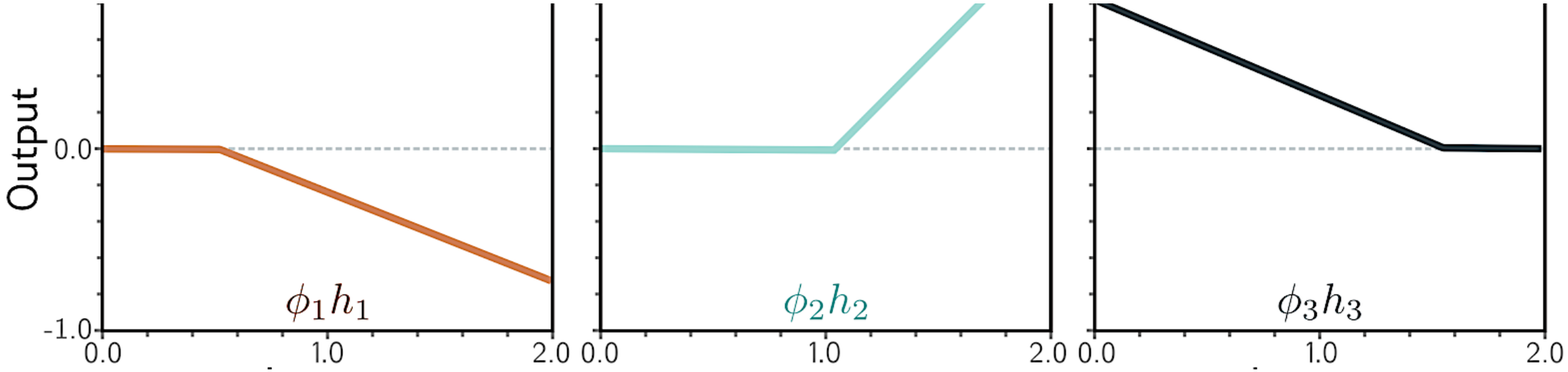
$$h_1 = a[\theta_{10} + \theta_{11}x]$$
$$h_2 = a[\theta_{20} + \theta_{21}x]$$
$$h_3 = a[\theta_{30} + \theta_{31}x]$$

- Model hidden units: where a(.) is ReLU

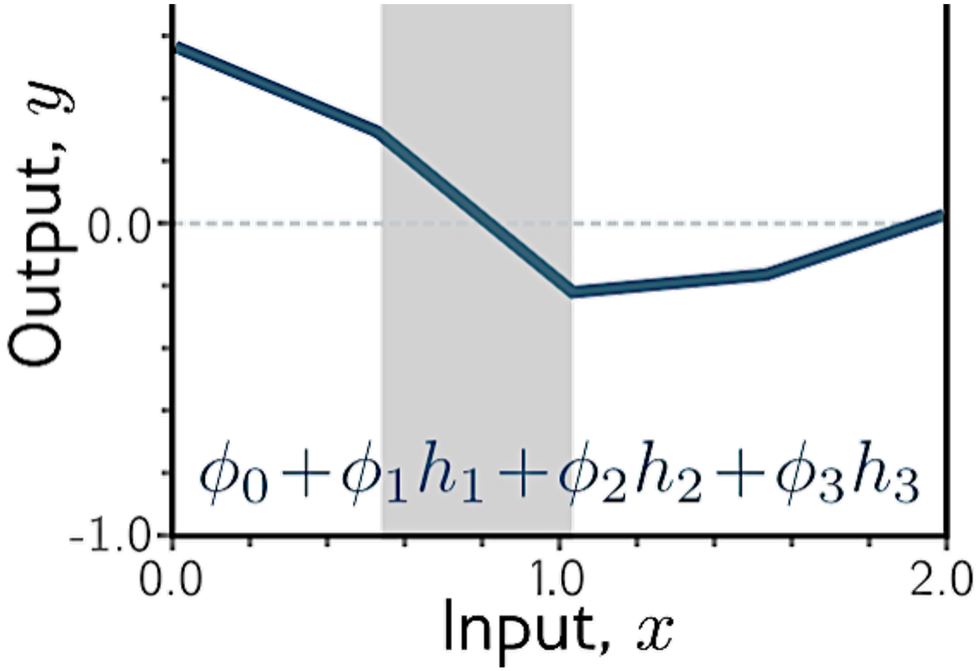


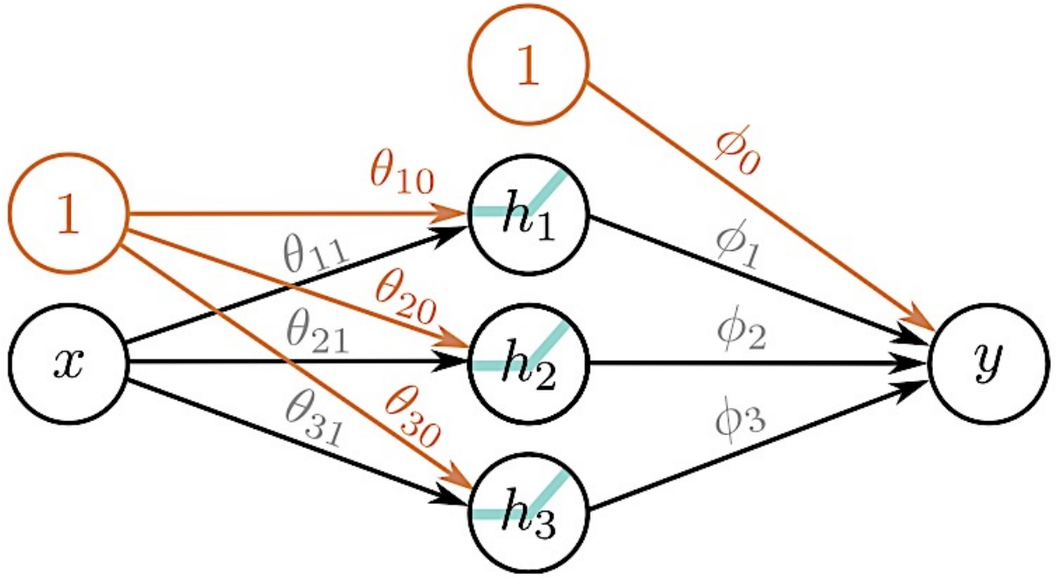
Shallow Neural Networks

- Model output $y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$



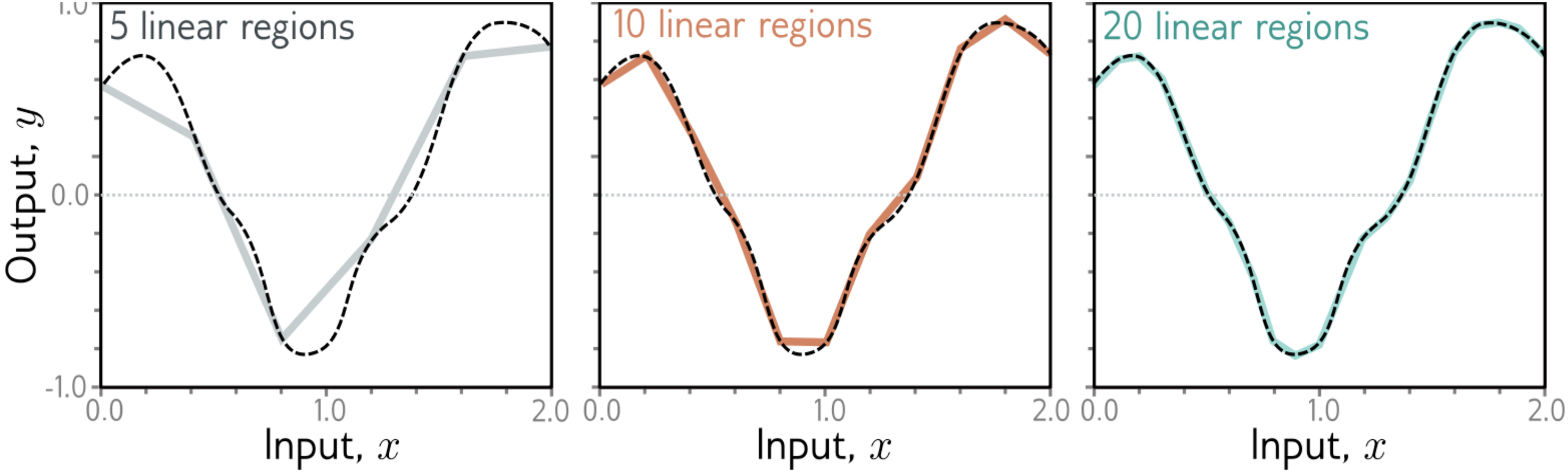
- Shaded region has h_1, h_3 "active"; h_2 "inactive"



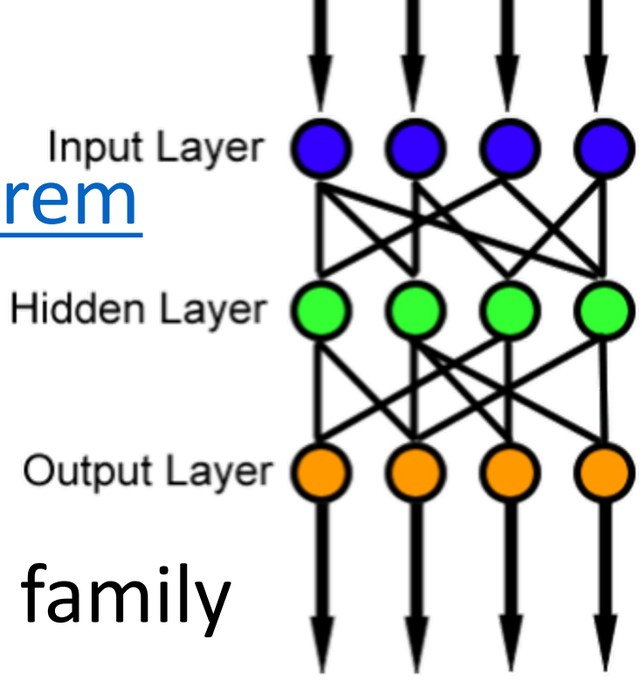


Shallow Neural Networks

- With ReLU activation, output of our 1-hidden-layer network with D>2 hidden units has at most D joints
- Output function is piecewise-linear with at most D+1 pieces
- So, by increasing number of hidden units, our 1-hidden-layer network can approximate more complex functions



Universal Approximation Theorem(s)



- en.wikipedia.org/wiki/Universal_approximation_theorem
- General form of theorems:
  Given a family of NNs,
  for each function f(.) from a certain space,
  there exists a sequence of NNs $NN_1(.)$, $NN_2(.)$, …, from family
  such that $\text{limit}_{T\to\infty}\ NN_T(.) \to f(.)$
  - i.e., that family of neural networks is "dense" in that function space
- One form of theorem is:
  Feedforward networks (i.e., NN with at least 1 hidden layer)
  with non-polynomial activation functions are dense
  in space of continuous functions between 2 Euclidean spaces
  - … with respect to "compact convergence" topology

Universal Approximation Theorem(s)

The Expressive Power of Neural Networks: A View from the Width

Neural Information Processing Systems (NeurIPS) 2017

Zhou Lu[1,3]
1400010739@pku.edu.cn

Hongming Pu[1]
1400010621@pku.edu.cn

Feicheng Wang[1,3]
1400010604@pku.edu.cn

Zhiqiang Hu[2]
huzq@pku.edu.cn

Liwei Wang[2,3]
wanglw@cis.pku.edu.cn

1, Department of Mathematics, Peking University
2, Key Laboratory of Machine Perception, MOE, School of EECS, Peking University
3, Center for Data Science, Peking University, Beijing Institute of Big Data Research

Deep Neural Networks (DNNs)

- Extend neural networks to use L>2 layers
- Layer l models:

$$\mathbf{z}^{(l)} = h^{(l)}\left(\mathbf{W}^{(l)}\mathbf{z}^{(l-1)}\right)$$

1) Layer input z^{l-1}
2) Weight matrix, including bias, W^l
3) Activation function's input = $W^l z^{l-1}$
4) Activation function = $h^l(\cdot)$
5) Activation function's output = "activation" = $z^l := h^l(W^l z^{l-1})$

- For layer 1: initial input $z^0 := x$. For layer L: final output := z^L
- "Initial" layers perform (focus more on) hierarchical/compositional representation/modeling of features: "representation learning"
- "Later" layers perform (focus more on) regressor-function learning

Deep Neural Networks (DNNs)

- Optimization using gradient descent: backpropagation
- x := input
- DNN output := g(x) := $h^L (W^L h^{L-1} (W^{L-1} \dots h^1 (W^1 x) \dots))$
- y := target
- Loss/cost: $C (y, g(x)) = C (y, h^L (W^L h^{L-1} (W^{L-1} \dots h^1 (W^1 x) \dots)))$
- Terms in derivative of loss (using chain rule) = (dC / dz^L) .
  $(dz^L / d[W^L z^{L-1}])$ $(d[W^L z^{L-1}] / dz^{L-1})$.
  $(dz^{L-1} / d[W^{L-1} z^{L-2}])$ $(d[W^{L-1} z^{L-2}] / dz^{L-2})$ …

1) Derivative of loss function $C (y, z^L)$ w.r.t. final activation z^L
2) Derivatives of activation functions h(.) w.r.t. their inputs
3) Derivatives of activation-function inputs w.r.t. previous activations

Deep Neural Network

- Optimization using gradient descent: Backpropagation
- Evaluates expression for derivative of cost function as a product of derivatives between each layer *from right to left* (“backwards”)
- Toolkits enable backpropagation automatically using “automatic differentiation”
- Toolkits also support various gradient-based **optimizers**, e.g.,
  - Stochastic gradient descent (SGD) [Rosenblatt 1958 Psychological Review]
  - Adam (Adaptive Moment Estimation) [Kingma & Ba 2015 ICLR]

Learning representations by back-propagating errors

David E. Rumelhart, Geoffrey E. Hinton & Ronald J. Williams

Nature **323**, 533–536 (1986) | Cite this article

Watching
a model train

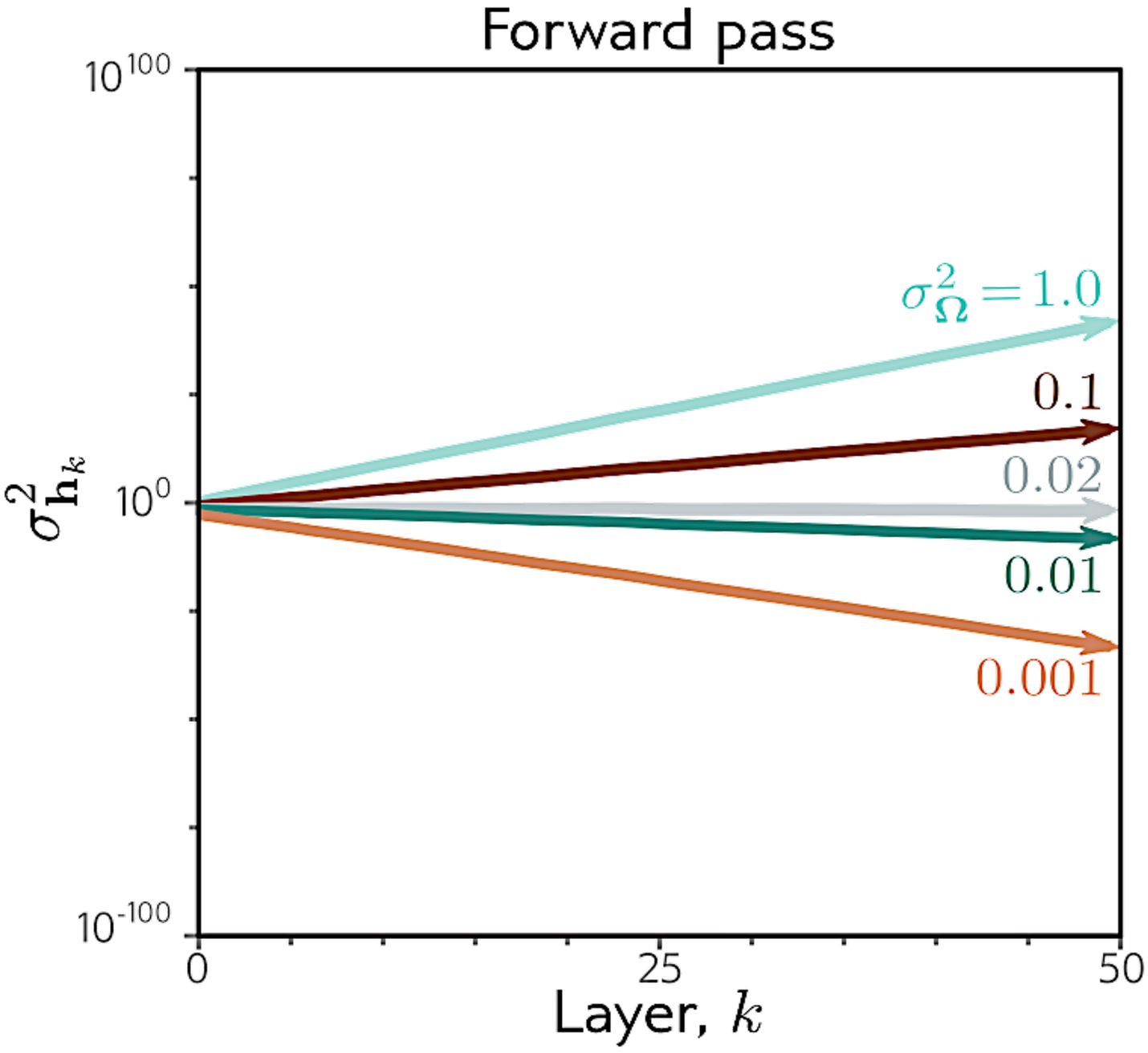
Watching
a model train

DNN: Parameter Initialization

- Consider initialization where biases → 0, weights ~ Gauss $(0,\sigma^2)$
- If σ^2 is tiny (e.g., 1e–5), then, in a forward pass, deeper-layer outputs may decay towards 0
  - Each linear layer does a weighted average with tiny weights
  - ReLu further cuts off some values with non-zero magnitudes
  - "Vanishing" gradient
- If σ^2 is huge (e.g., 1e+5), then, in a forward pass, deeper-layer outputs may grow towards ∞
  - "Exploding" gradient
- In both cases, finite-precision arithmetic may fail
- But vanishing/exploding gradients can occur even when σ^2 isn't too large and isn't too small → …

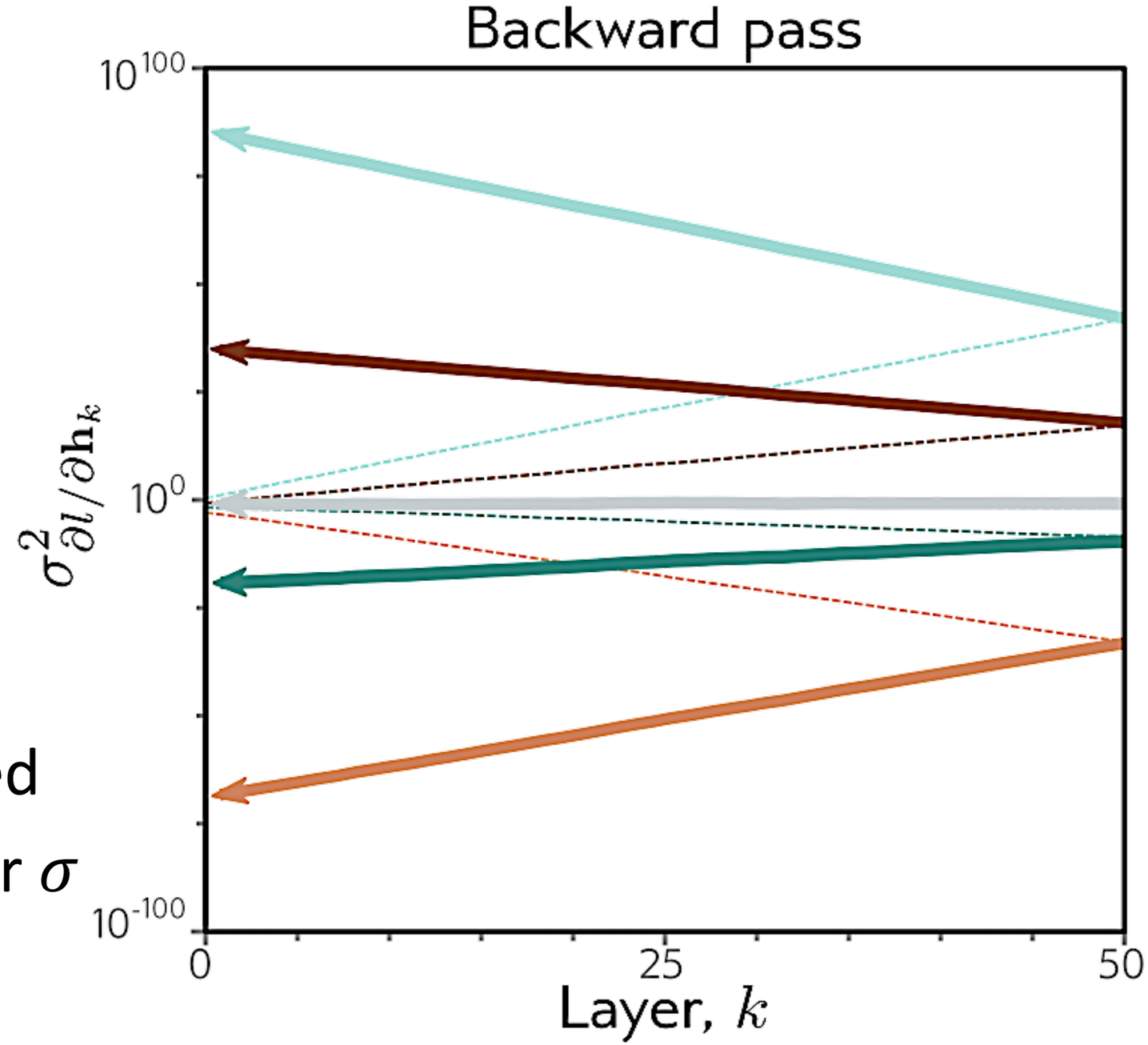
DNN: Parameter Initialization

- Consider initialization where biases → 0, weights ~ Gauss $(0,\sigma_\Omega^2)$
- Consider forward **pass**
- Vertical axis (log scale) := observed variance (in a DNN) of hidden-unit activations computed in forward pass
- Difficult to tune free-parameter σ in order to maintain stability



DNN: Parameter Initialization

- Consider initialization where biases → 0, weights ~ Gauss $(0,\sigma_\Omega^2)$
- Consider backward pass
- Vertical axis (log scale) := observed variance (in a DNN) computed in backward pass
- Problem persists in backward pass, where derivatives get multiplied
- Difficult to tune free-parameter σ in order to maintain stability



DNN: Parameter Initialization

- Better strategy by [Kaiming He et al. 2015 IEEE ICCV]
- Choose σ to initialize weights in some layer based on number of input units to that layer
- Insights:

1) Hidden unit with N inputs does weighted sum of N random variables
2) If variables independent, variances add up (scaled by |weight|)

Delving Deep into Rectifiers:
Surpassing Human-Level Performance on ImageNet Classification

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun
Microsoft Research

DNN: Parameter Initialization

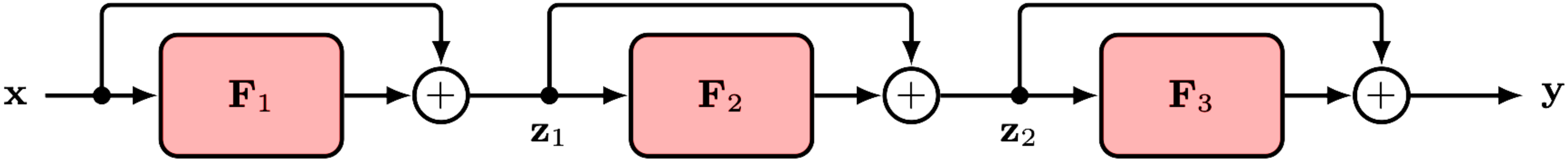
- Better strategy by [Kaiming He et al. 2015 IEEE ICCV]
- Assumptions (**linear** layer i)
  - Initial biases = 0
  - Initial weights (random variables) w_{ij} are independent, and 0-mean
  - Number of initial layer-inputs (random variables) $h_j = D$
  - Initial layer-inputs h_j independent of weights w_{ij}
- Then Mean (linear-layer output i) = $E[\sum_j w_{ij} h_j] = 0$
- Then Var (layer output) = $Var(\sum_j w_{ij} h_j) = E[(\sum_j w_{ij} h_j)^2] = \sum_j E[(w_{ij})^2 (h_j)^2]$
- $= \sum_j E[(w_{ij})^2] E[(h_j)^2] = \sigma^2 \sum_j E[(h_j)^2] = \sigma^2 D \, Var(h) / 2$
  - /2 if we assume half of initial layer inputs are already clipped to 0 by ReLU
- So, to keep Var (layer input) = Var (layer output),
  set, for layer i, $\sigma^2 := 2/D$

DNN: Parameter Initialization

- Better strategy by [Kaiming He et al. 2015 IEEE ICCV]
- Similar argument for backward pass
- Gradient w.r.t. unit is a
  linear combination of gradients w.r.t. units h_j in next layer
- Typically, number of units in subsequent layers changes;
  So, set for layer i, $\sigma^2 := 2 / (D_i + D_{i+1})$

DNN: Residual Connection

- "Residual" connection learns residual between input & desired output
  - A kind of a "skip" connection



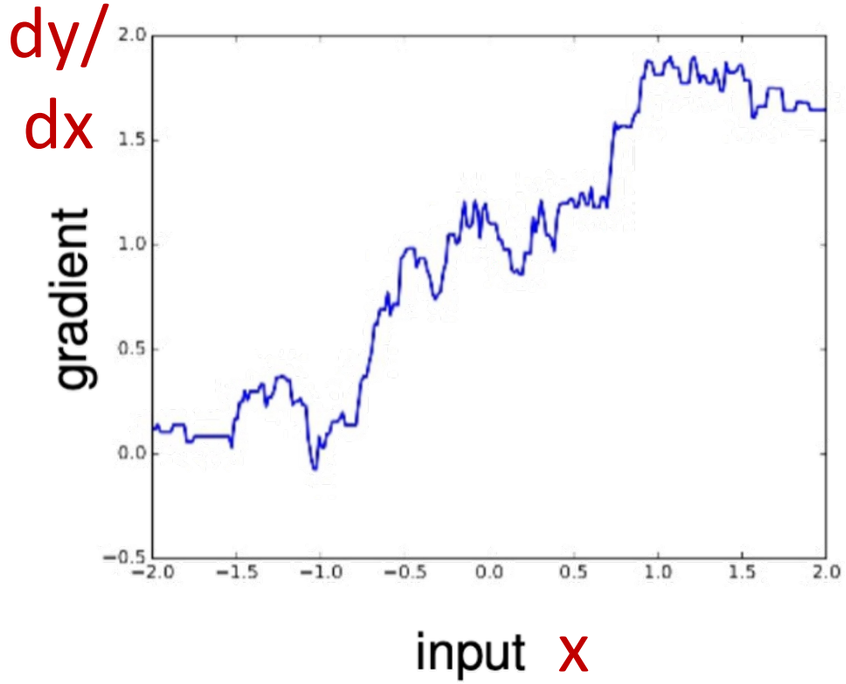
$$\mathbf{z}_1 = \mathbf{F}_1(\mathbf{x}) + \mathbf{x}$$

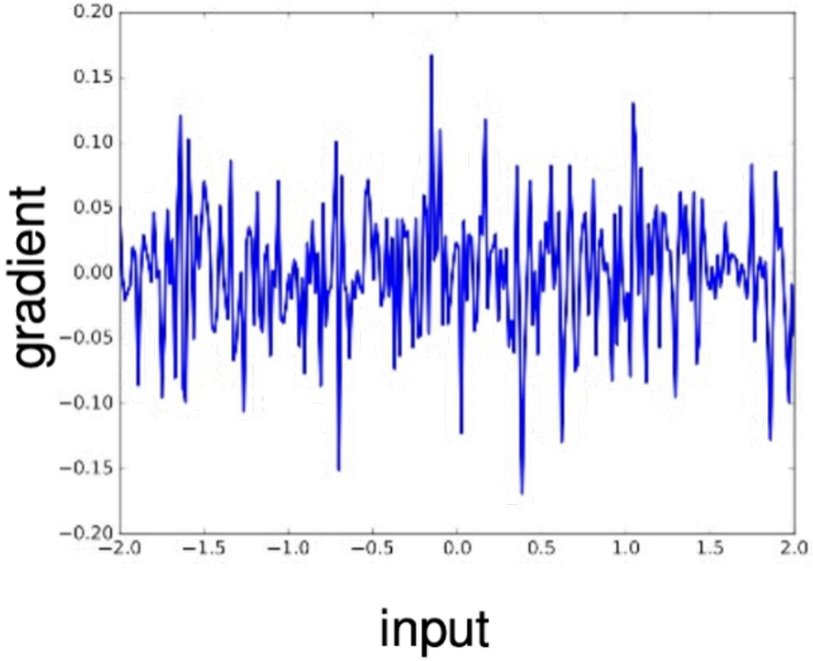
$$\mathbf{z}_2 = \mathbf{F}_2(\mathbf{z}_1) + \mathbf{z}_1$$

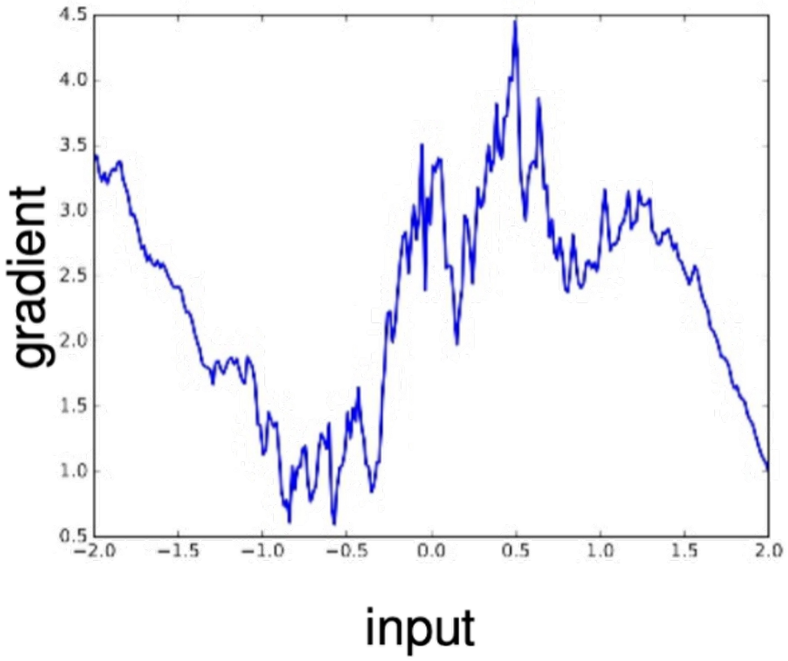
$$\mathbf{y} = \mathbf{F}_3(\mathbf{z}_2) + \mathbf{z}_2$$

DNN: Residual Connection

- Why residual ? For deep NNs (especially using ReLu), tiny changes in input (or early-layer weights) can lead to significant changes in gradient
- **Left**: **1**-hidden-layer feedforward NN
- **Middle**: **24**-hidden-layer feedforward DNN
- **Right**: **50**-hidden-layer DNN with residual connections (ResNet)







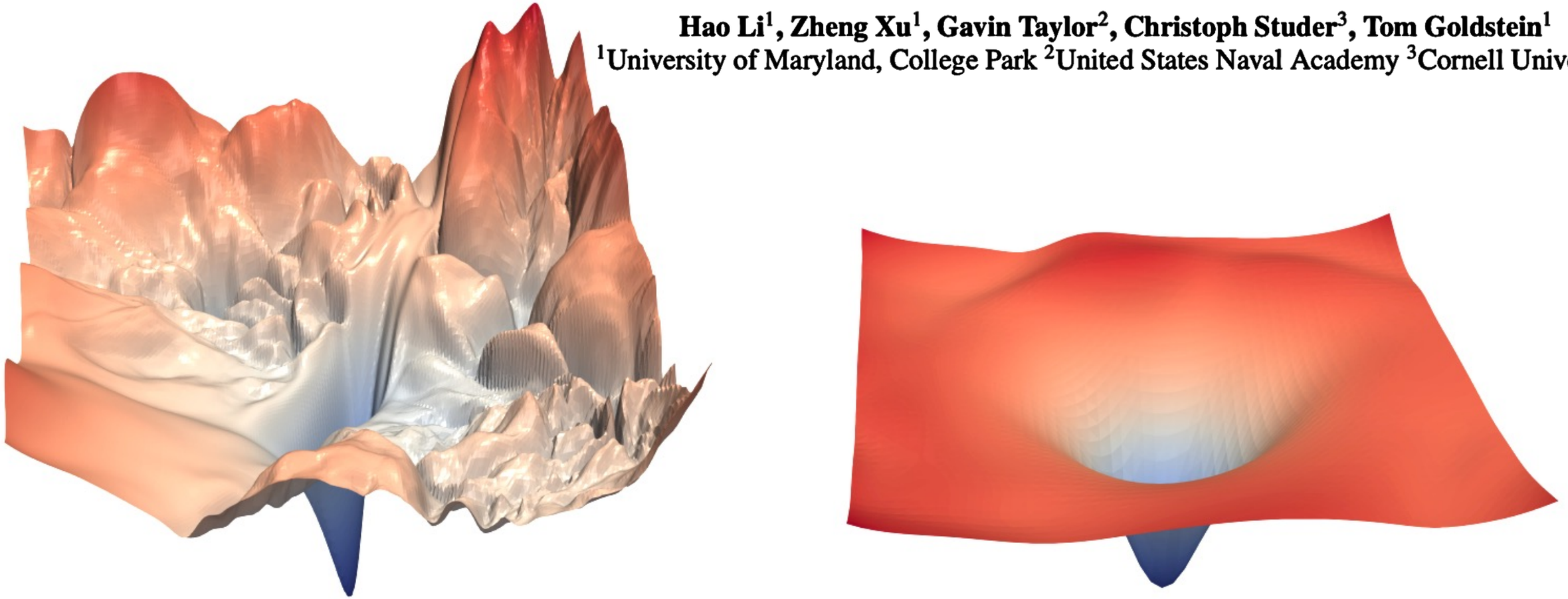
DNN: Residual Connecti

- Loss, in terms of weights w

Visualizing the Loss Landscape of Neural Nets

-Neural Information Processing Systems (NeurIPS 2018)-

Hao Li[1], Zheng Xu[1], Gavin Taylor[2], Christoph Studer[3], Tom Goldstein[1]
[1]University of Maryland, College Park [2]United States Naval Academy [3]Cornell University



(a) without skip connections

(b) with skip connections

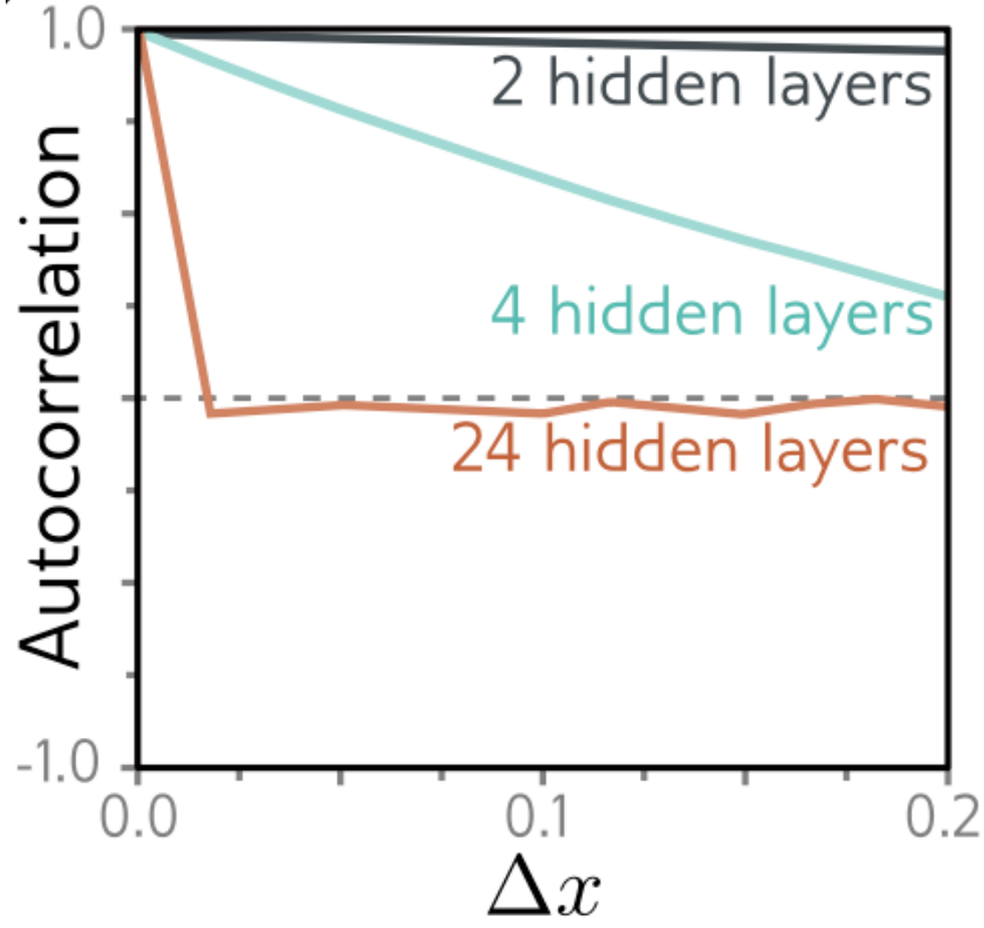
Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

DNN: Residual Connection

- In shallow NNs, gradients of loss function evaluated at nearby inputs or weights (x & x+Δx, or w & w+Δw) are correlated
- As NNs get deeper, this correlation quickly drops to 0 → "shattered gradients"
  - Paper calls it "auto"correlation

$$R_{ff}(\tau) = \int_{-\infty}^{\infty} f(t+\tau)\overline{f(t)}\,\mathrm{d}t$$

- This makes gradient-descent trajectories non-smooth



The Shattered Gradients Problem:
If resnets are the answer, then what is the question?

Int. Conf. Machine Learning 2017

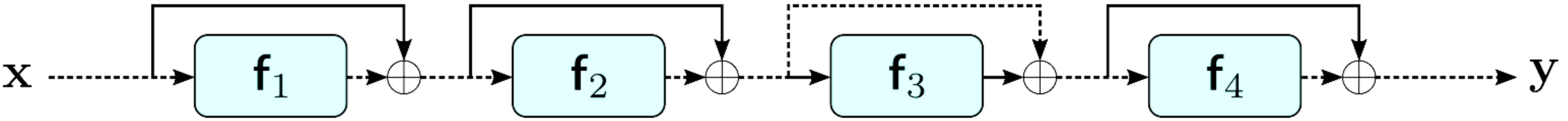
David Balduzzi [1] Marcus Frean [1] Lennox Leary [1] JP Lewis [1 2] Kurt Wan-Duo Ma [1] Brian McWilliams [3]

DNN: Residual Connection

- "Shattered gradients" [Balduzzi et al. 2017 ICML]
- Let DNN model $y := f_3(f_2(f_1(x)))$
- Change in y w.r.t. change in x: $dy/dx = dy/df_3 \cdot df_3/df_2 \cdot df_2/df_1 \cdot df_1/dx$
- Derivative of current output y' evaluated at current x' uses derivatives of/at function values
  $f_3(f_2(f_1(x'))), f_2(f_1(x')), f_1(x')), x'$
- Derivative of current output y' evaluated at current x'+Δx uses derivatives of/at function values
  $f_3(f_2(f_1(x'+\Delta x))), f_2(f_1(x+\Delta x)), f_1(x+\Delta x)), x'+\Delta x$
- Small change Δx may drastically change values in DNN's deeper layers
- Problem in gradient descent

DNN: Residual Connection

- Let a DNN model residual connections as:



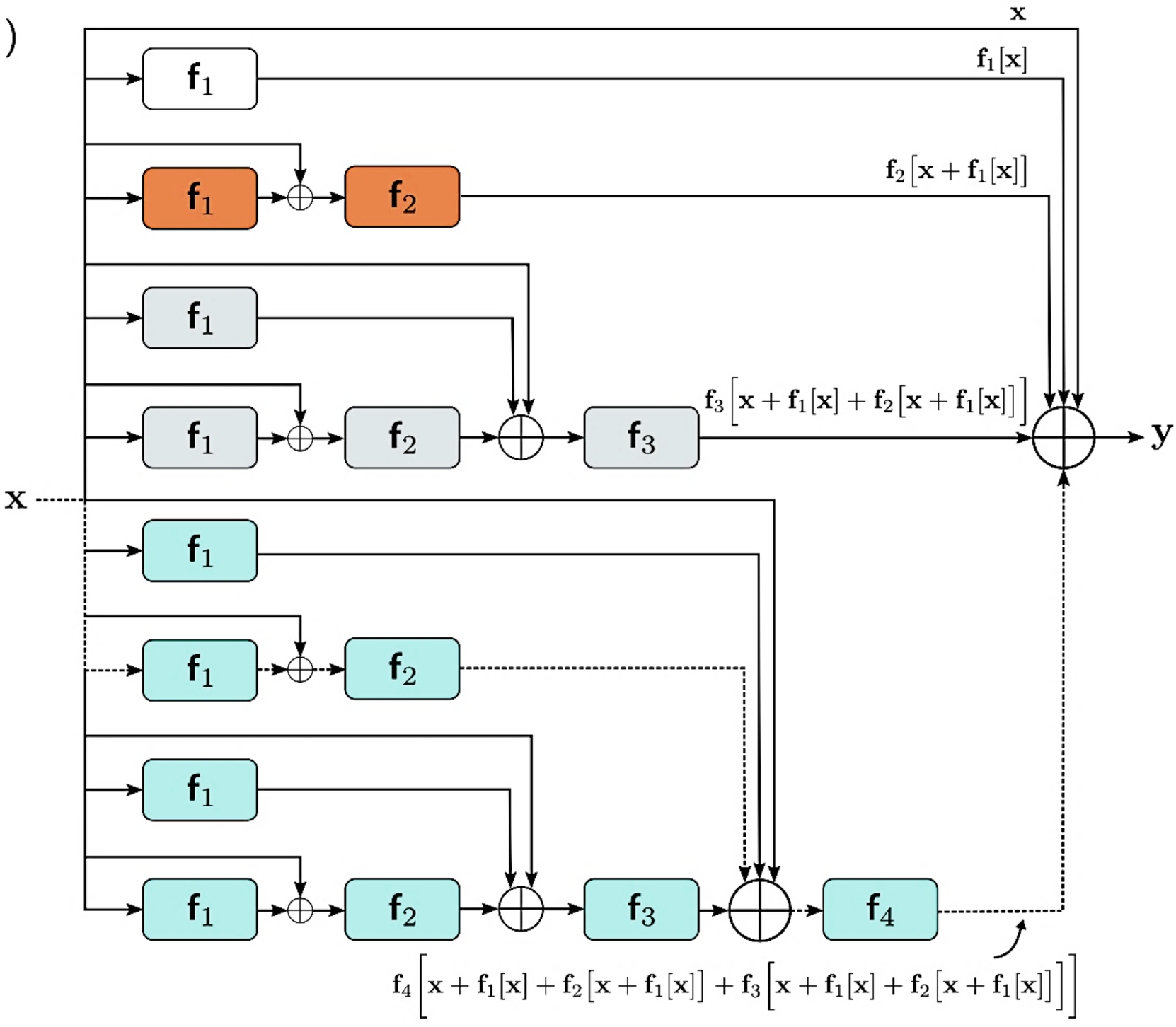
- This DNN can be unraveled/unrolled as:

$$\mathbf{h}_1 = \mathbf{x} + \mathbf{f}_1[\mathbf{x}, \boldsymbol{\phi}_1]$$
$$\mathbf{h}_2 = \mathbf{h}_1 + \mathbf{f}_2[\mathbf{h}_1, \boldsymbol{\phi}_2]$$
$$\mathbf{h}_3 = \mathbf{h}_2 + \mathbf{f}_3[\mathbf{h}_2, \boldsymbol{\phi}_3]$$
$$\mathbf{y} = \mathbf{h}_3 + \mathbf{f}_4[\mathbf{h}_3, \boldsymbol{\phi}_4]$$

$$\mathbf{y} = \mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]] + \mathbf{f}_3[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]]] + \mathbf{f}_4[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]] + \mathbf{f}_3[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]]]]$$

DNN: Residual Conne)

- Unraveled DNN output
  = input +
  output of 4 smaller NNs
- Residual connections
  convert DNN into
  ensemble of
  smaller NNs
  whose
  outputs are summed
  to get DNN's output
- Creates 16 paths
  between input & output



DNN: Residual Connection

- Change in 1st layer's output $f_1(x)$ directly adds into change in output y
  - Of course, it also contributes indirectly through $f_2(.)$, $f_3(.)$, etc.
- Gradients through shorter paths (less nonlinearity) → better behaved
- Reduces problem of shattered gradients (smoother energy landscape)

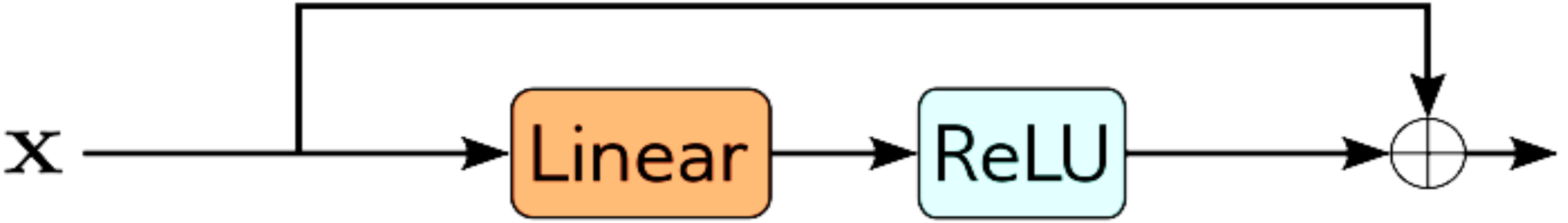
$$\frac{\partial \mathbf{y}}{\partial \mathbf{f}_1} = \mathbf{I} + \frac{\partial \mathbf{f}_2}{\partial \mathbf{f}_1} + \left(\frac{\partial \mathbf{f}_3}{\partial \mathbf{f}_1} + \frac{\partial \mathbf{f}_2}{\partial \mathbf{f}_1}\frac{\partial \mathbf{f}_3}{\partial \mathbf{f}_2}\right) + \left(\frac{\partial \mathbf{f}_4}{\partial \mathbf{f}_1} + \frac{\partial \mathbf{f}_2}{\partial \mathbf{f}_1}\frac{\partial \mathbf{f}_4}{\partial \mathbf{f}_2} + \frac{\partial \mathbf{f}_3}{\partial \mathbf{f}_1}\frac{\partial \mathbf{f}_4}{\partial \mathbf{f}_3} + \frac{\partial \mathbf{f}_2}{\partial \mathbf{f}_1}\frac{\partial \mathbf{f}_3}{\partial \mathbf{f}_2}\frac{\partial \mathbf{f}_4}{\partial \mathbf{f}_3}\right)$$

$$\begin{aligned}
\mathbf{y} = \mathbf{x} &+ \mathbf{f}_1[\mathbf{x}] \\
&+ \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]] \\
&+ \mathbf{f}_3[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]]] \\
&+ \mathbf{f}_4[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]] + \mathbf{f}_3[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]]]]
\end{aligned}$$

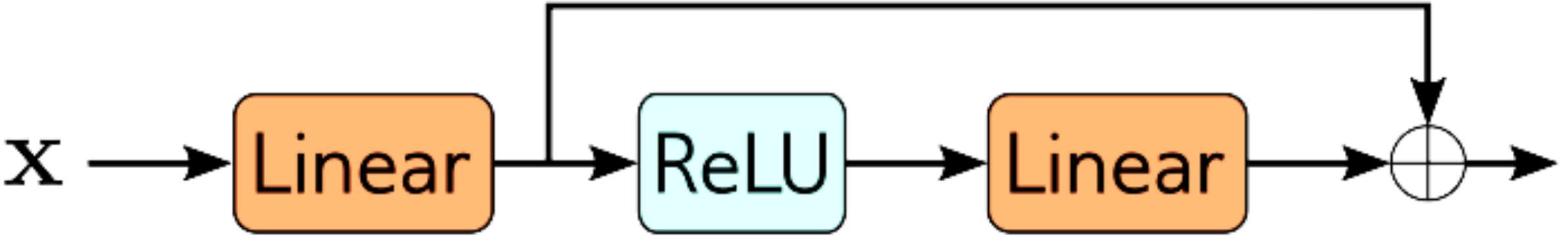
DNN: Residual Connection

- Designing a residual DNN (guidelines)

1) Following design only allows positive additions to x (in every block)



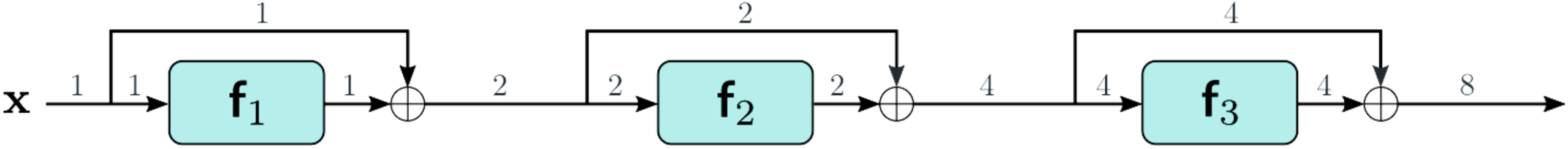
2) Following design allows both positive and negative “additions” to x



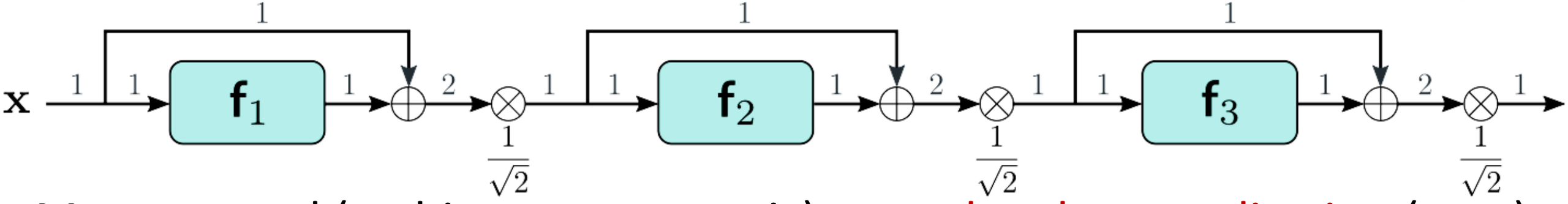
3) Starting with ReLU won’t do anything if input $x < 0$

DNN: Vanishing/Exploding Gradients in Residual DNNs

1) Residual DNN doesn't have to worry about vanishing gradients, because direct path exists from each layer to DNN output
2) In residual DNN, because layer input is added back to layer output, (variance of) values in deeper layers doubles each time (exp growth)



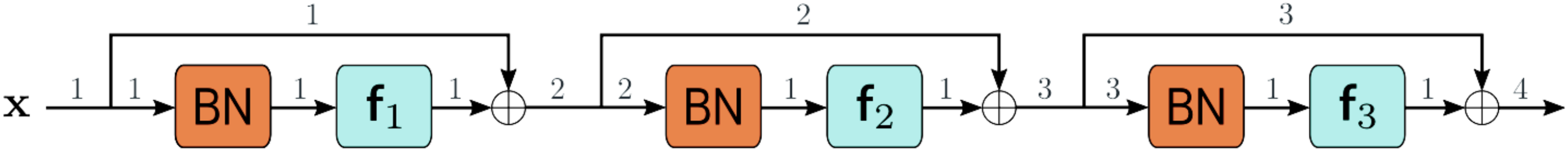
- Kaiming's initialization needs to be modified:
  To get stability, we can scale down output of residual unit by sqrt(2)



- More general (architecture-agnostic) way = batch normalization (next)

DNN: Batch Normalization

- During <u>training</u>, at a DNN layer, batch normalization shifts & rescales activation of each unit/layer such that their mean & standard deviation (across examples in batch) are δ & γ
  - Parameters δ & γ can be fixed to 0 and 1, during training
  - BatchNorm aims to controls gradients, especially during initial training phases



- During <u>testing</u>, because we don't have a batch, we can compute δ & γ from entire training set and freeze them; [or learn a scaling factor and a shifting offset during training]
- In convolutional DNN, normalization is done a bit differently: mean & variance computed over batch AND spatial location together

DNN: Other Normalization Schemes

- 2018 Euro. Conf. Computer Vision
  - Tensor dimension 1: Layer size HxW
  - Tensor dimension 2: Num of channels C
  - Tensor dimension 3: Batch size N

Group Normalization

Yuxin Wu Kaiming He

Facebook AI Research (FAIR)

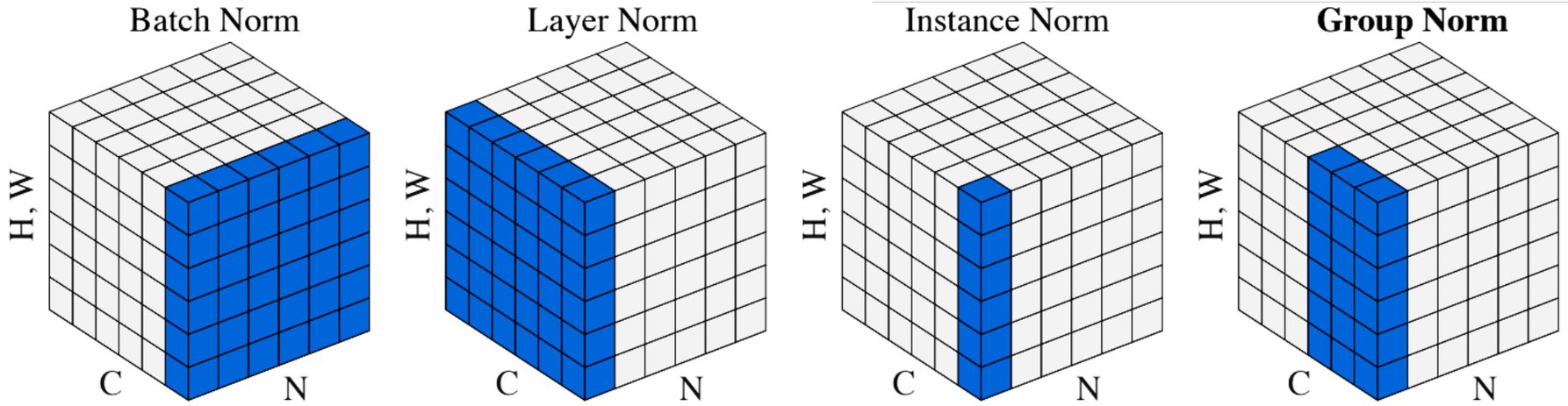


Figure 2. Normalization methods. Each subplot shows a feature map tensor. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels. Group Norm is illustrated using a group number of 2.

Convolutional Deep Neural Network (CNN)

- **Motivation: classification**
- We desire invariance to translation
- e.g., a DNN **classifier f(.)** should classify both images to the “mountain” class

$$\mathbf{f[t[x]] = f[x]}$$

CNN

- **Motivation: segmentati**
- We desire "equivariance"/ "covariance" to translation
- e.g., when objects in input image are translated, detected segments should translate in same way

$$\mathbf{f}[\mathbf{t}[\mathbf{x}]] = \mathbf{t}[\mathbf{f}[\mathbf{x}]]$$

CNN

- Convolution is “equivariant” to translation
  → helps semantic segmentation

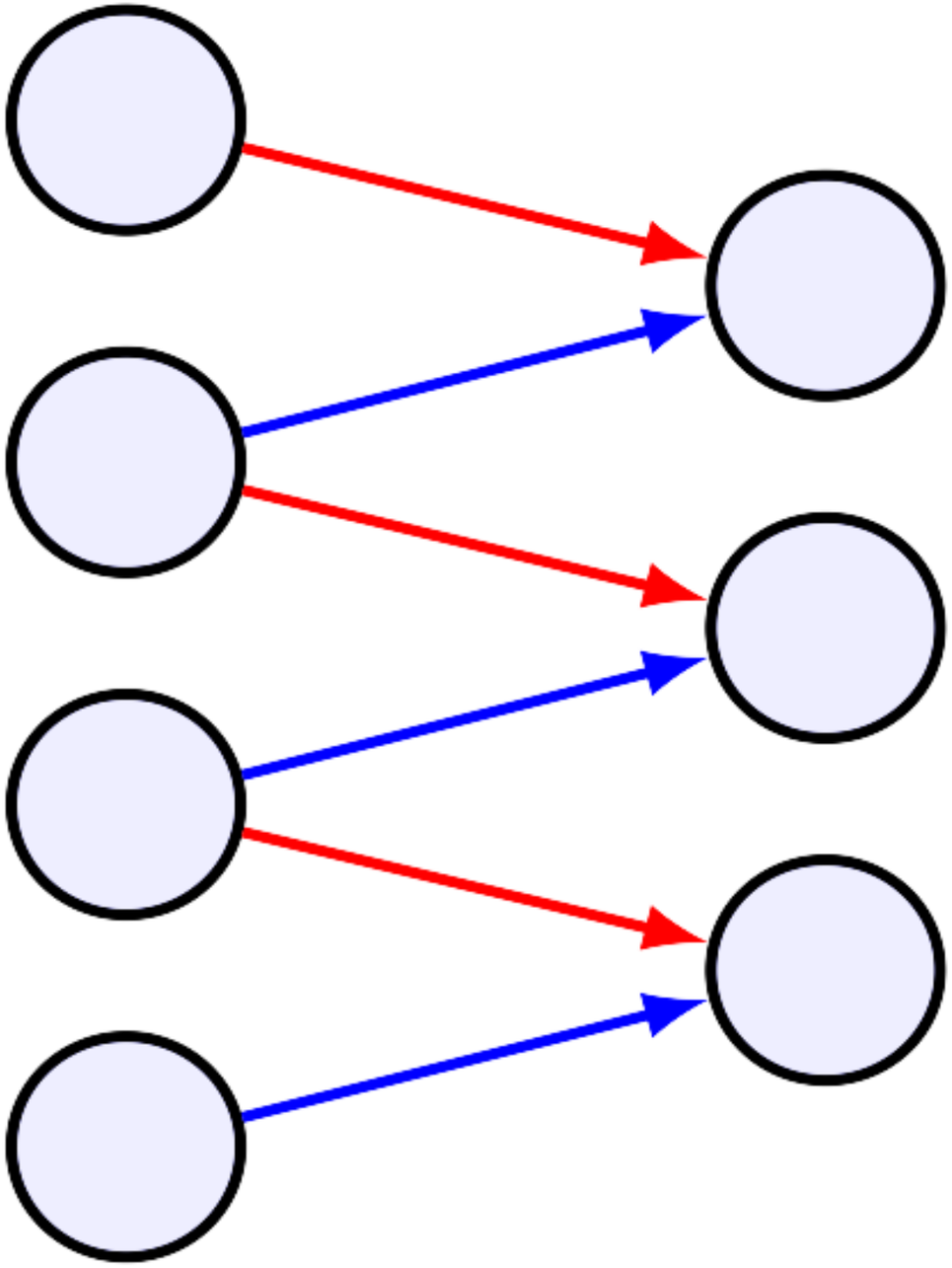
- Convolution and pooling (discussed later)
  together help classifier with “invariance”

- Goal:
  Design CNN such that convolution can detect
  fine-scale local features (e.g., edges) through initial layers,
  coarser-scale less-local features (e.g., patterns of edges) at later layers,
  etc.

CNN

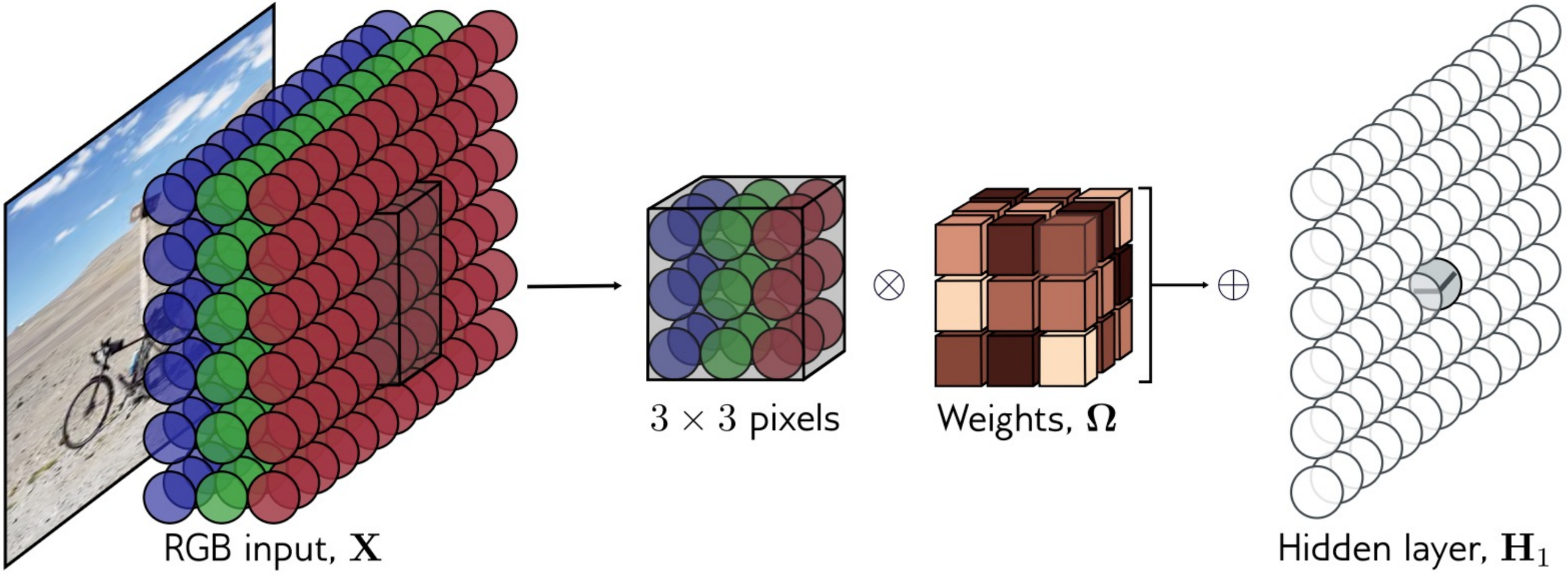
- A convolution layer (here, kernel size 2x1)

1) Far fewer connections
   (in this case, 6)
   compared to
   that in fully-connected layer, i.e.,
   (# nodes in layer 1) x (# nodes in layer 2)
2) Moreover, far fewer parameters
   (in this case, 2)
   compared to number of connections
   (in this case, 6)
   - In fully-connected layer,
     # parameters = # connections



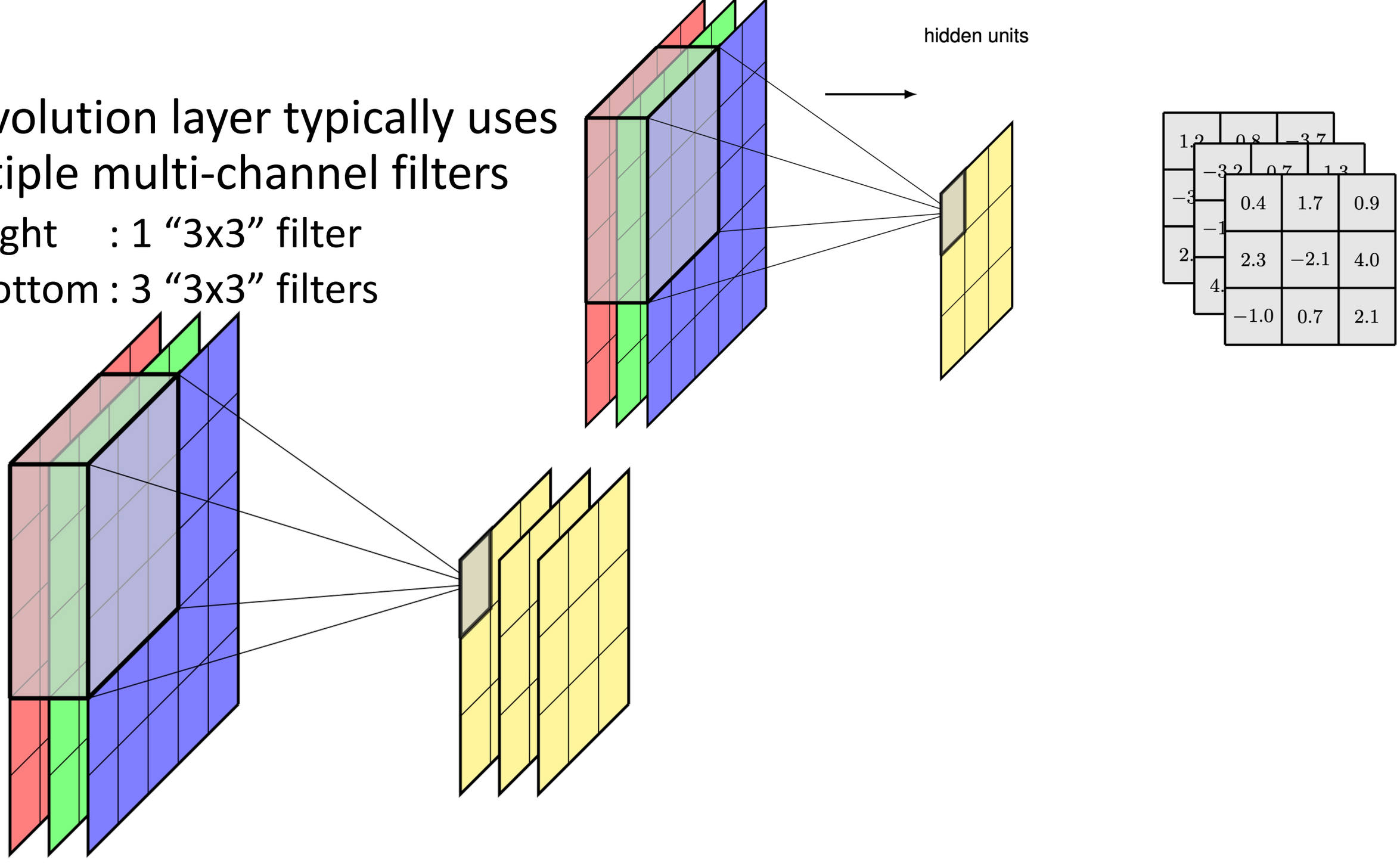
CNN

- Convolution for multi-“channel” (e.g., RGB) input
- Parameters: convolution kernel size = 3x3x3; often called “3x3” filter
- Parameters: 1 (bias)



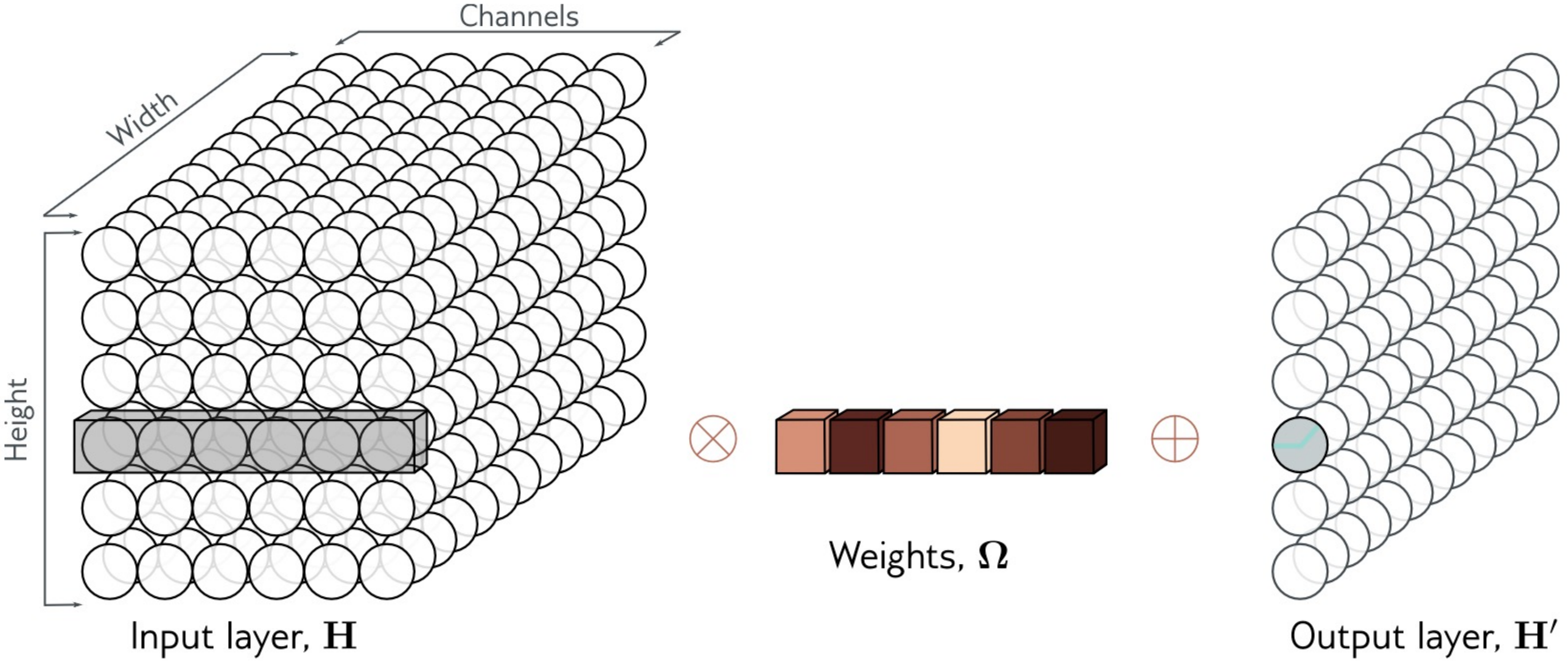
CNN

- Convolution layer typically uses multiple multi-channel filters
  - Right : 1 “3x3” filter
  - Bottom : 3 “3x3” filters



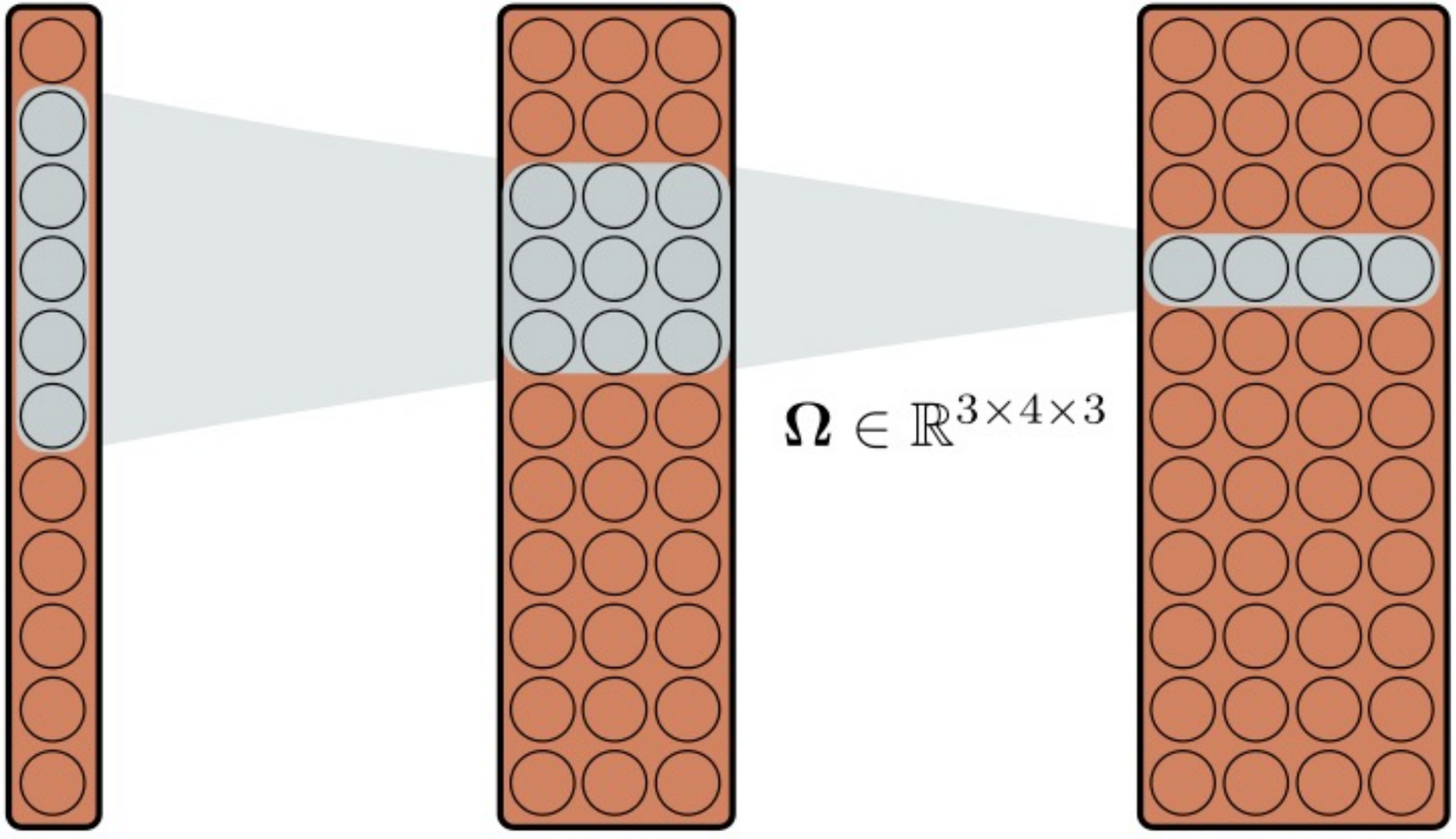
CNN

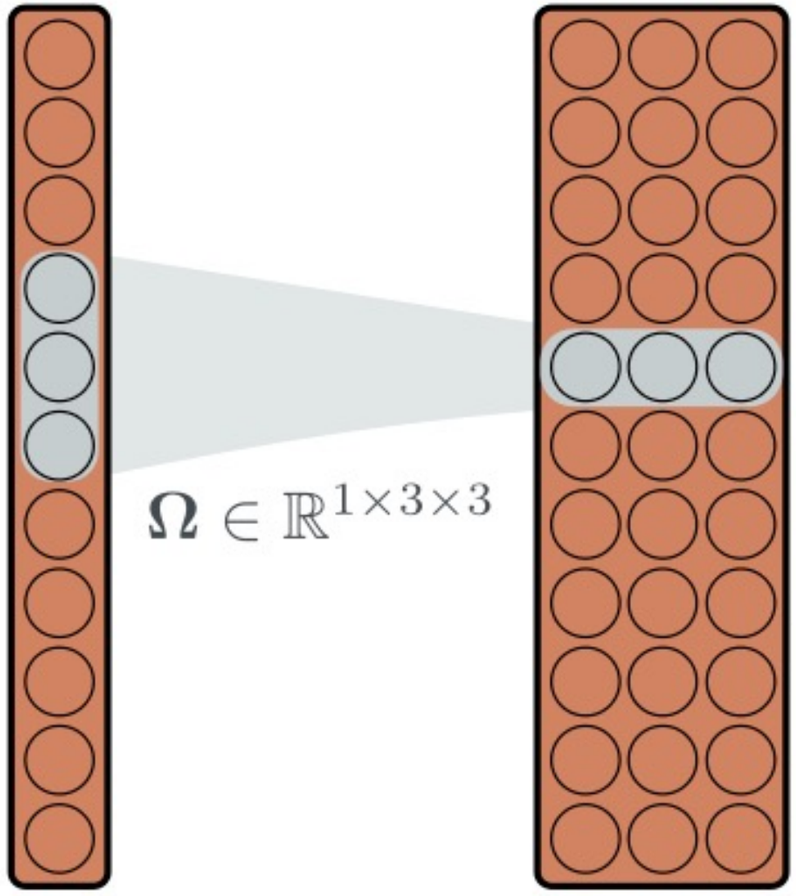
- So-called “1x1” convolution uses kernel size = 1 x 1 x NumOfChannels



CNN

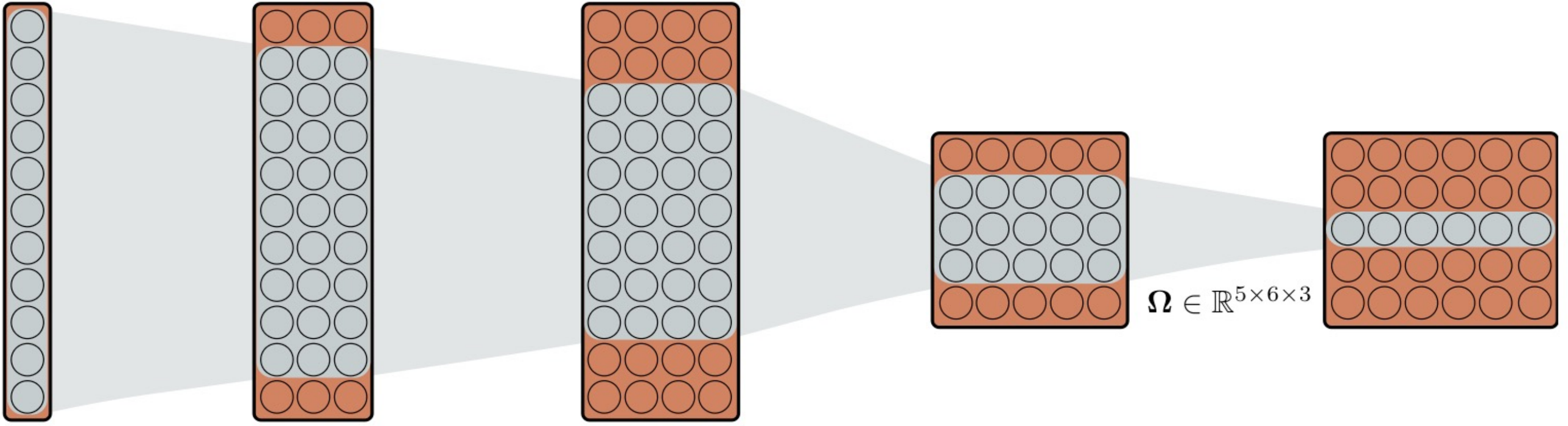
- "Receptive field" of a hidden unit in DNN := region of input image that affects unit's output
- Receptive field of units in deeper layers is typically larger





CNN

- Receptive field of a hidden unit in a DNN :=
  region of input image that affects unit's output
- Receptive field of units in deeper layers is typically larger



CNN

1) Convolution layer encodes translation equivariance

- If a small patch of pixels,
  representing receptive field of a hidden unit,
  is moved to a different location in image,
  then associated outputs of convolution will
  move to corresponding location in feature map

2) We want CNN to be able to learn hierarchical representations
   in which
   complex features at a particular level
   are built up from simpler features at previous level

- Sequence of convolution layers

CNN

- A simple dataset
- MNIST-1D
- 2024 ICML

Scaling Down Deep Learning with MNIST-1D

Samuel James Greydanus, Dmitry Kobak *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235:16404-16415, 2024.

Abstract

Although deep learning models have taken on commercial and political relevance, key aspects of their training and operation remain poorly understood. This has sparked interest in science of deep learning projects, many of which require large amounts of time, money, and electricity. But how much of this research really needs to occur at scale? In this paper, we introduce MNIST-1D: a minimalist, procedurally generated, low-memory, and low-compute alternative to classic deep learning benchmarks. Although the dimensionality of MNIST-1D is only 40 and its default training set size only 4000, MNIST-1D can be used to study inductive biases of different deep architectures, find lottery tickets, observe deep double descent, metalearn an activation function, and demonstrate guillotine regularization in self-supervised learning. All these experiments can be conducted on a GPU or often even on a CPU within minutes, allowing for fast prototyping, educational use cases, and cutting-edge research on a low budget.

CNN

- MNIST-1D
- 2024 ICML
- Scaling Down Deep Learning with MNIST-1D

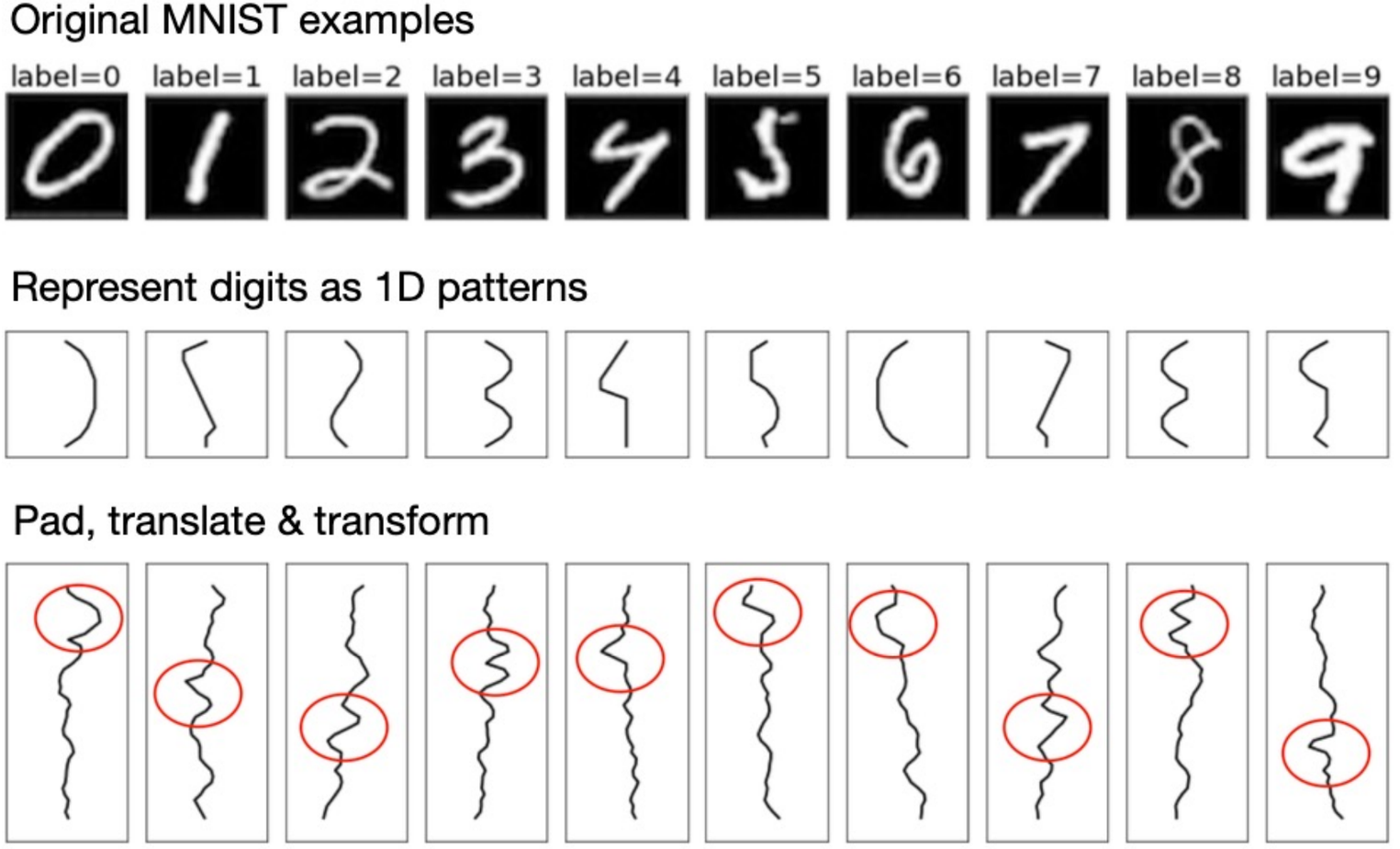
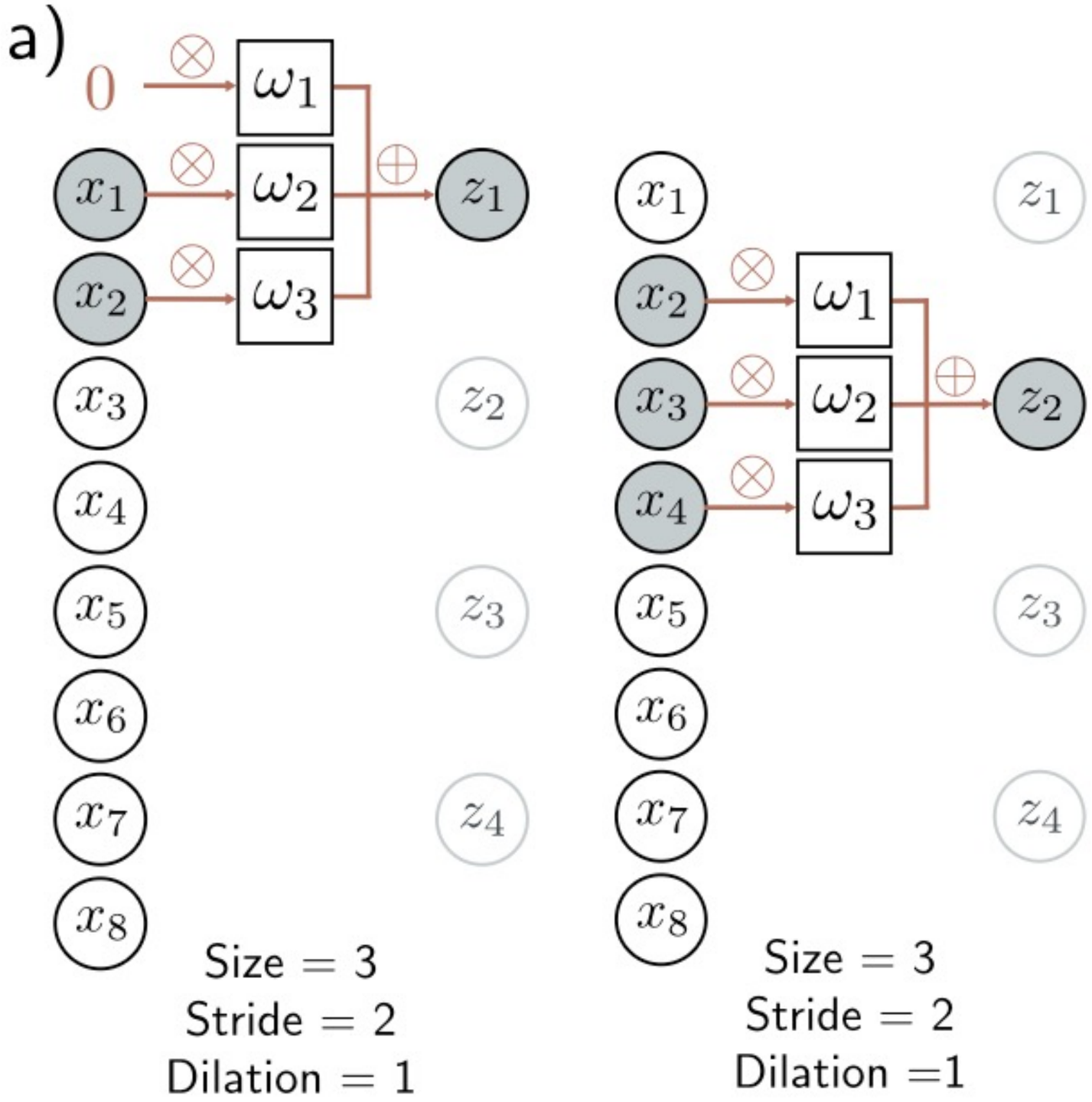


Figure 1: Constructing the MNIST-1D dataset. Unlike MNIST, each sample is a one-dimensional sequence. To generate each sample, we begin with a hand-crafted digit template loosely inspired by MNIST shapes. Then we randomly pad, translate, and add noise to produce 1D sequences with 40 points each. [CODE]

CNN

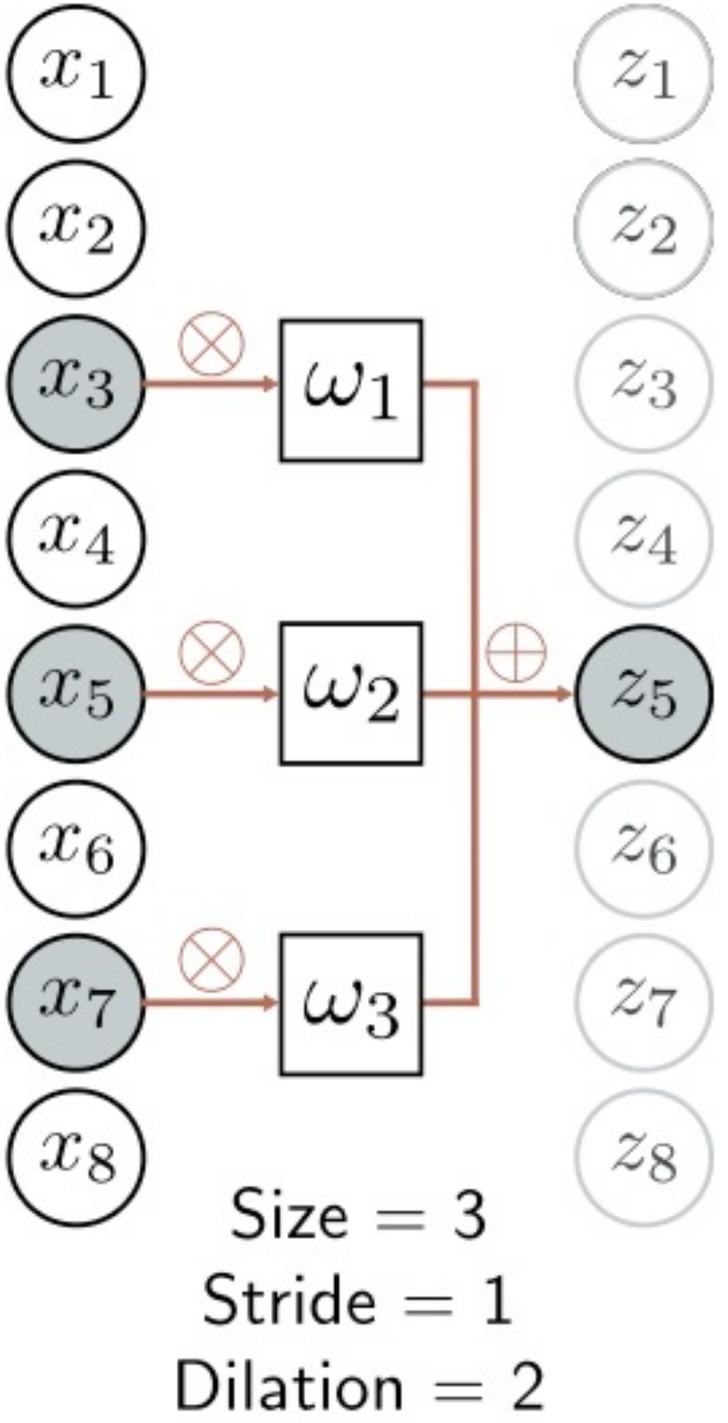
- "Stride" = 2

1) Zero padding used to handle boundary condition
2) Another option could be to use "valid" convolution outputs → smaller output



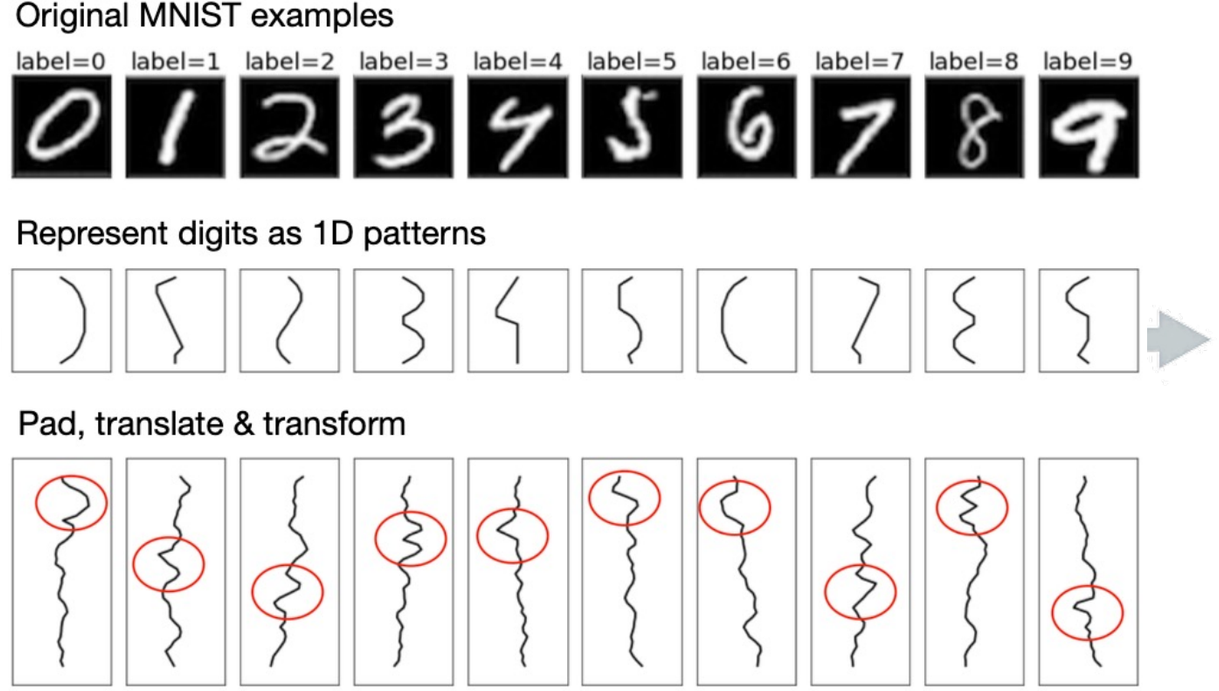
CNN

- “Dilation” = 2



CNN

- Classify MNIST-1D data
  - “valid” convolution output used



$\mathbf{x} \in \mathbb{R}^{40}$

$\mathbf{H}_1 \in \mathbb{R}^{19\times 15}$

Convolution Size=3, Stride=2, Channels=15

$\mathbf{H}_2 \in \mathbb{R}^{9\times 15}$

Convolution Size=3, Stride=2, Channels=15

$\mathbf{H}_3 \in \mathbb{R}^{4\times 15}$

Convolution Size=3, Stride=2, Channels=15

Fully connected Input=60, Output=10

$\mathbf{f} \in \mathbb{R}^{10}$

Softmax

$\boldsymbol{\Omega}_0 \in \mathbb{R}^{1\times 15\times 3}$
$\boldsymbol{\beta}_0 \in \mathbb{R}^{15}$

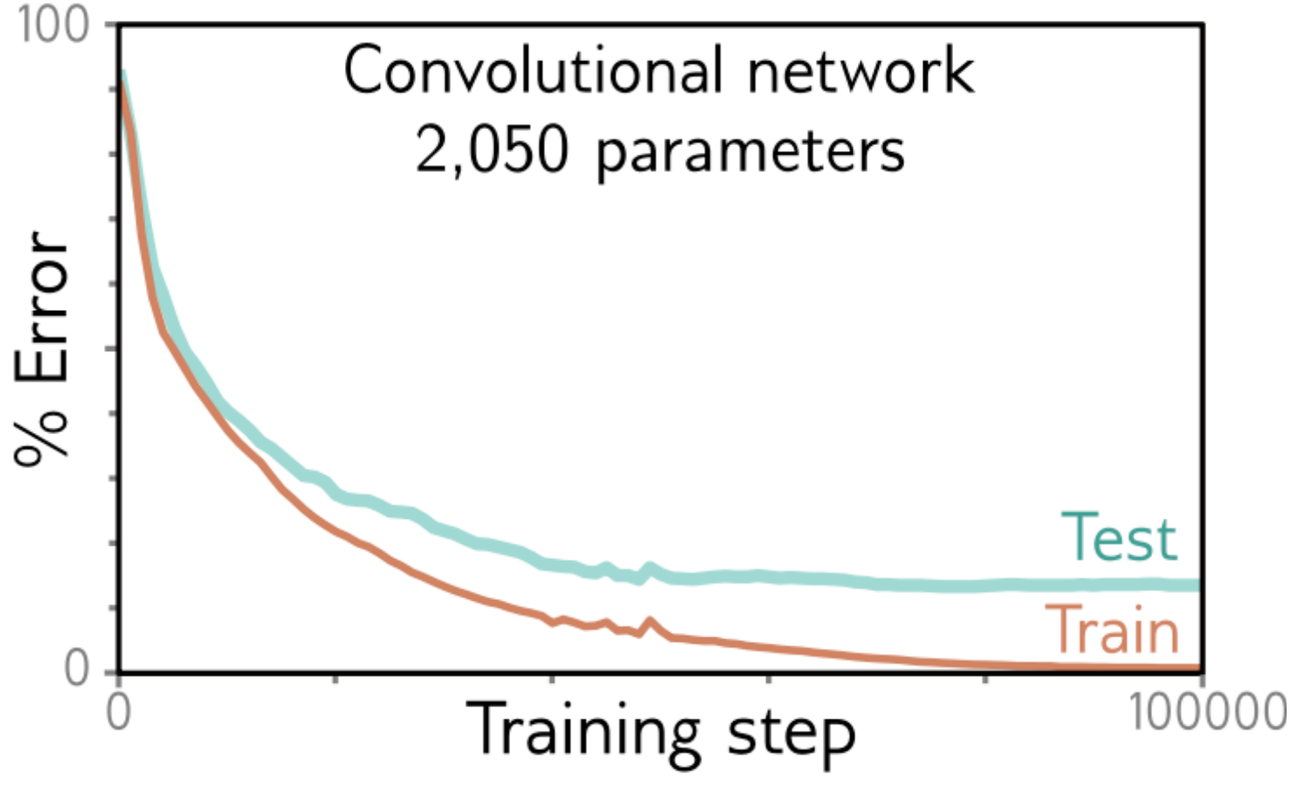
$\boldsymbol{\Omega}_1 \in \mathbb{R}^{15\times 15\times 3}$
$\boldsymbol{\beta}_1 \in \mathbb{R}^{15}$

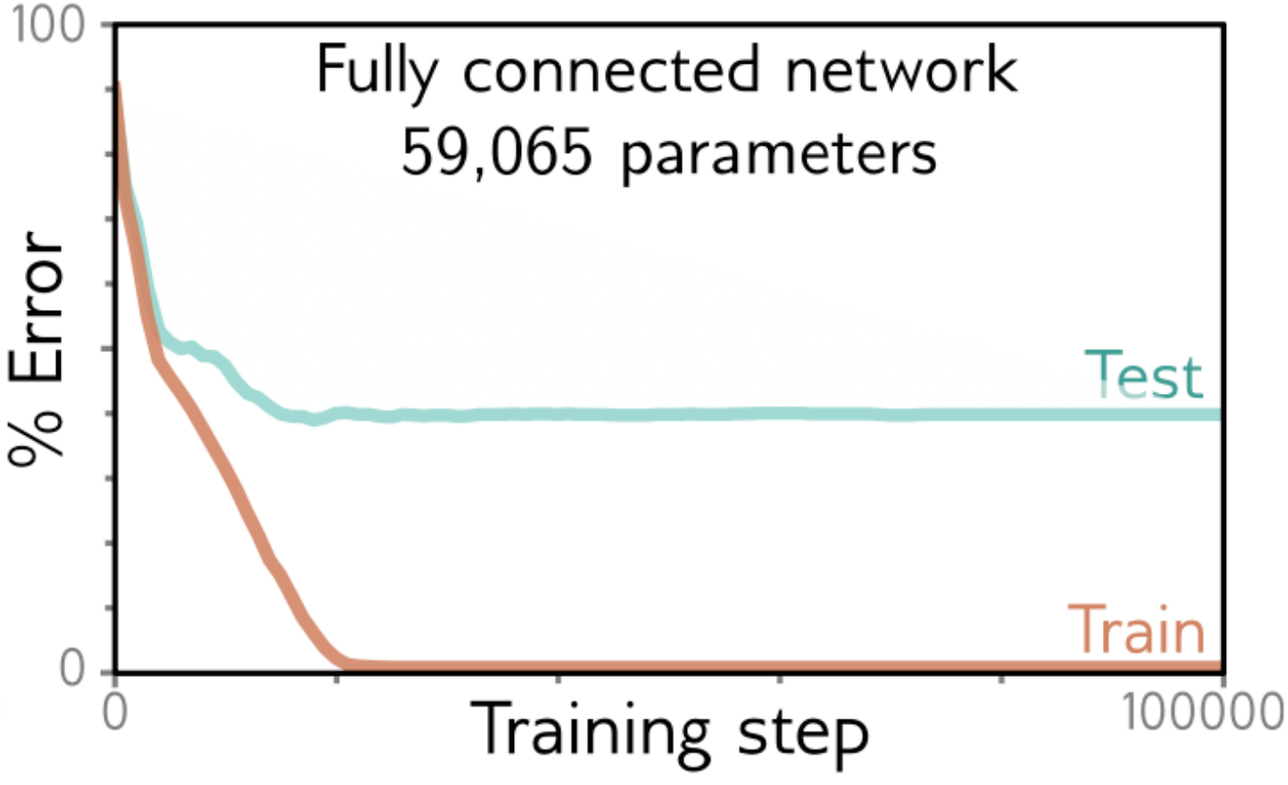
$\boldsymbol{\Omega}_2 \in \mathbb{R}^{15\times 15\times 3}$
$\boldsymbol{\beta}_2 \in \mathbb{R}^{15}$

$\boldsymbol{\Omega}_3 \in \mathbb{R}^{10\times 60}$
$\boldsymbol{\beta}_3 \in \mathbb{R}^{10}$

CNN

- Classifying MNIST-1D data
- Convolutional structure restricts possible mappings to those that process every position similarly
- This restriction (think regularization) can improve performance





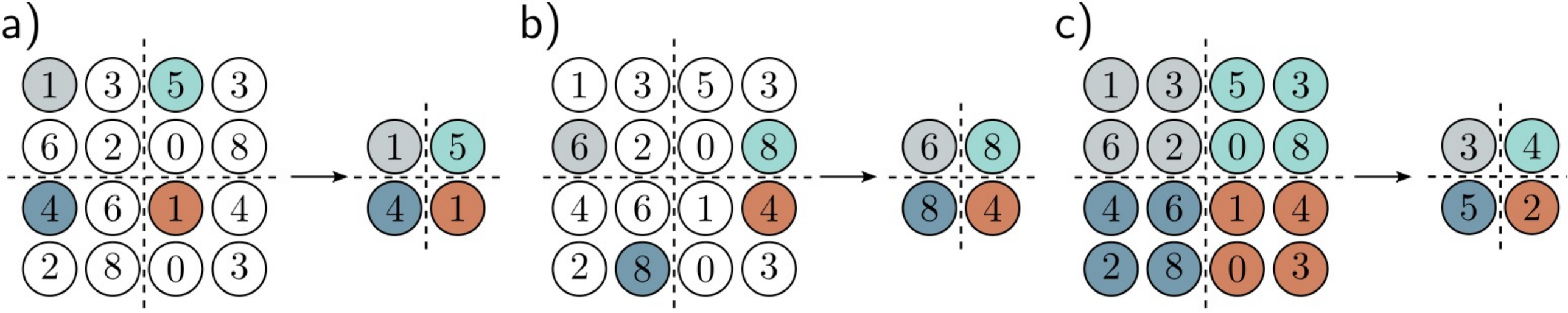
CNN: Pooling

- "Pooling" helps sequentially learn coarser-scale more-complex features by increasing size of receptive field of nodes in later layers

a) Subsampling

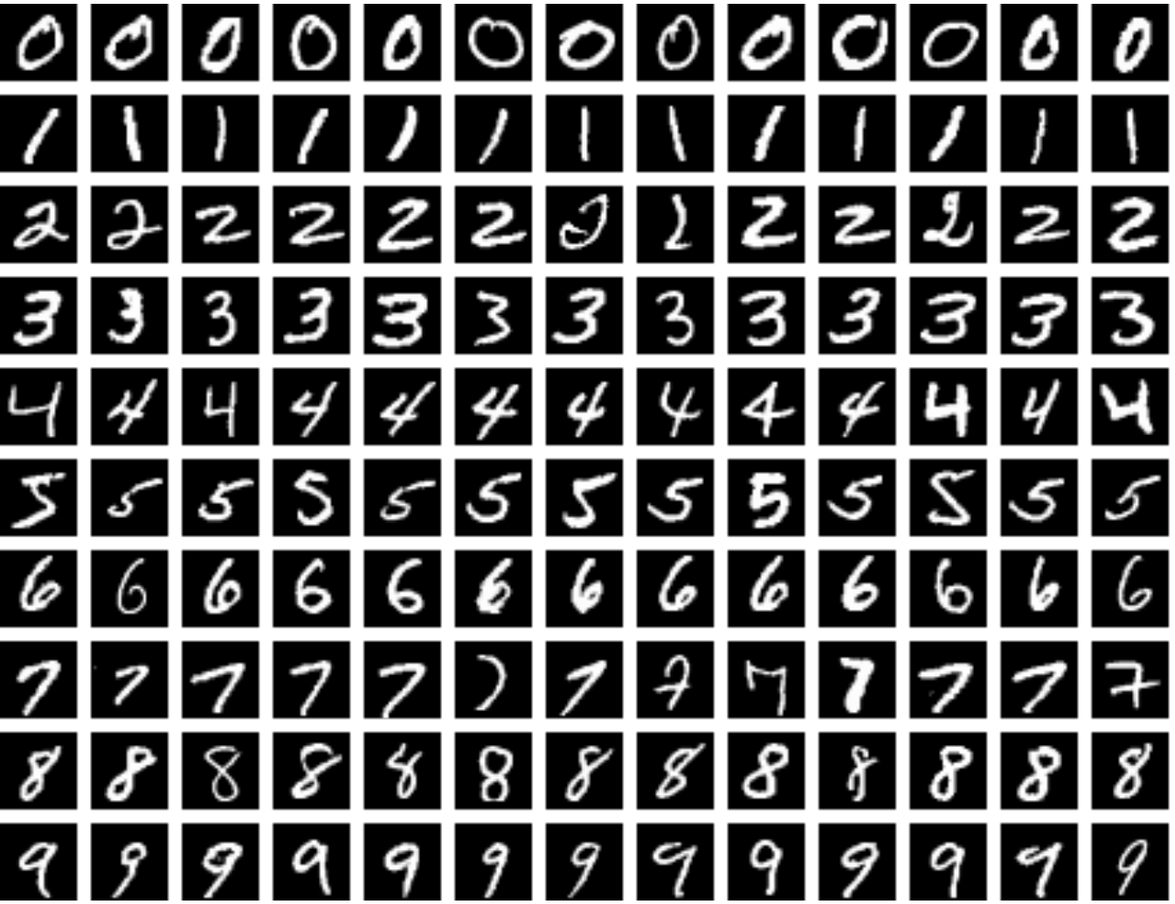
b) Max pooling

c) Mean/average pooling



CNN: LeNet

- Series of CNNs over 1988–1998
- Designed by Yann LeCun



LeNet

Image: 28 (height) × 28 (width) × 1 (channel)

↓

Convolution with 5×5 kernel+2 padding: 28×28×6

↓ sigmoid

Pool with 2×2 average kernel+2 stride: 14×14×6

↓

Convolution with 5×5 kernel (no pad): 10×10×16

↓ sigmoid

Pool with 2×2 average kernel+2 stride: 5×5×16

↓ flatten

Dense: 120 fully connected neurons

↓ sigmoid

Dense: 84 fully connected neurons

↓ sigmoid

Dense: 10 fully connected neurons

↓

Output: 1 of 10 classes

CNN: LeNet

- 1989

Backpropagation Applied to Handwritten Zip Code Recognition

Y. LeCun
B. Boser
J. S. Denker
D. Henderson
R. E. Howard
W. Hubbard
L. D. Jackel
AT&T Bell Laboratories, Holmdel, NJ 07733 USA

The ability of learning networks to generalize can be greatly enhanced by providing constraints from the task domain. This paper demonstrates how such constraints can be integrated into a backpropagation network through the architecture of the network. This approach has been successfully applied to the recognition of handwritten zip code digits provided by the U.S. Postal Service. A single network learns the entire recognition operation, going from the normalized image of the character to the final classification.

CNN: LeNet

- 1998

PROC. OF THE IEEE, NOVEMBER 1998

Gradient-Based Learning Applied to Document Recognition

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner

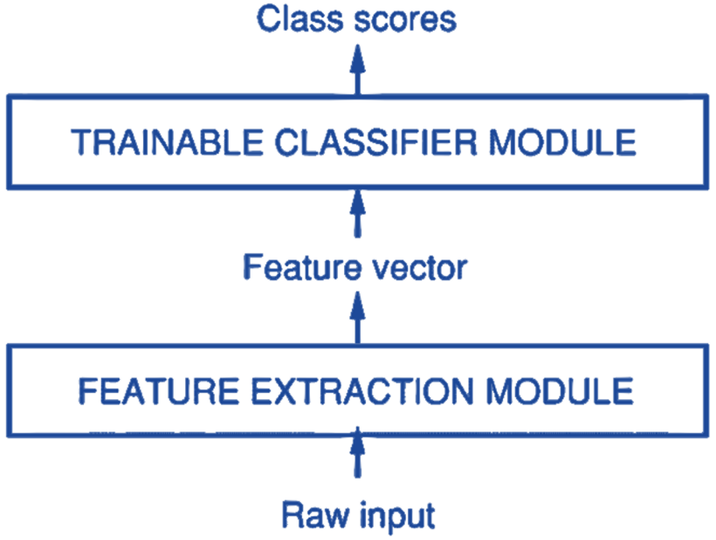


Fig. 1. Traditional pattern recognition is performed with two modules: a fixed feature extractor and a trainable classifier.

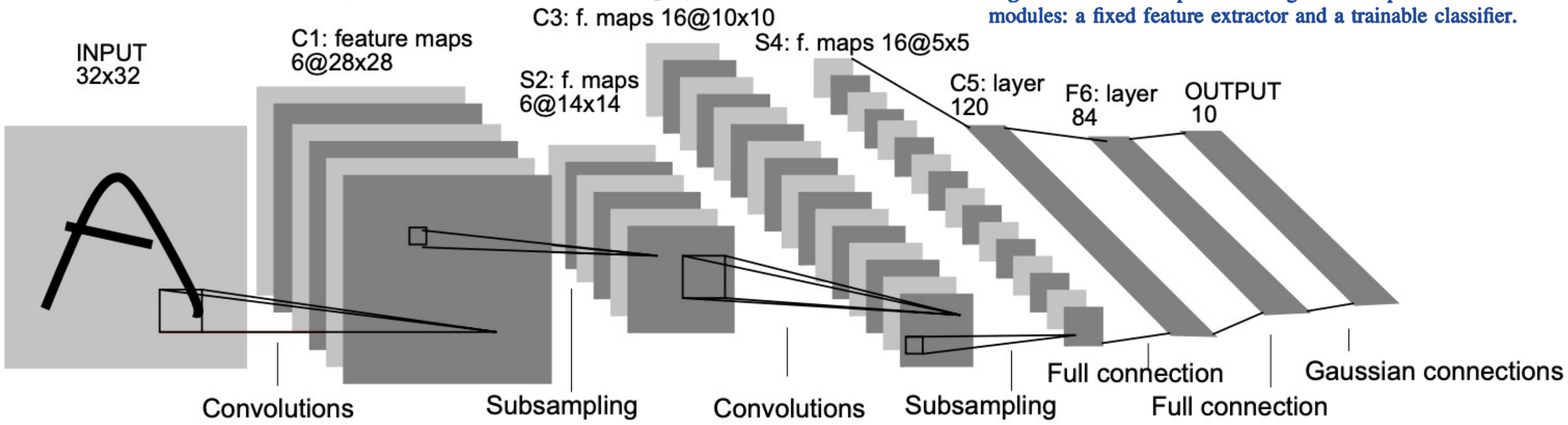


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

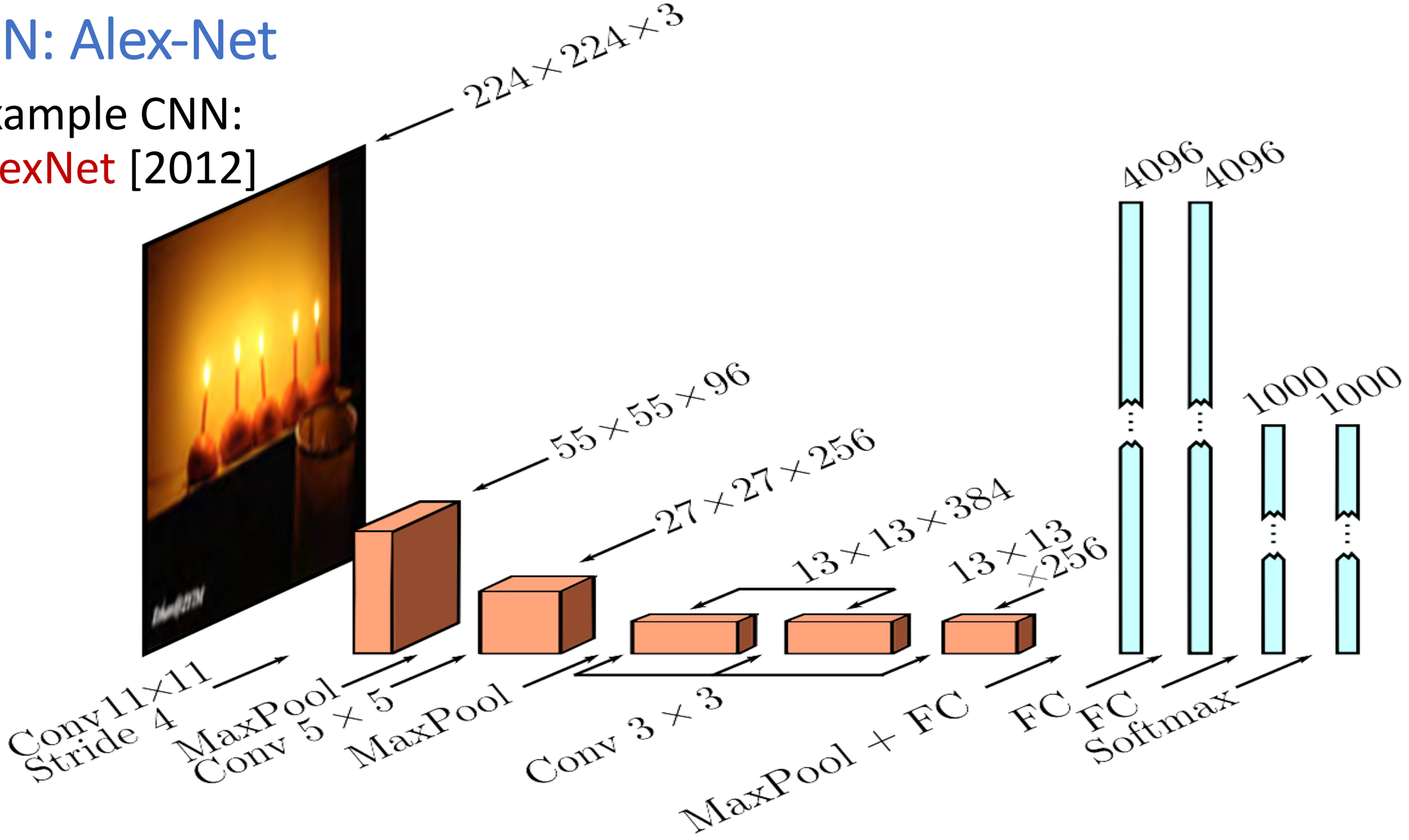
ImageNet

- Image database
  - Organized according to WordNet hierarchy (currently only nouns), in which each node of hierarchy is depicted by hundreds and thousands of images
  - image-net.org
- ImageNet dataset, 2010
  - 20,000 classes, 14 million images
  - en.wikipedia.org/wiki/ImageNet
- ImageNet Large Scale Visual Recognition Challenge (ILSVRC)
  - Evaluates methods for object detection and image classification at large scale
  - Uses a subset of ImageNet dataset
  - 1000 object classes, total 1,281,167 training images
  - image-net.org/challenges/LSVRC/index.php

ImageNet

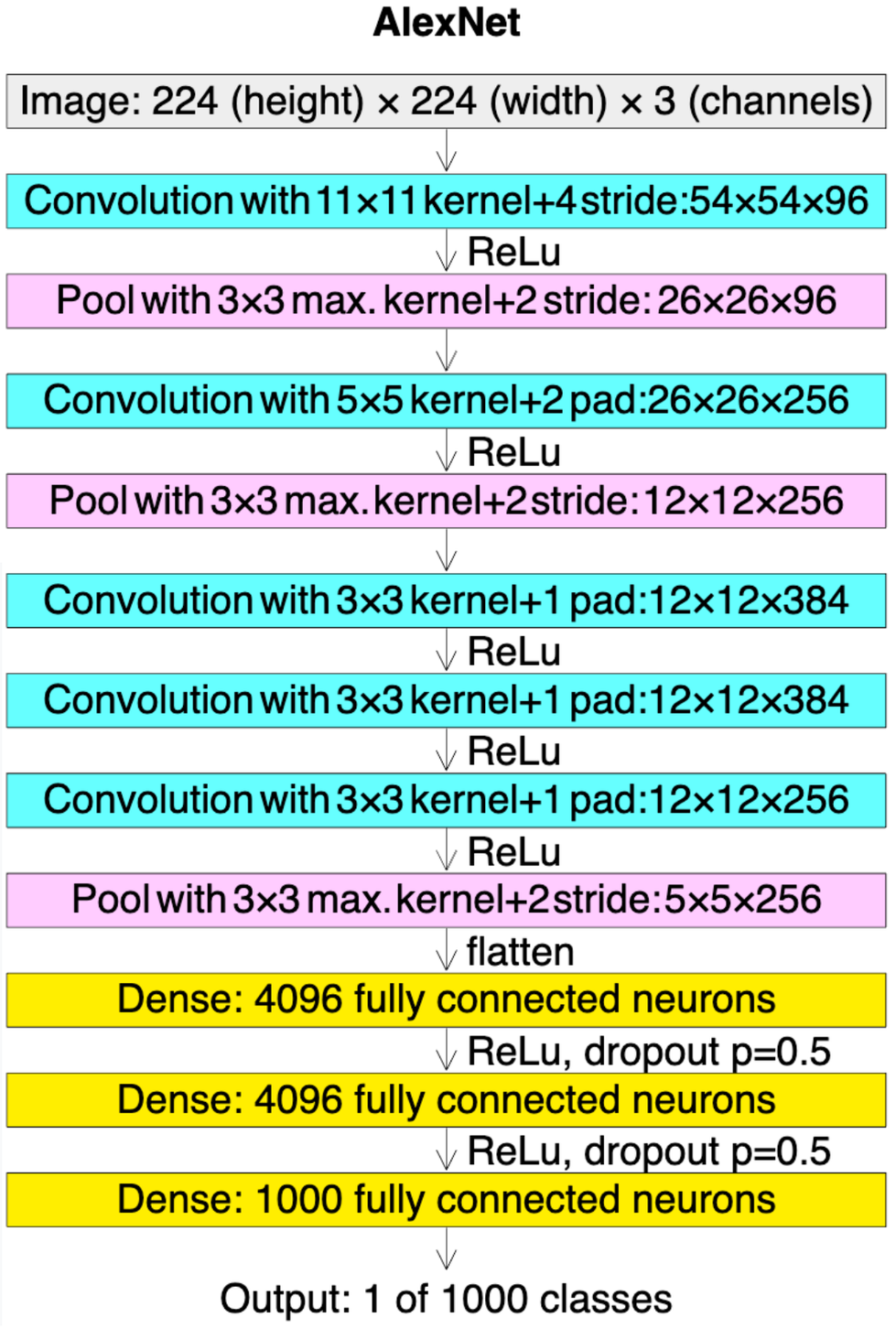
CNN: Alex-Net

- Example CNN: AlexNet [2012]



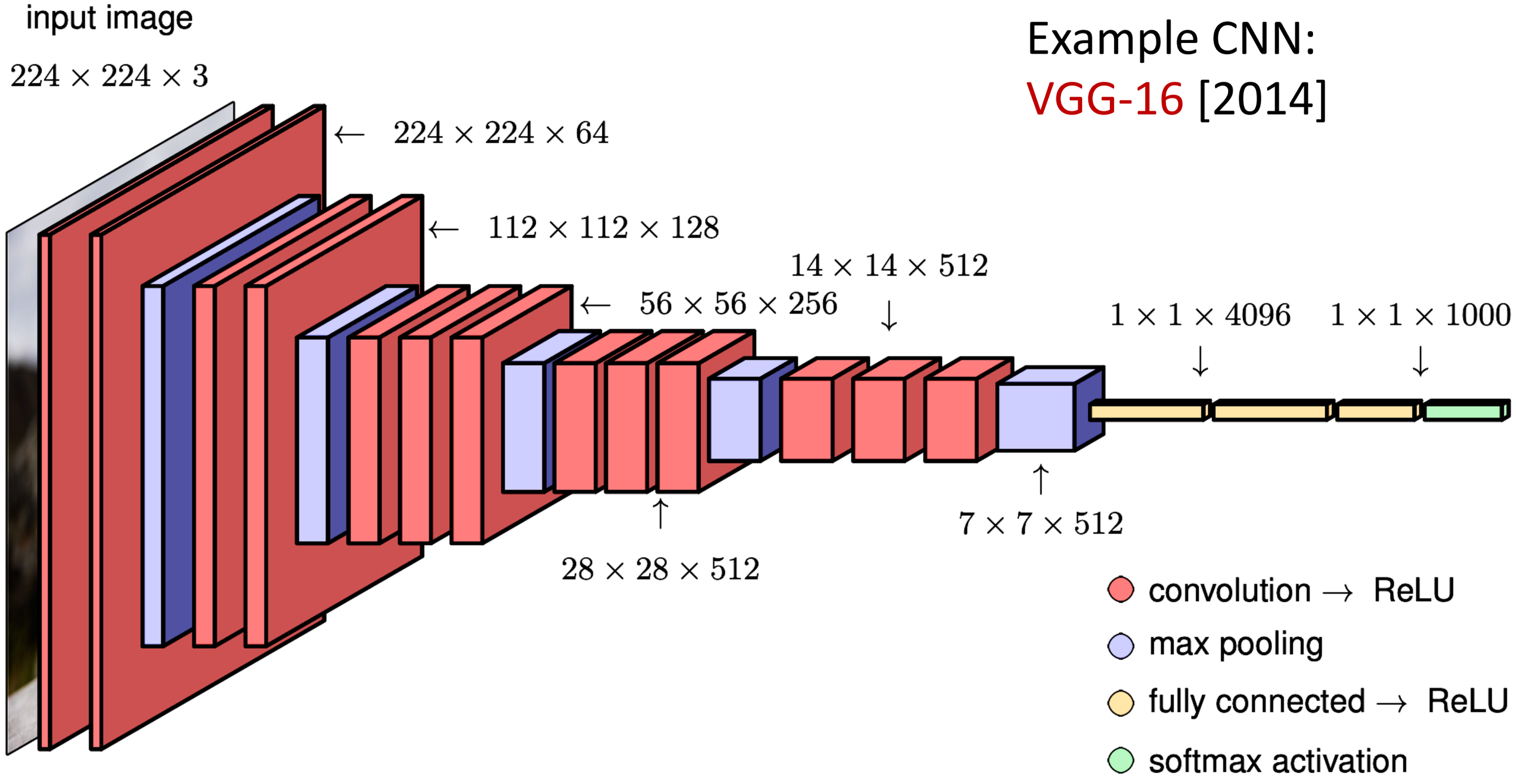
CNN: Alex-Net

- Example CNN: AlexNet [2012]
- More details



(AlexNet image size should be 227×227×3, instead of 224×224×3, so the math will come out right. The original paper said different numbers, but Andrej Karpathy, the former head of computer vision at Tesla, said it should be 227×227×3 (he said Alex didn't describe why he put 224×224×3). The next convolution should be 11×11 with stride 4: 55×55×96 (instead of 54×54×96). It would be calculated, for example, as: [(input width 227 - kernel width 11) / stride 4] + 1 = [(227 - 11) / 4] + 1 = 55. Since the kernel output is the same length as width, its area is 55×55.)

CNN: VGG-Net

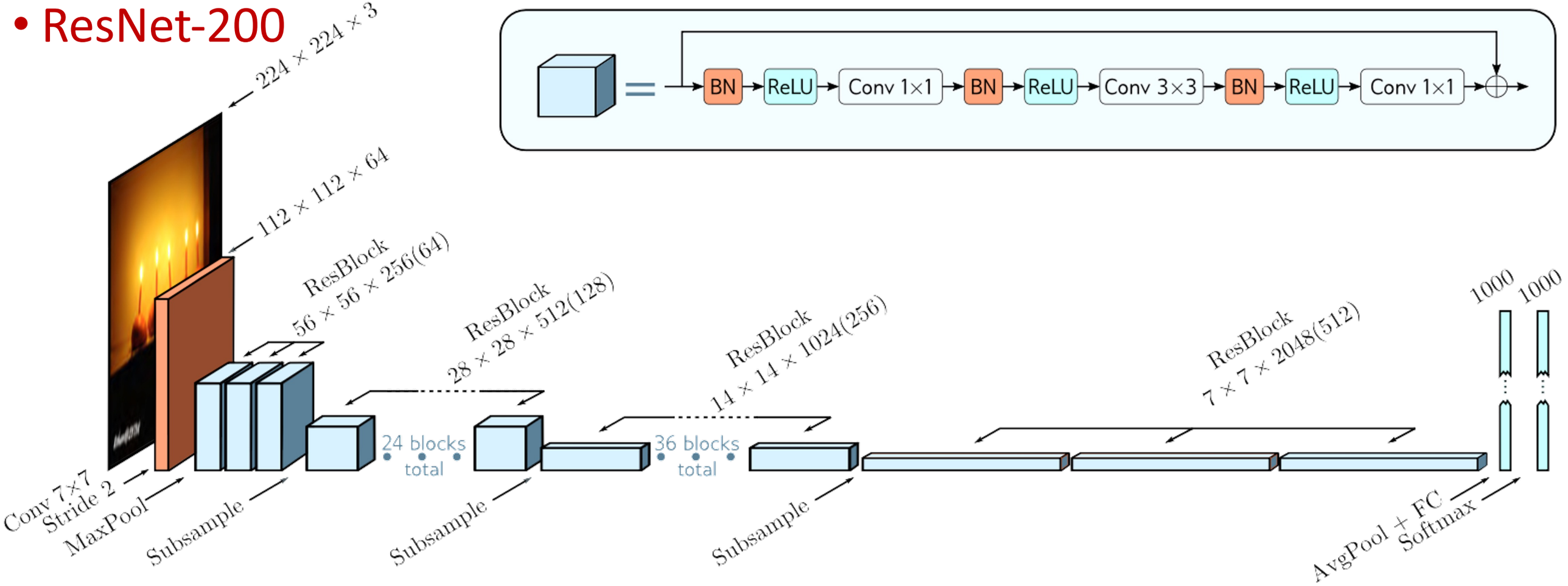


Example CNN:

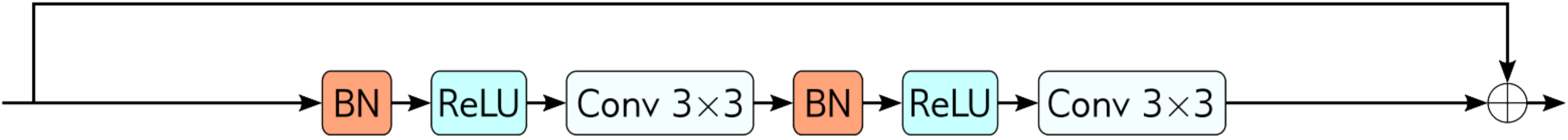
VGG-16 [2014]

CNN: ResNet

- ResNet-200



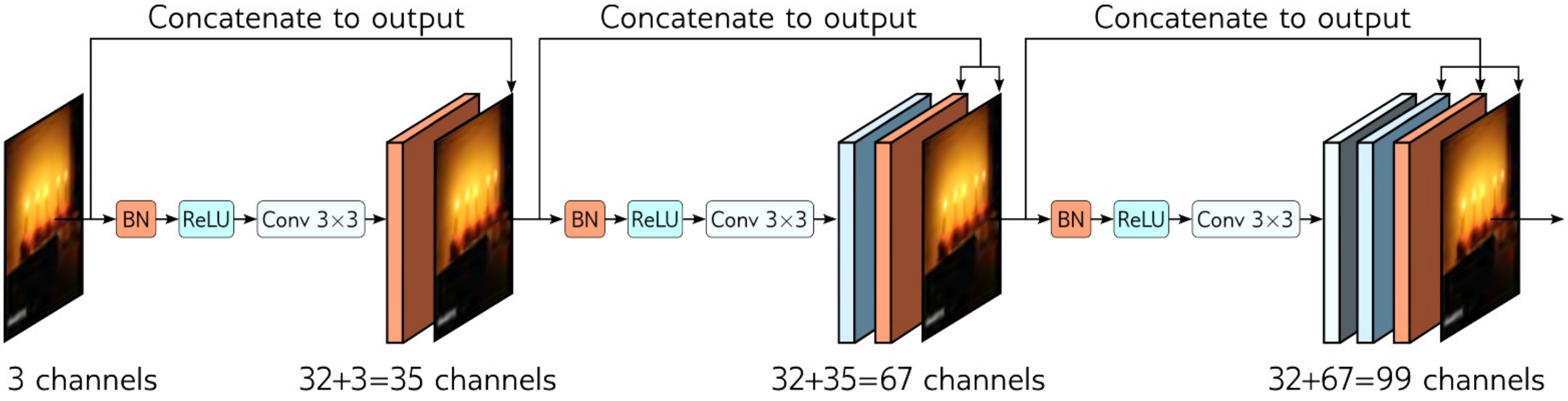
- Standard block in ResNet architecture



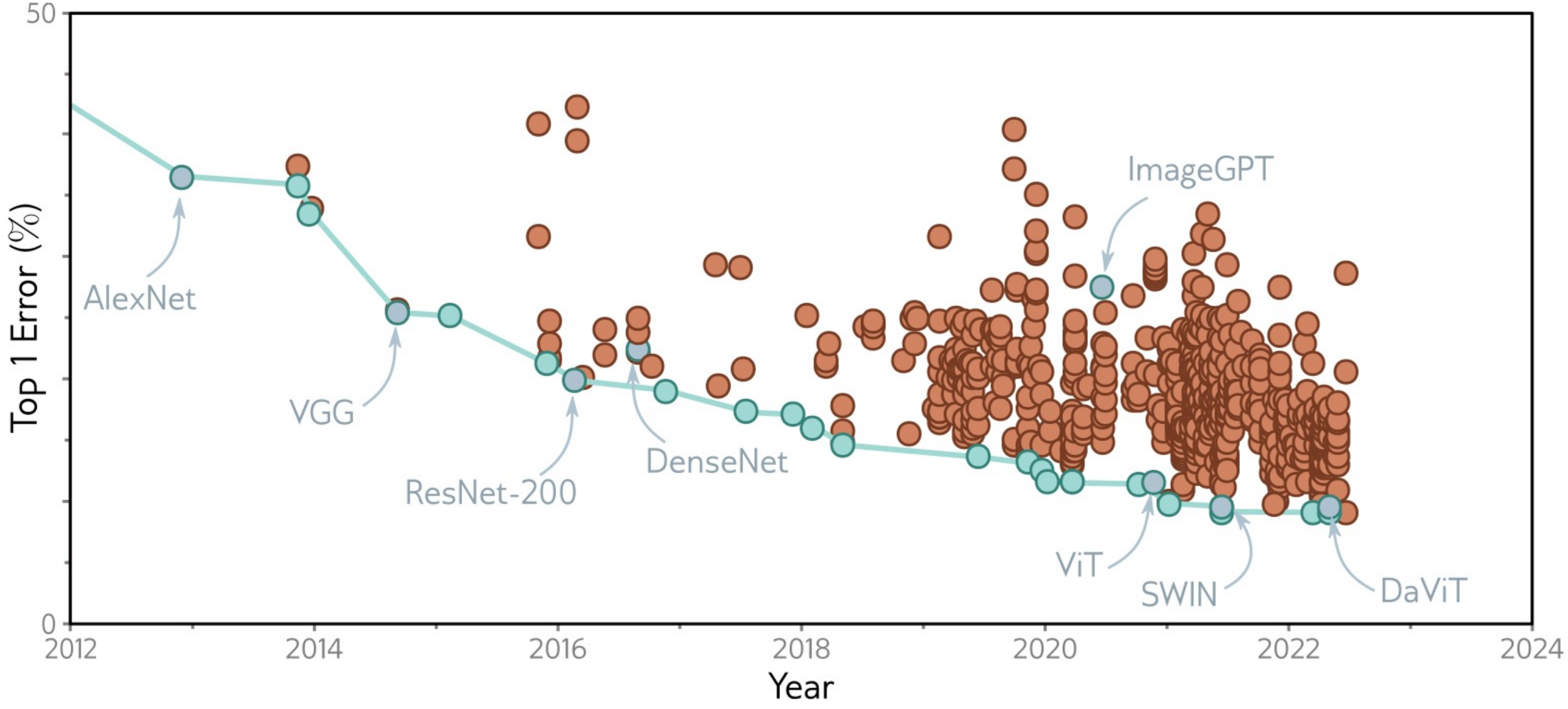
CNN: DenseNet

1) 3-channel input image transformed to 32-channel representation
2) Input image is concatenated to this representation → 35 channels
3) This 35-channel representation transformed → 32-channel output
4) Concatenate to 35-channel input → 67 channels

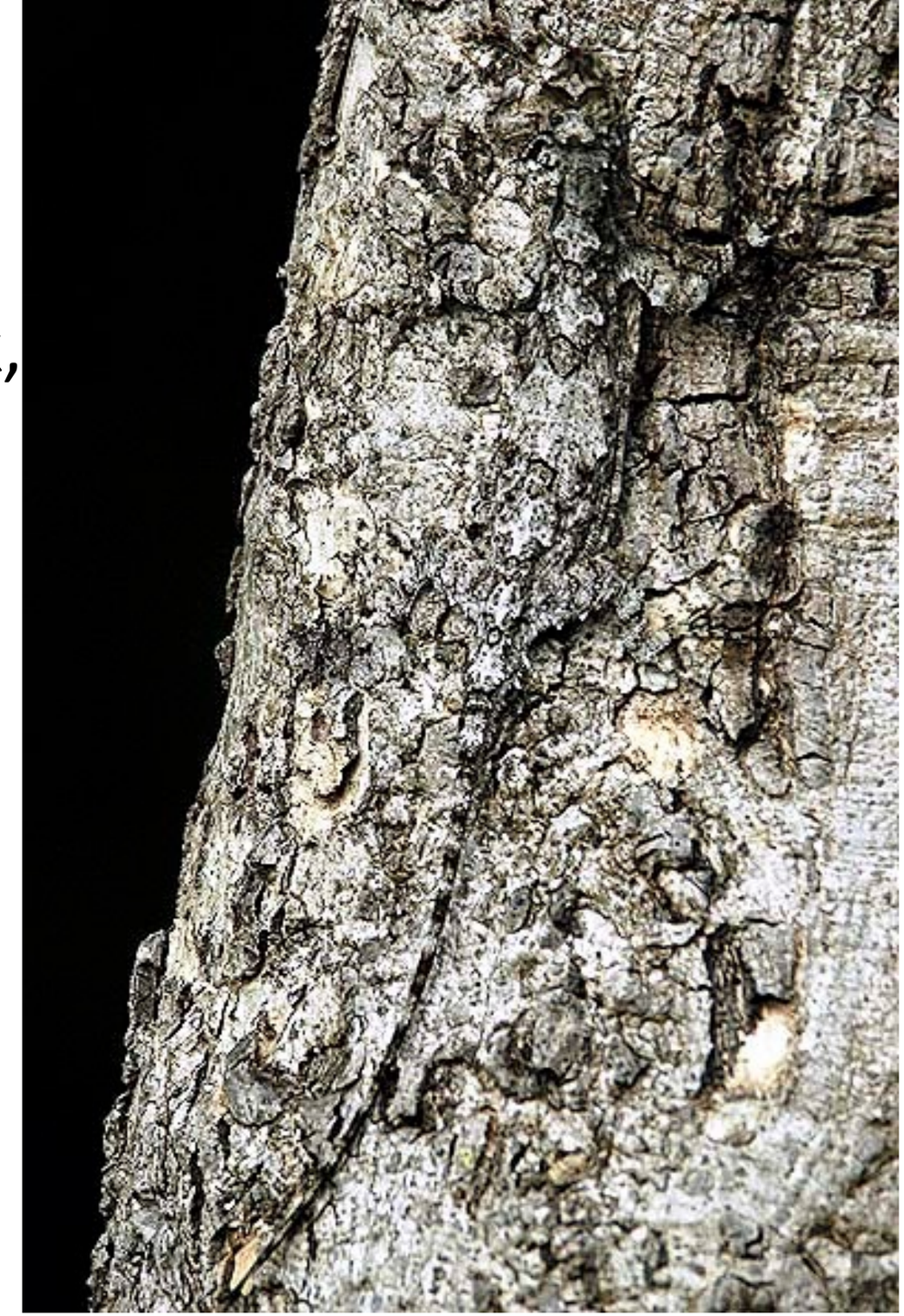
- Input to layer comprises concatenated outputs from all previous layers



CNNs and DNNs for ImageNet Classification



- Draco dussumieri from Bandipur National Park, India

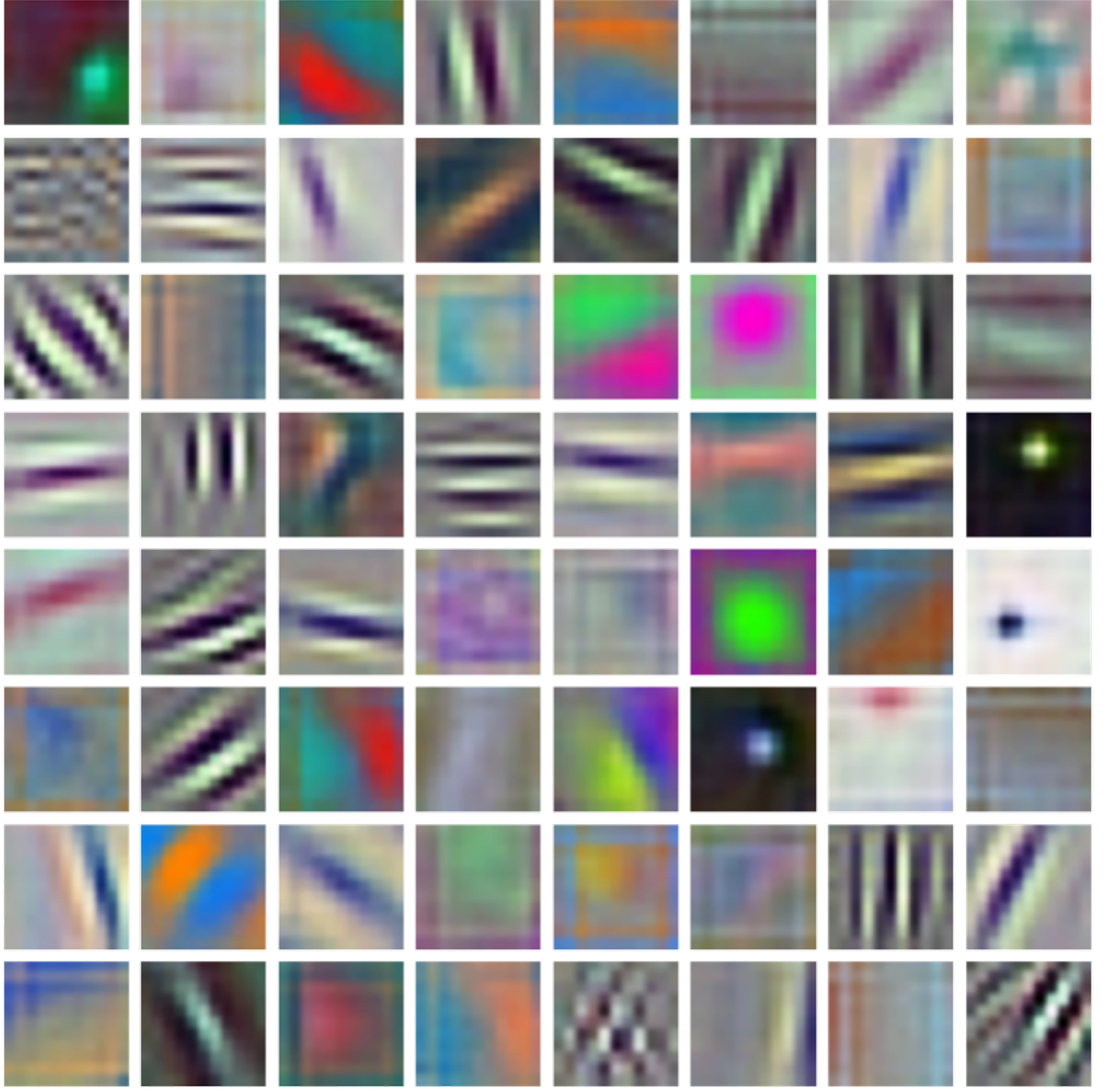


- Indian leaf butterfly

- Lion, Kruger National Park, South Africa

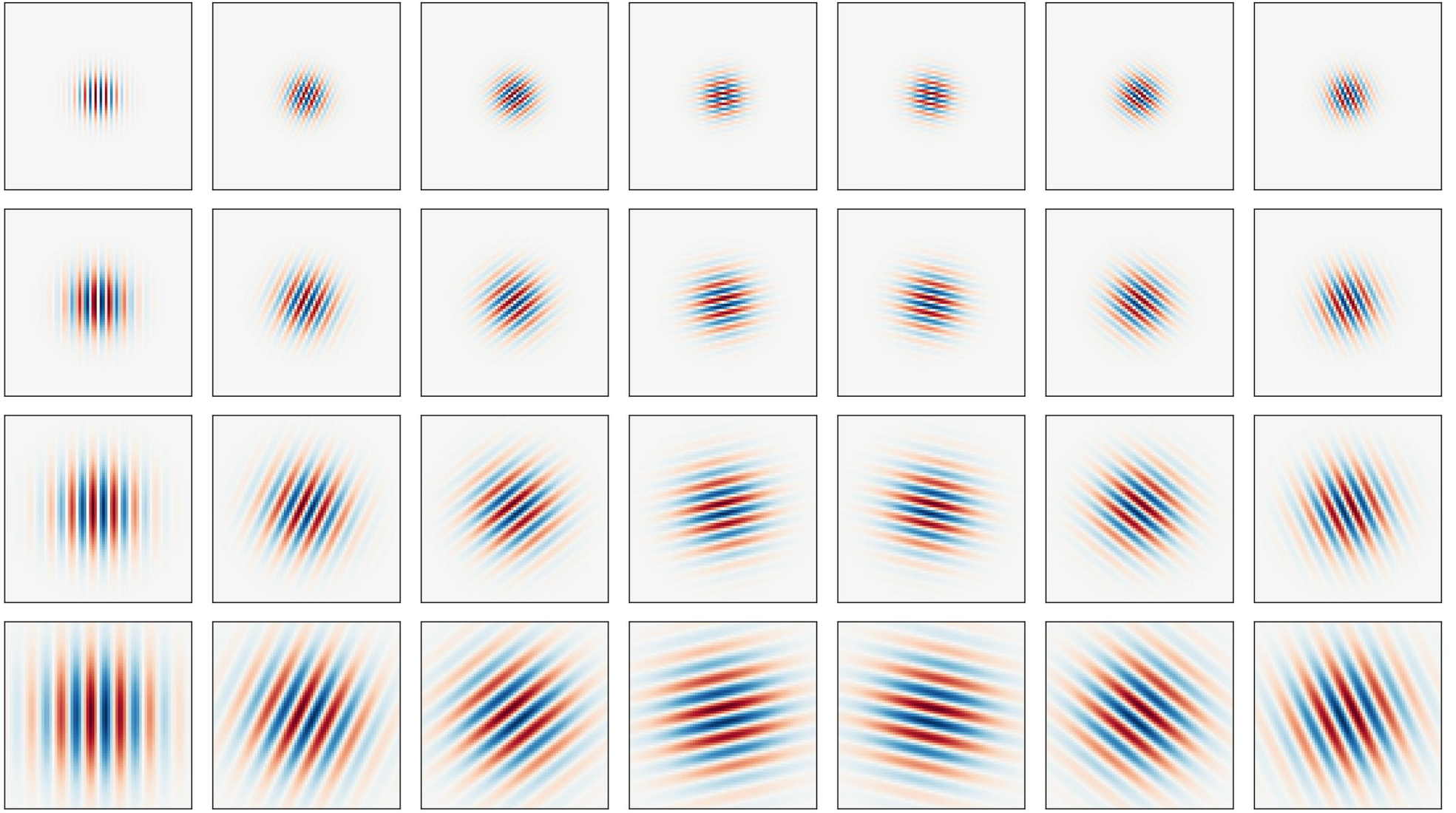
CNN: Visualize

- Visualize convolutional filters learned in first layer of AlexNet
- Similar to those found in many other frameworks
  - e.g., Gabor filters
  - e.g., patch-based statistical modeling of texture



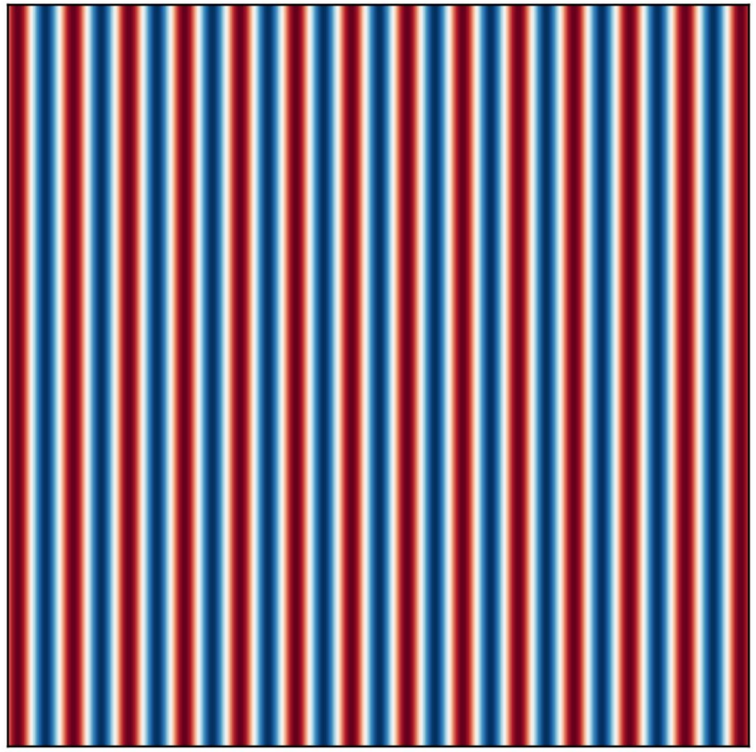
CNN: Visualize

- Gabor filter set: design inspired by processing of visual information by neurons in human brain
- **Example**
- **Rows**: different frequencies/ scales
- **Columns**: different orientations

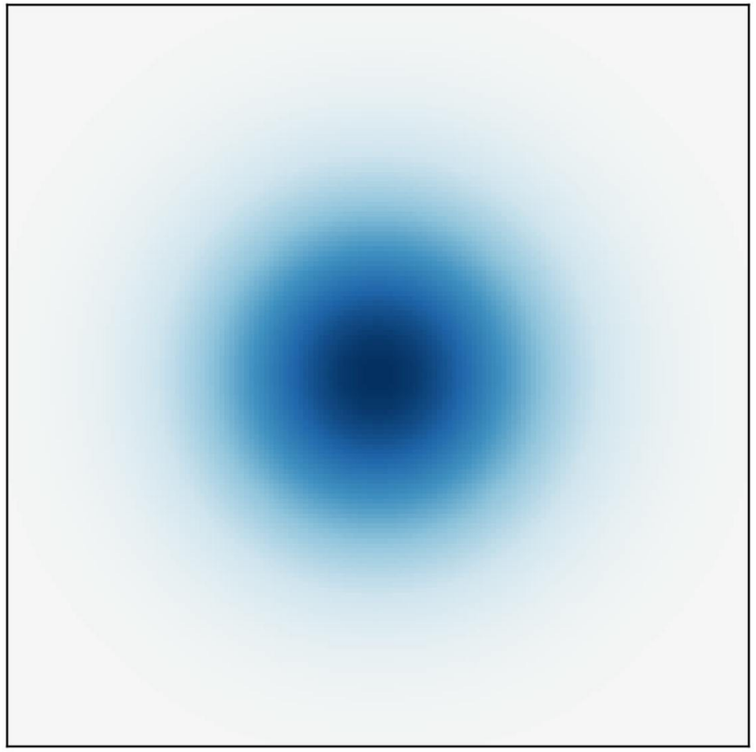


Gabor Filters

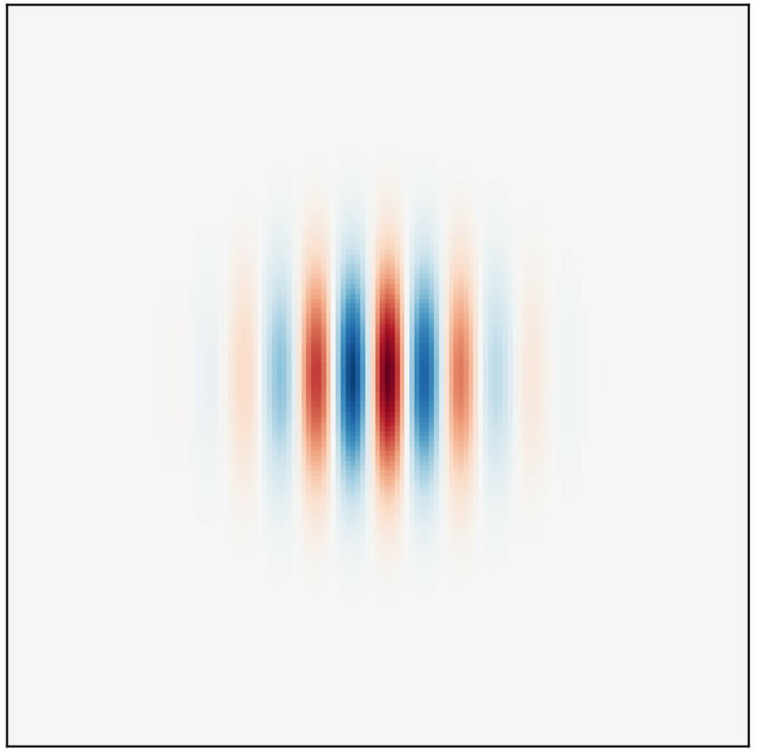
(a) Sinusoidal filter



(b) Gaussian filter



(c) Gabor filter



world.org/blog/engineering/iris-feature-extraction

Gabor Filters

- 2D Gabor filter: Gaussian kernel multiplied by complex-valued sinusoid

1) Wave-length = λ
2) Orienta-tion = θ
   - (x',y') = rotated (x,y)
3) Aspect ratio = γ

Complex

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \exp\left(i\left(2\pi\frac{x'}{\lambda} + \psi\right)\right)$$

Real

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cos\left(2\pi\frac{x'}{\lambda} + \psi\right)$$

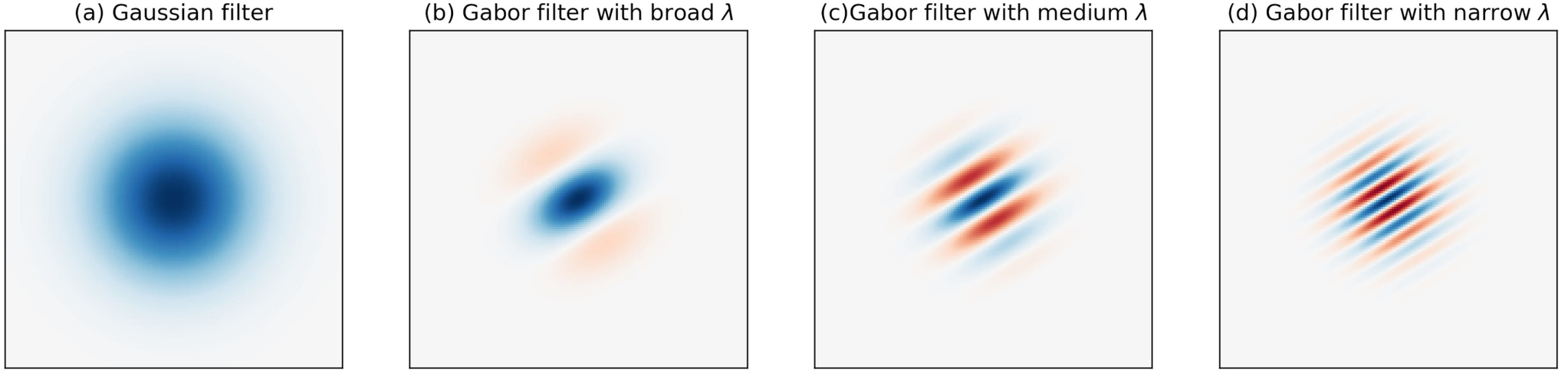
Imaginary

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \sin\left(2\pi\frac{x'}{\lambda} + \psi\right)$$

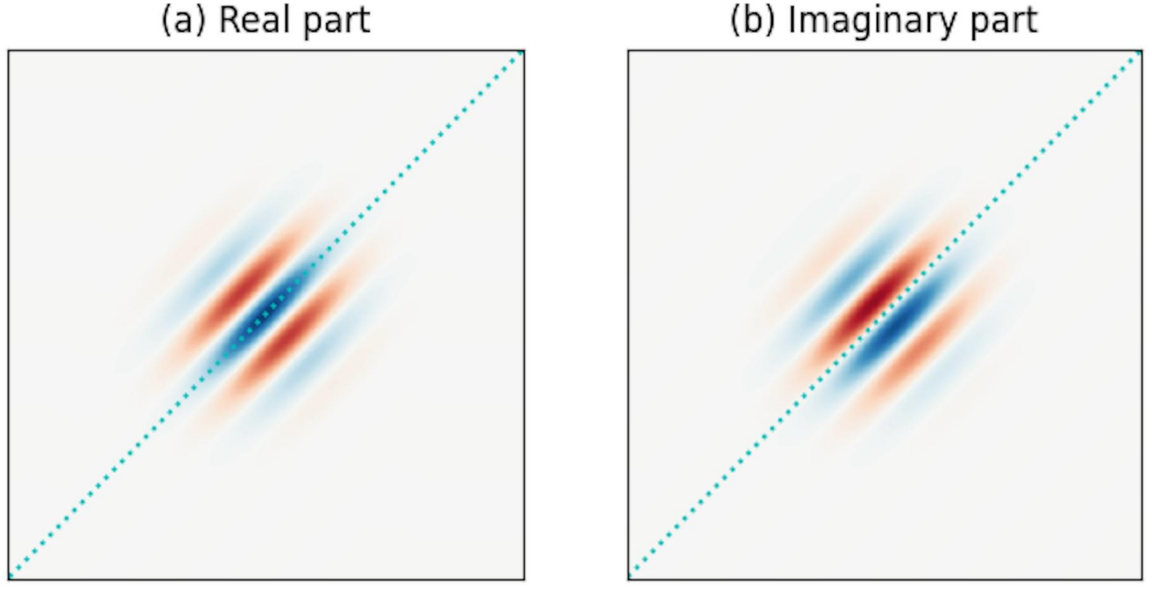
where $x' = x\cos\theta + y\sin\theta$ and $y' = -x\sin\theta + y\cos\theta$.

Gabor Filters

- Variation w.r.t. wavelength λ

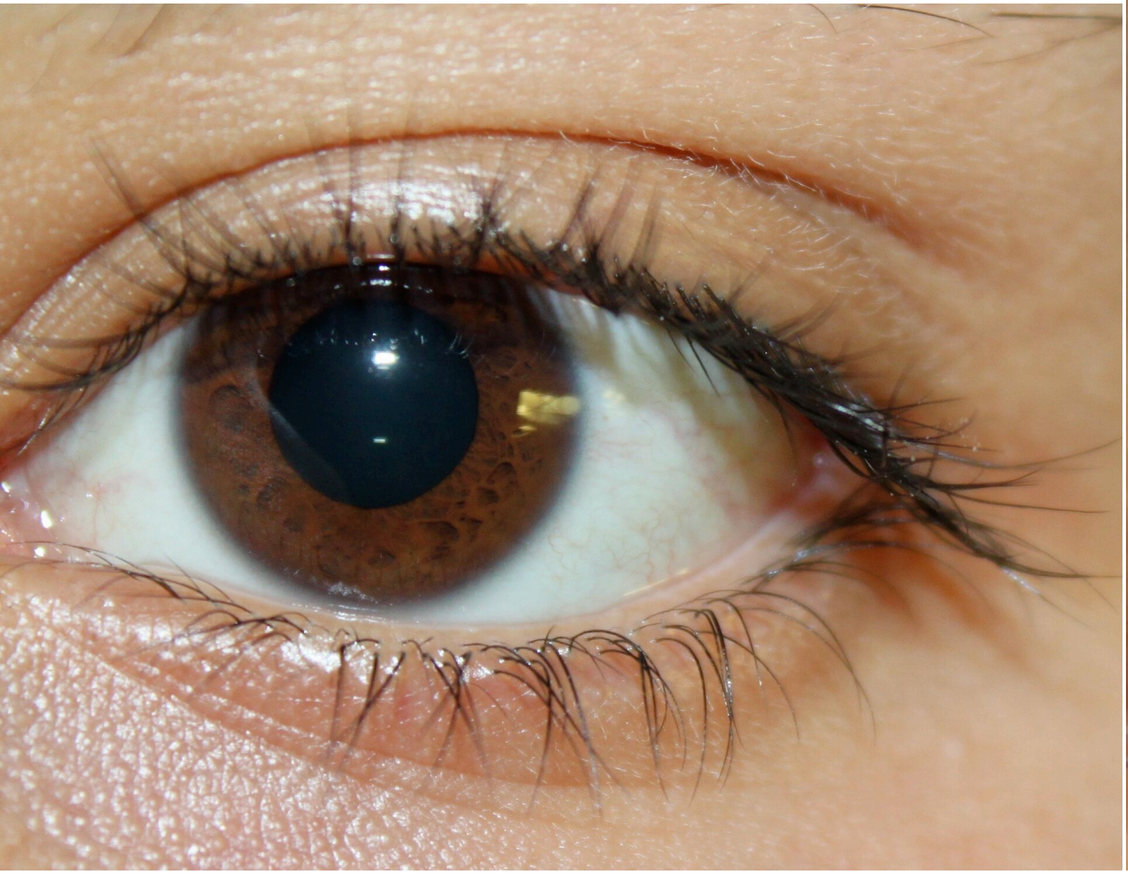


- Aspect ratio γ controls ellipticity of the starting Gaussian
- Real and Imag parts produces 2 filters as →

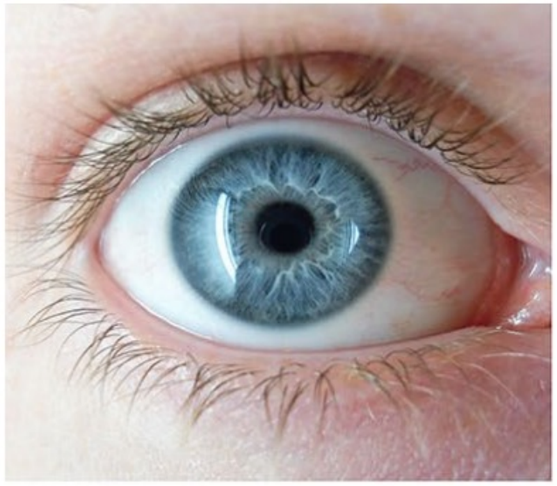


world.org/blog/engineering/iris-feature-extraction

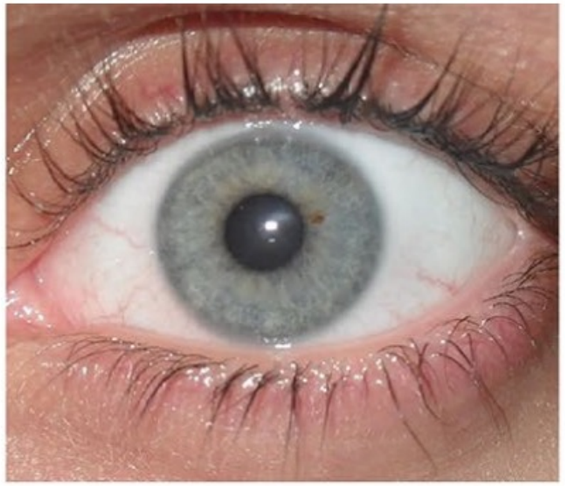
Iris Recognition



Iris F



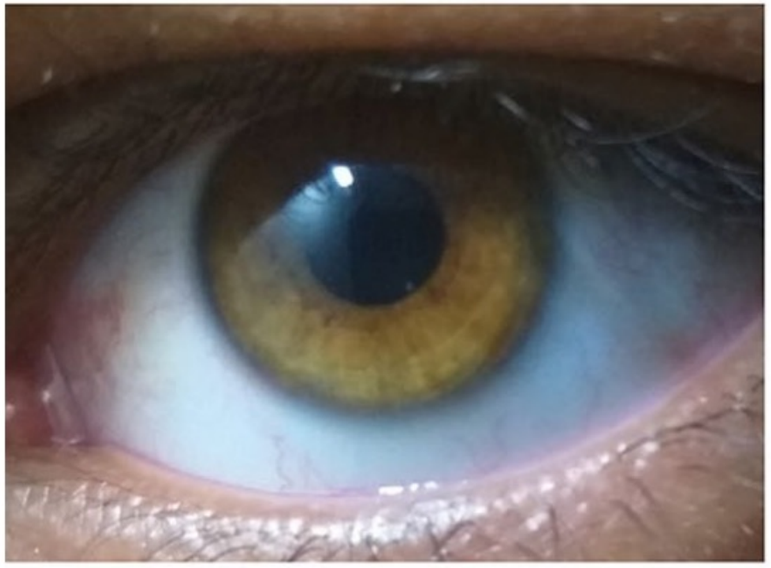
Light Blue



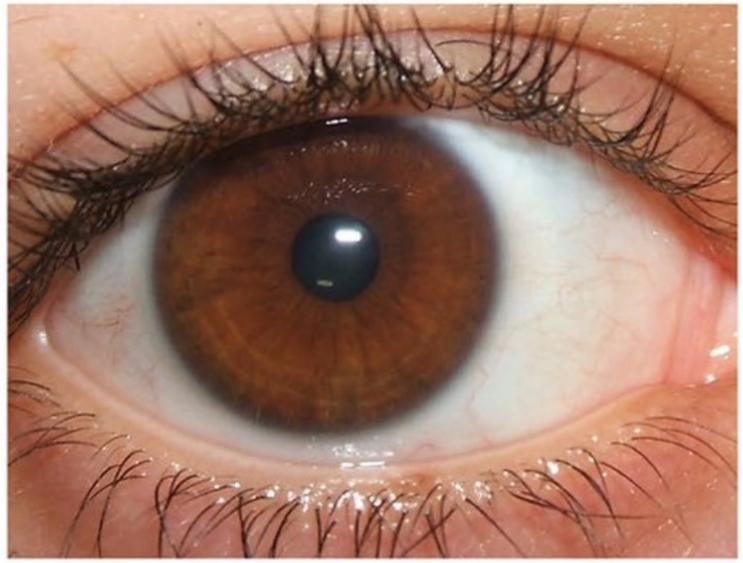
Gray

Green

Hazel



Amber



Light Brown

Dark Brown

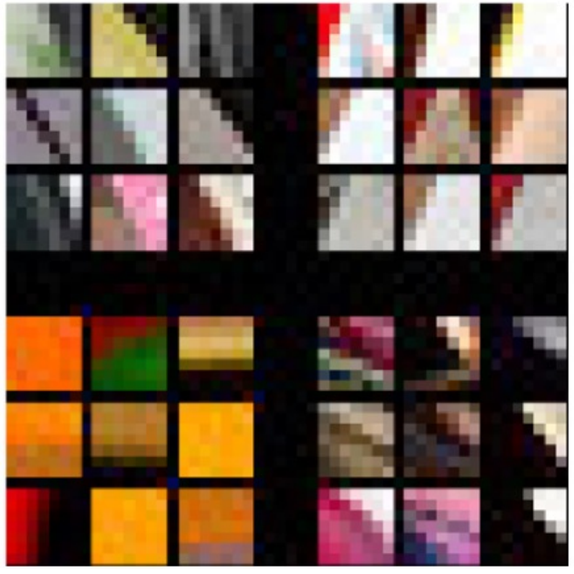
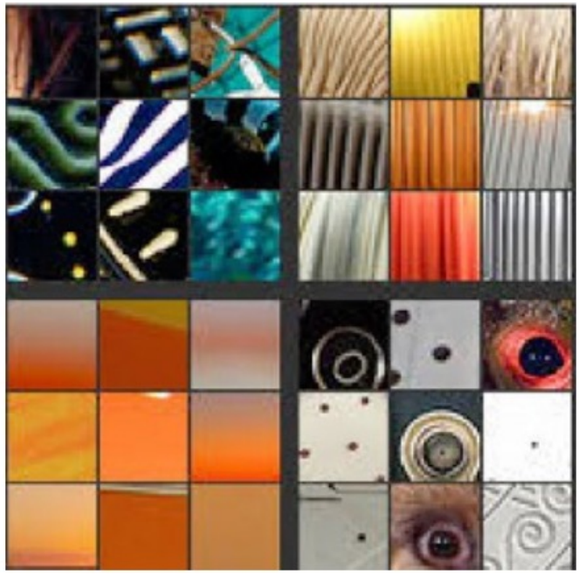
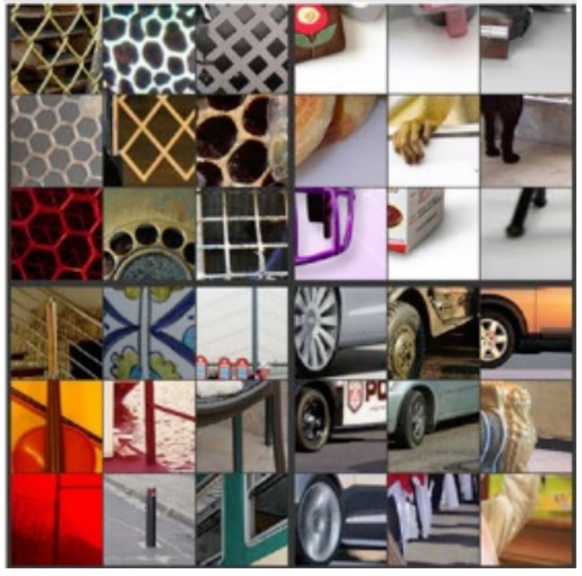
www.nature.com/articles/s41598-024-52807-9

Figure 1. Examples of Iris colors, used for classification of participants. Pictures are taken from Wikipedia, the free encyclopedia (https://en.wikipedia.org/w/index.php?title=Eye_color&oldid=1187612730).

CNN: Visualize

- Visually interpreting a filter in later layers isn't straightforward; its input isn't an image patch, but filter response from earlier layer

1) Instead, visualize effect of filter in later layers as follows: Search patch, in dataset, giving highest activation in chosen neuron
   - e.g., in progressively deeper layers, choose 4 random channels, find top 9 activations across nodes; show those patches [Visualizing and understanding convolutional networks : Zeiler & Fergus 2014 ECCV]

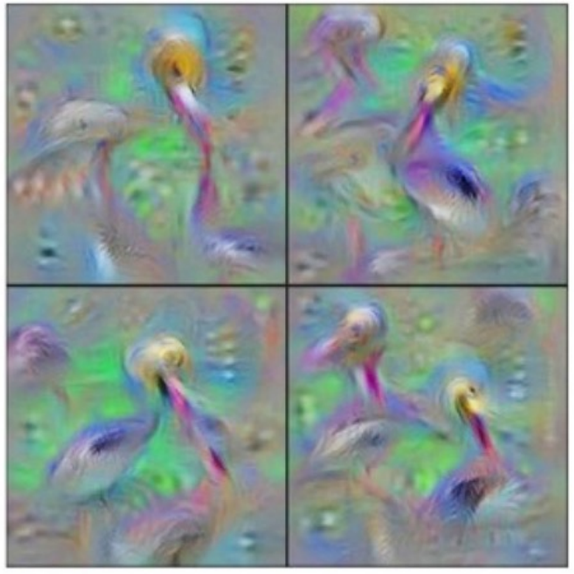




CNN: Visualize

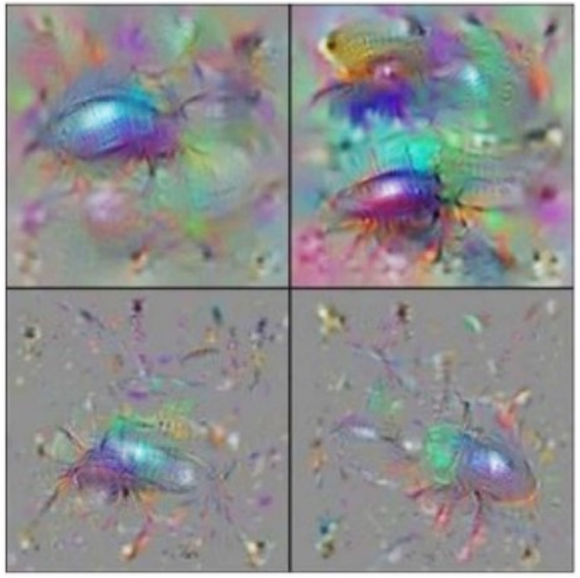
2) Instead of searching for patches empirically (from some given set), solve for optimal image that maximizes activation for specific neuron

- e.g., chose a neuron from last/output layer of a classifier;
  then optimal image is “most representative” of corresponding class label
  [Understanding neural networks through deep visualization,
  Yosinski et al. 2015 ICML]

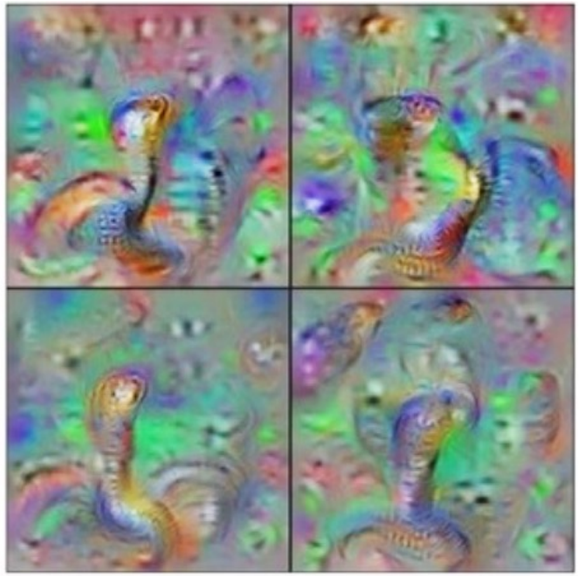
Flamingo



Pelican



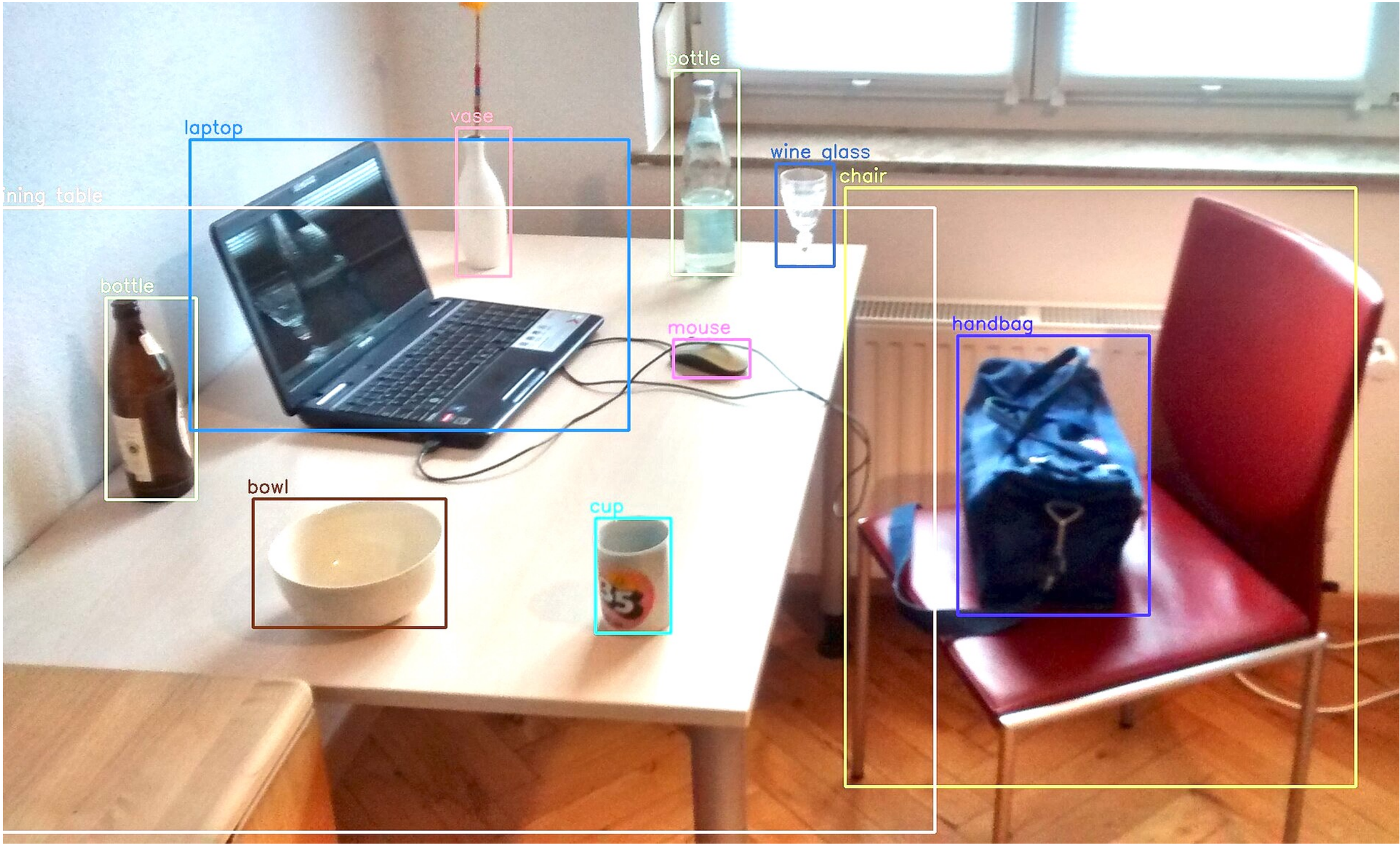
Ground Beetle



Indian Cobra

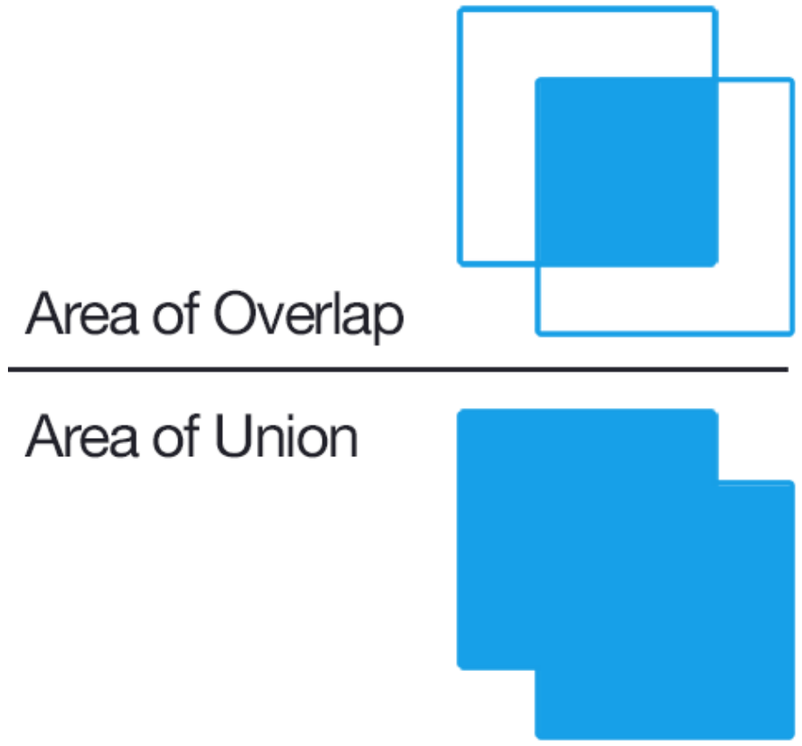
CNN: Object Detection

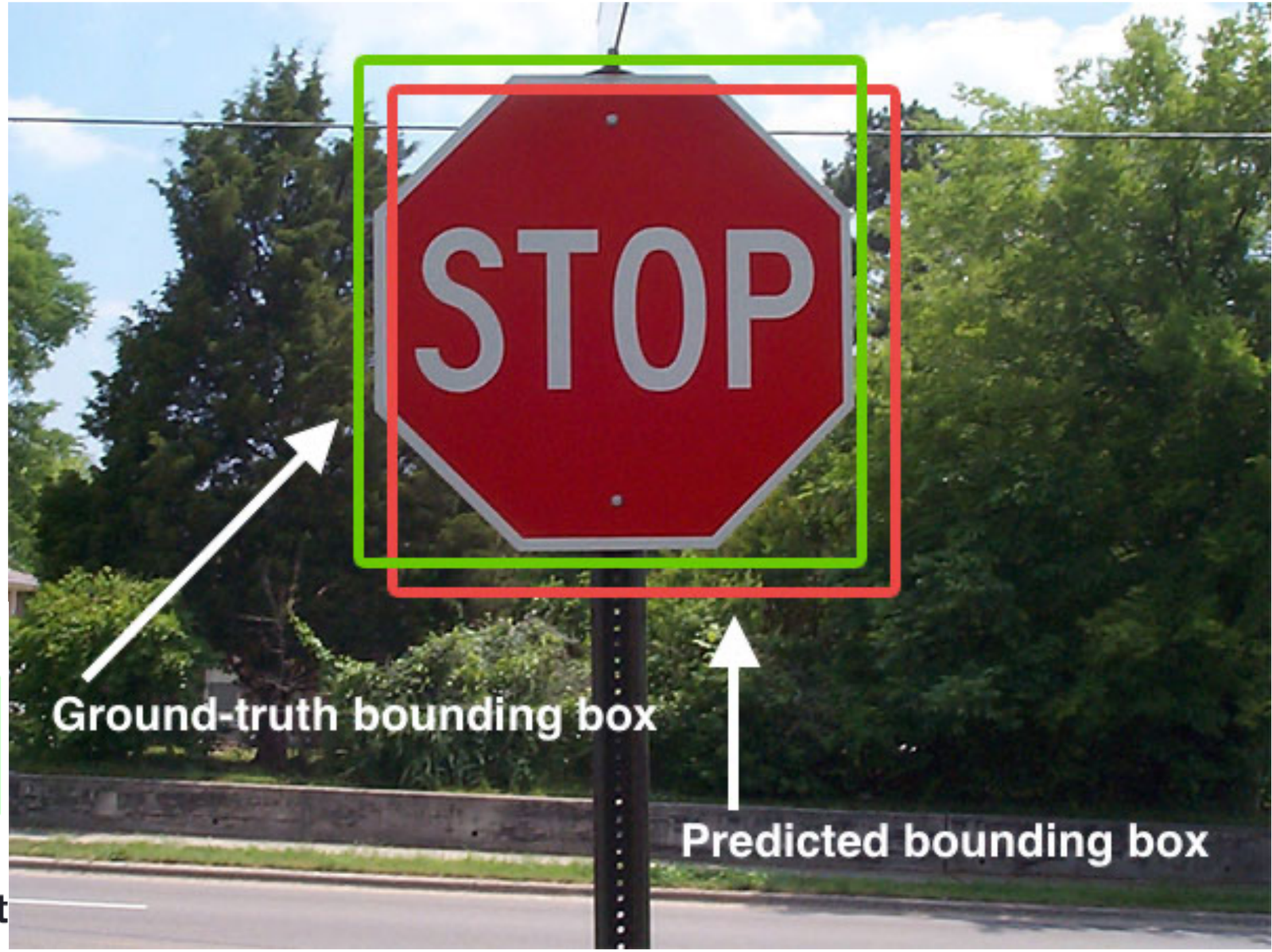
- Bounding boxes for objects



CNN: Object Detection

- Intersection over union (IoU): metric for learning, evaluation





CNN: Object Detection

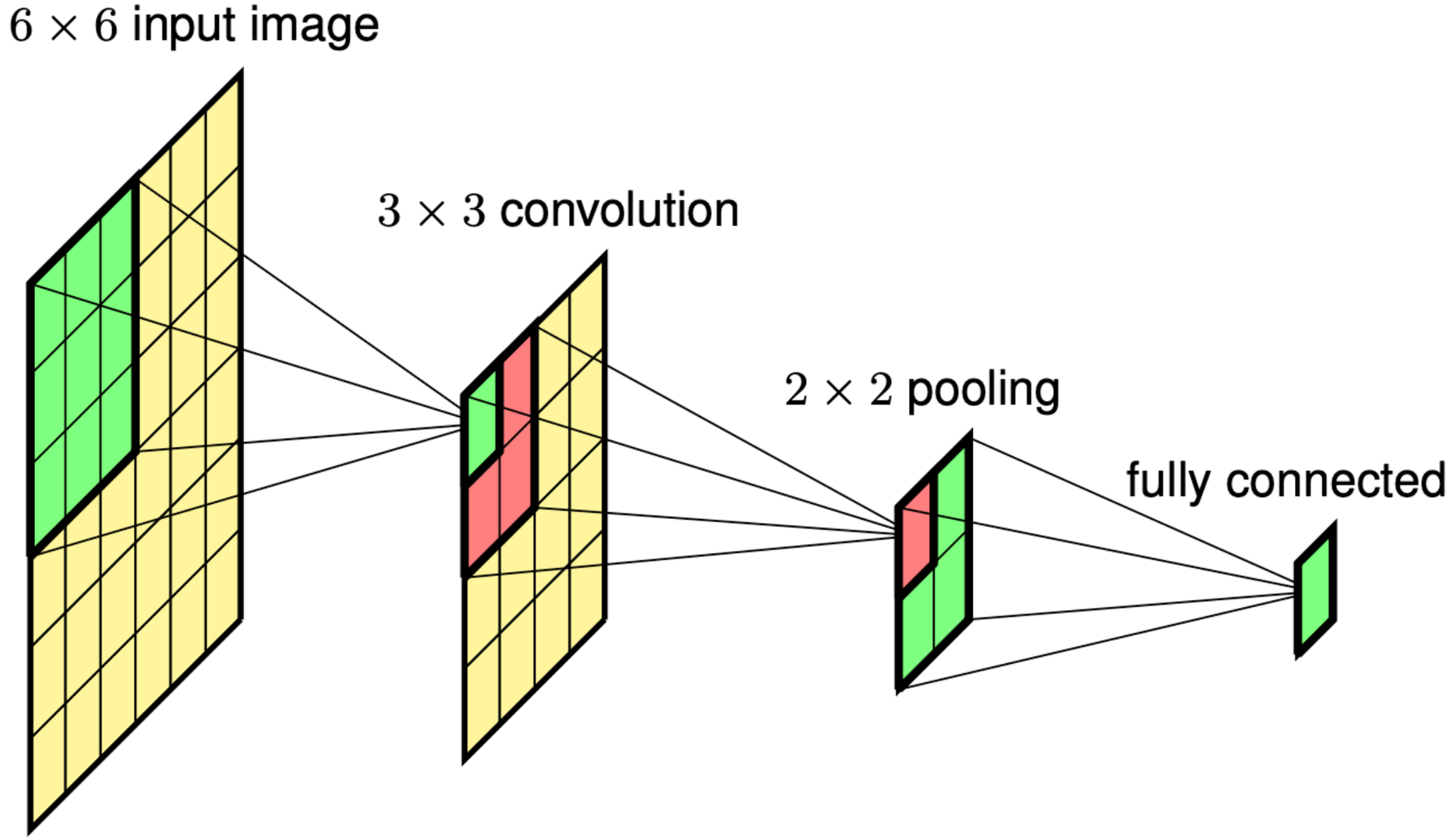
- "Sliding windows" approach
- Training set comprises:

1) Examples of cropped image regions of object to be detected
   - All crops should be of same size (or aspect ratio)
2) Examples of similarly-cropped image regions devoid of object

- Train CNN classifier to output probability of image region having object
- For test image, apply classifier to each region → probability map
- Can be efficiently implemented by a fully-convolutional DNN (see next slide)

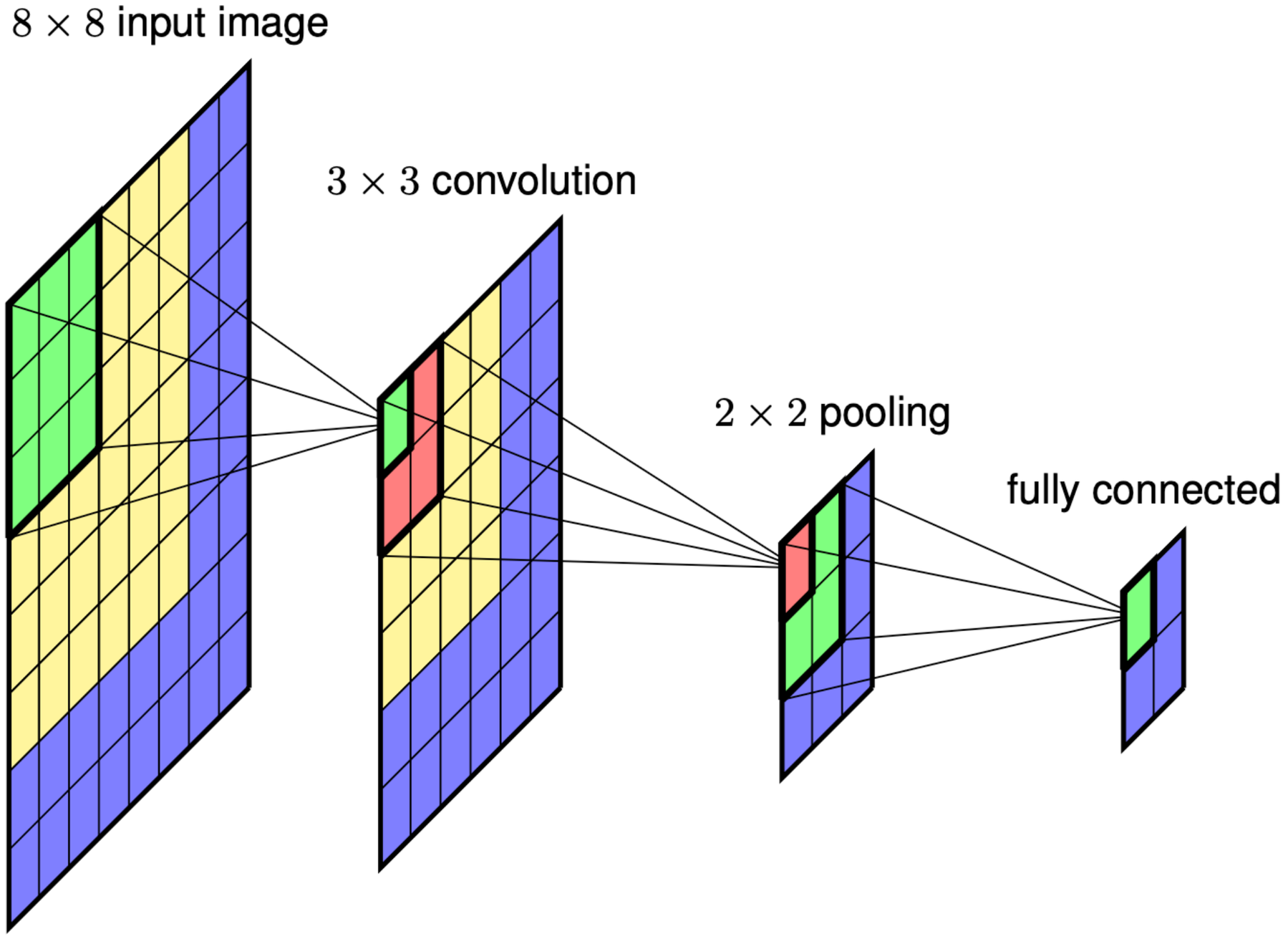
CNN: Object Detection

- "Sliding windows" example: training fully-conv CNN on 6x6 patches
- Implement last/"fully-connected" layer also as a convolution layer



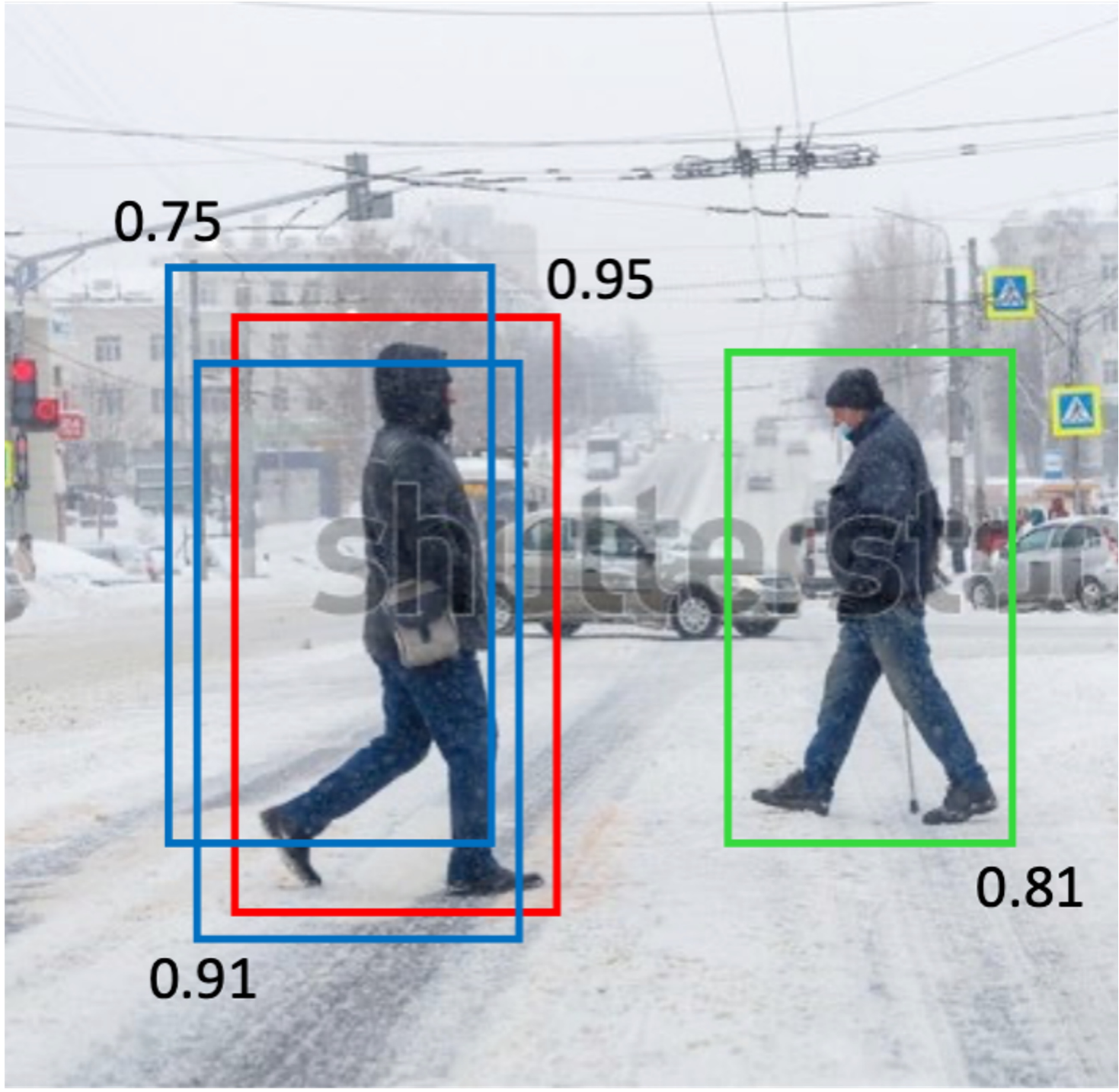
CNN: Object Detection

- "Sliding windows" example
- Process larger images (e.g., 8x8) with little computational overhead



CNN: Object Detection

- "Sliding windows" approach
- Can produce multiple detections of same object at similar locations
- Each box has an associated probability
- Use non-maximum suppression to select only 1 object from a set of overlapping box predictions



CNN: Object Detection

- YOLO
- 2015 IEEE CVPR

You Only Look Once: Unified, Real-Time Object Detection

Joseph Redmon*, Santosh Divvala*†, Ross Girshick¶, Ali Farhadi*†

University of Washington*, Allen Institute for AI†, Facebook AI Research¶

CNN: Object Detection

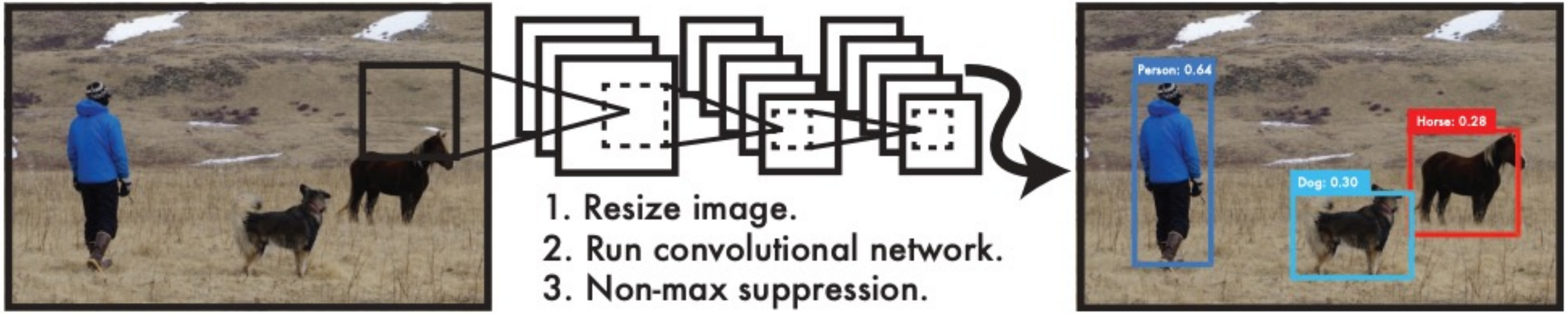


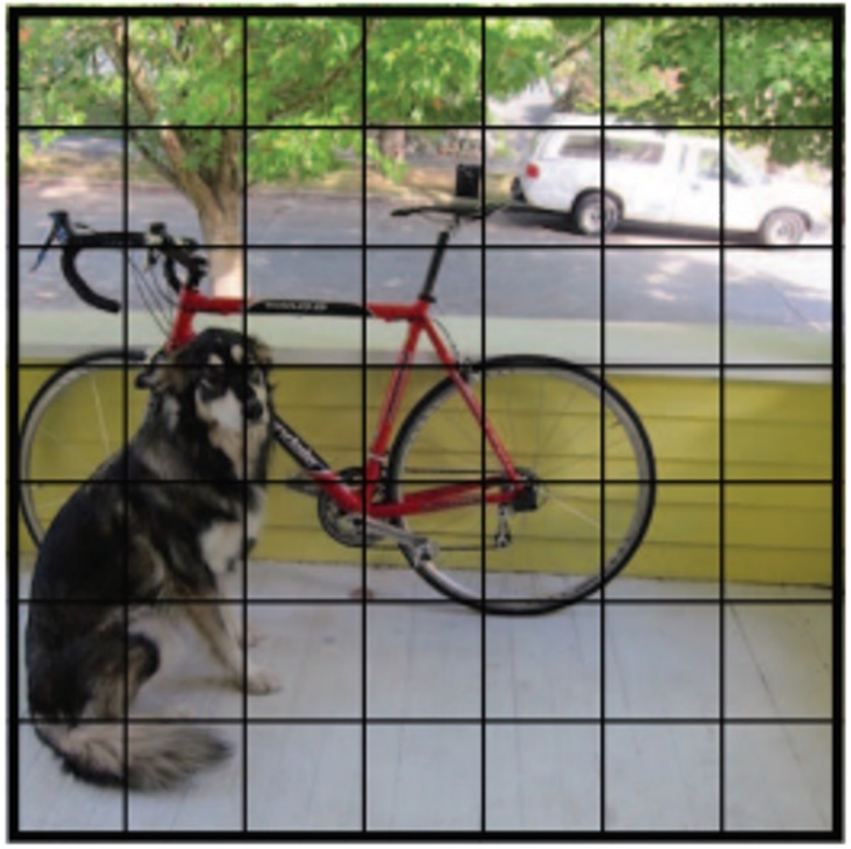
Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448, (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

CNN: Object Detection

- YOLO

1) Divide image into cells in SxS grid
2) For each cell in grid:
   output B bounding boxes
   (indicates possible locations of object)
   - Each bounding box associated with 5 predictions
     (x,y, w,h, confidence) …
   - … where confidence (they claimed) models
     weighted IOU between predicted box & any ground-truth box
     = P (Object) * IOU^{truth}_{pred}
3) For each cell in grid:
   output C class probabilities = P ($Class_i$ | Object observed in cell)

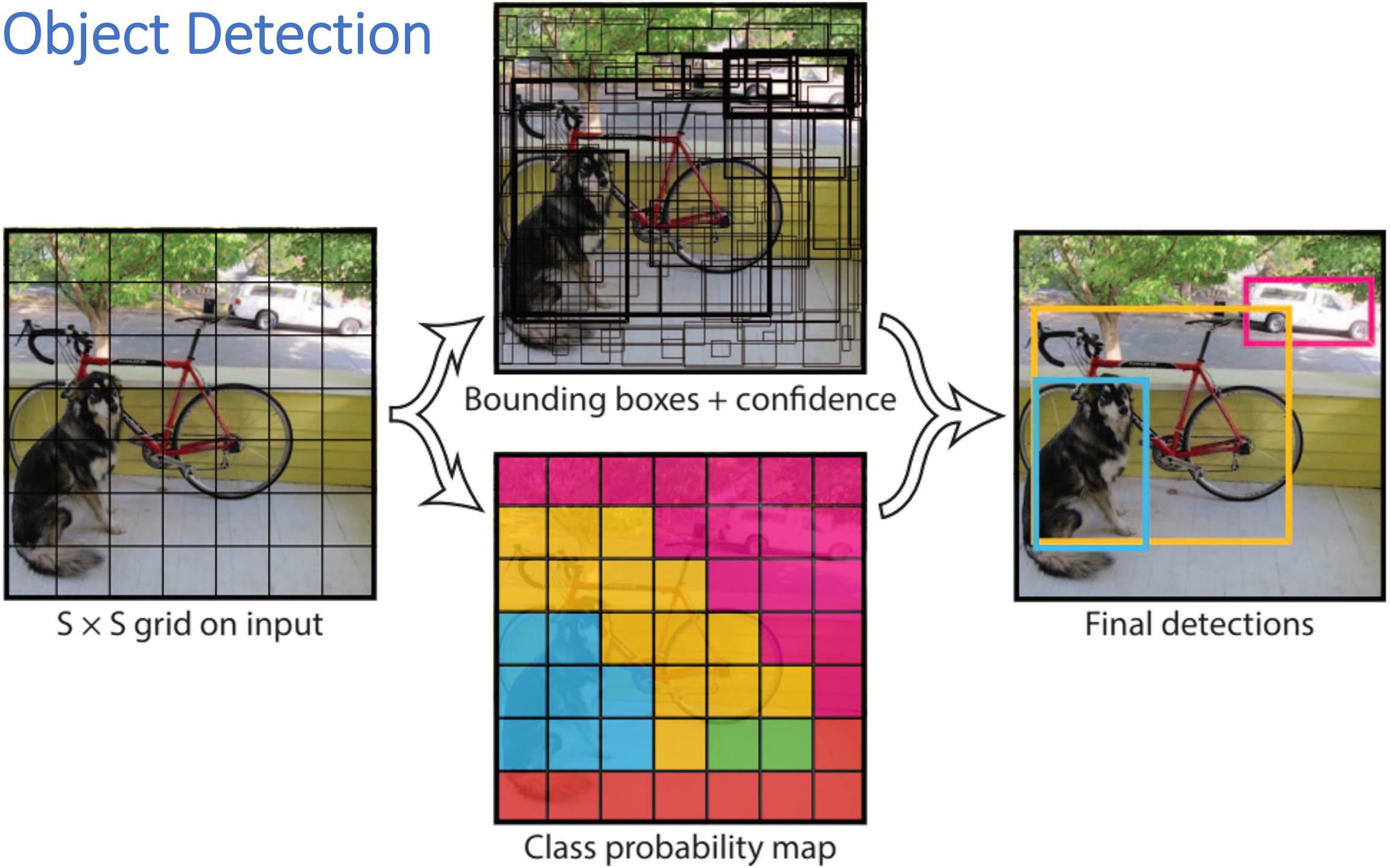
- Number of outputs per image: S*S * (5*B + C)



S × S grid on input

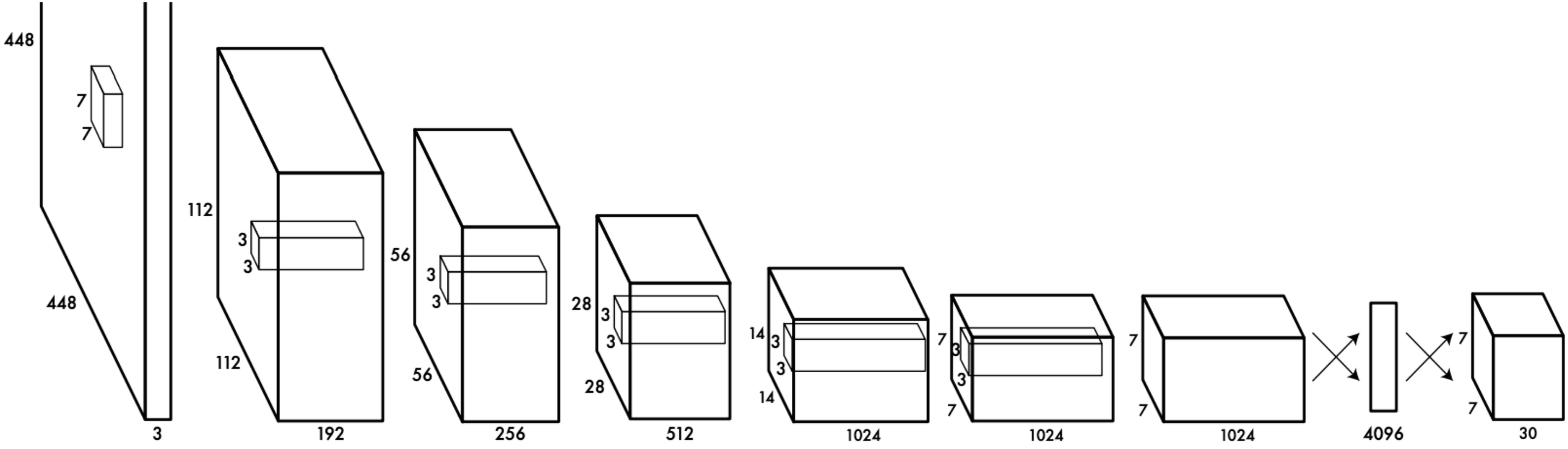
CNN: Object Detection

- YOLO



CNN: Object Detection

- YOLO
- SxS = 7x7 B = 2 C = 20 S*S * (5*B + C) = 7*7 * 30



Conv. Layer
7x7x64-s-2
Maxpool Layer
2x2-s-2

Conv. Layer
3x3x192
Maxpool Layer
2x2-s-2

Conv. Layers
1x1x128
3x3x256
1x1x256
3x3x512
Maxpool Layer
2x2-s-2

Conv. Layers
1x1x256
3x3x512
} ×4
1x1x512
3x3x1024
Maxpool Layer
2x2-s-2

Conv. Layers
1x1x512
3x3x1024
} ×2
3x3x1024
3x3x1024-s-2

Conv. Layers
3x3x1024
3x3x1024

Conn. Layer

Conn. Layer

CNN: Object Detection

- YOLO predicts multiple (B) bounding boxes per grid cell
- During training,
  from all predicted boxes within a grid cell,
  YOLO wants to assign
  (at most) 1 predicted box
  to be "responsible" for each object
- If it exists,
  this is the one used to compute loss, and update DNN parameters
- During each training iteration/epoch,
  choose this 1 predictor as one having
  highest current IOU with ground truth
- [Training uses dropout, lots of data augmentation]

CNN: Object Detection

- YOLO's loss

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_{ij})^2 + (y_i - \hat{y}_{ij})^2 \right]$$

- 1_{ij}^{obj} indicates if object appears in cell i, j^{th} bounding box predictor in cell i is "responsible" for prediction →

$$+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_{ij}} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_{ij}} \right)^2 \right]$$

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_{ij} \right)^2$$

- 1_{ij}^{noobj} indicates if object absent in cell i: if 1, then confidence $C_i := 0$ →

$$+ \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_{ij} \right)^2$$

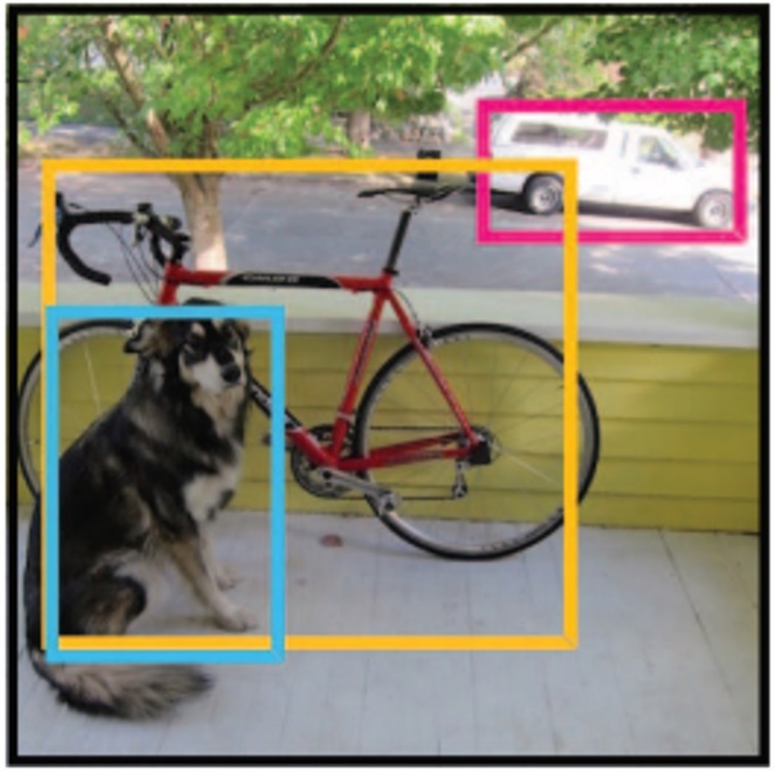
- 1_{i}^{obj} indicates if any object appears in cell i →

$$+ \sum_{i=0}^{S^2} \mathbb{1}_{i}^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

CNN: Object Detection

- At test time, **class-specific confidence scores for each box** (for a cell) = conditional **class probabilities** [$P(Class_i \mid Object)$] (for cell) * individual box **confidence** prediction [$P(Object) * IOU^{truth}_{pred}$] (for cell)

- **YOLO**



Final detections

$$\Pr(\text{Class}_i \mid \text{Object}) * \Pr(\text{Object}) * \text{IOU}^{\text{truth}}_{\text{pred}} = \Pr(\text{Class}_i) * \text{IOU}^{\text{truth}}_{\text{pred}}$$

CNN: Object Detection

- 2025 NeurIPS

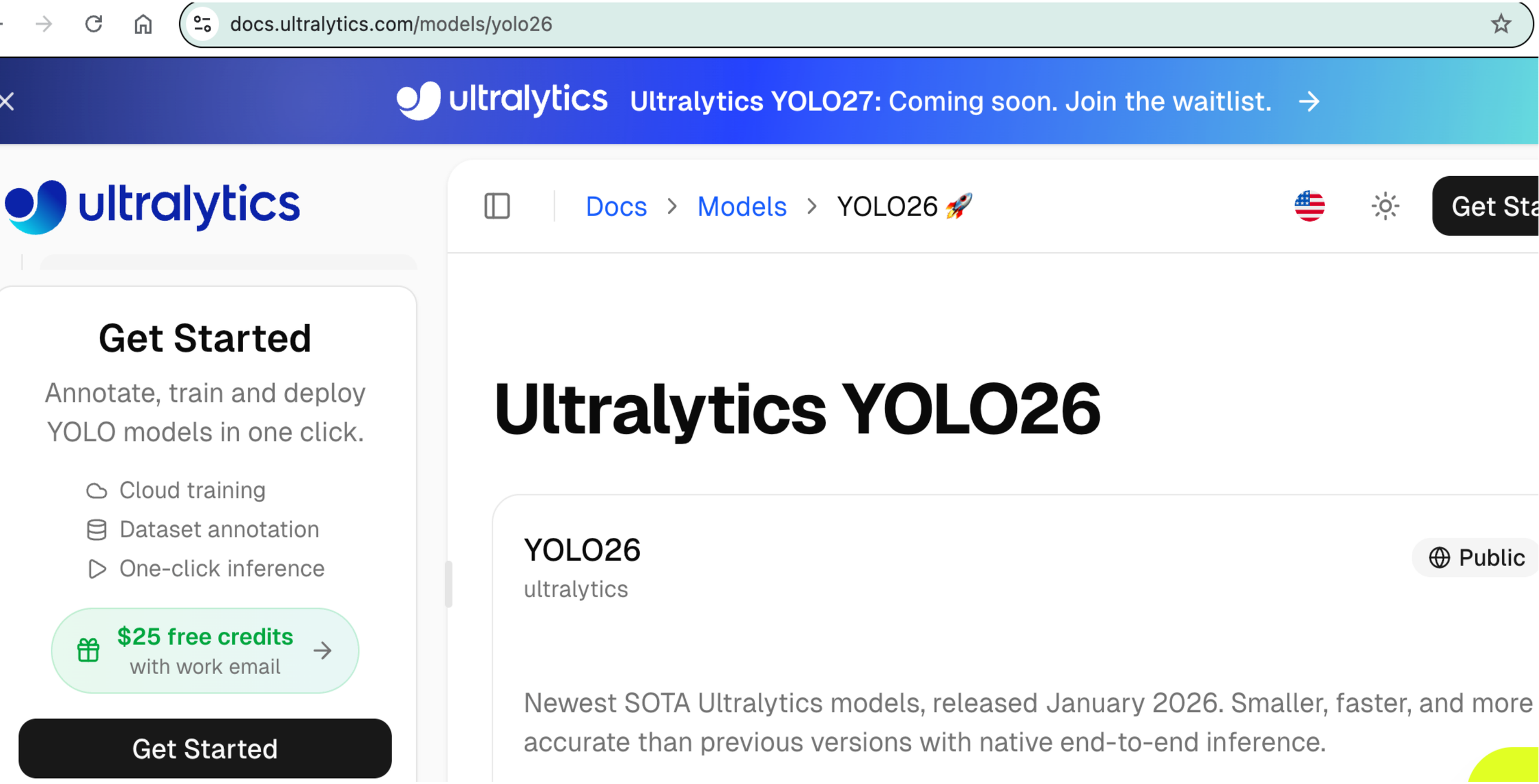
YOLOv12

YOLOv12: Attention-Centric Real-Time Object Detectors

Yunjie Tian[1], Qixiang Ye[2], David Doermann[1]

[1] University at Buffalo, SUNY, [2] University of Chinese Academy of Sciences.

CNN: Object Detection



CNN: Object Detection

- docs.ultralytics.com/guides/yolo-architecture
- arxiv.org/abs/2602.14582

YOLO26: A Comprehensive Architecture Overview and Key Improvements

A PREPRINT

Priyanto Hidayatullah[a*], Refdinal Tubagus[b],
[a] Computer Engineering and Informatics Department, Politeknik Negeri Bandung, Kab. Bandung Barat, Indonesia
[b] Stunning Vision AI, Kota Cimahi, Indonesia
priyanto@polban.ac.id[a], refdinal@stunningvisionai.com[b]

Priyanto Hidayatullah
Refdinal Tubagus

Model variant	d (depth_multiple)	w (width_multiple)	mc (max_channels)
n	0.50	0.25	1024
s	0.50	0.50	1024
m	0.50	1.00	512
l	1.00	1.00	512
xl	1.00	1.50	512

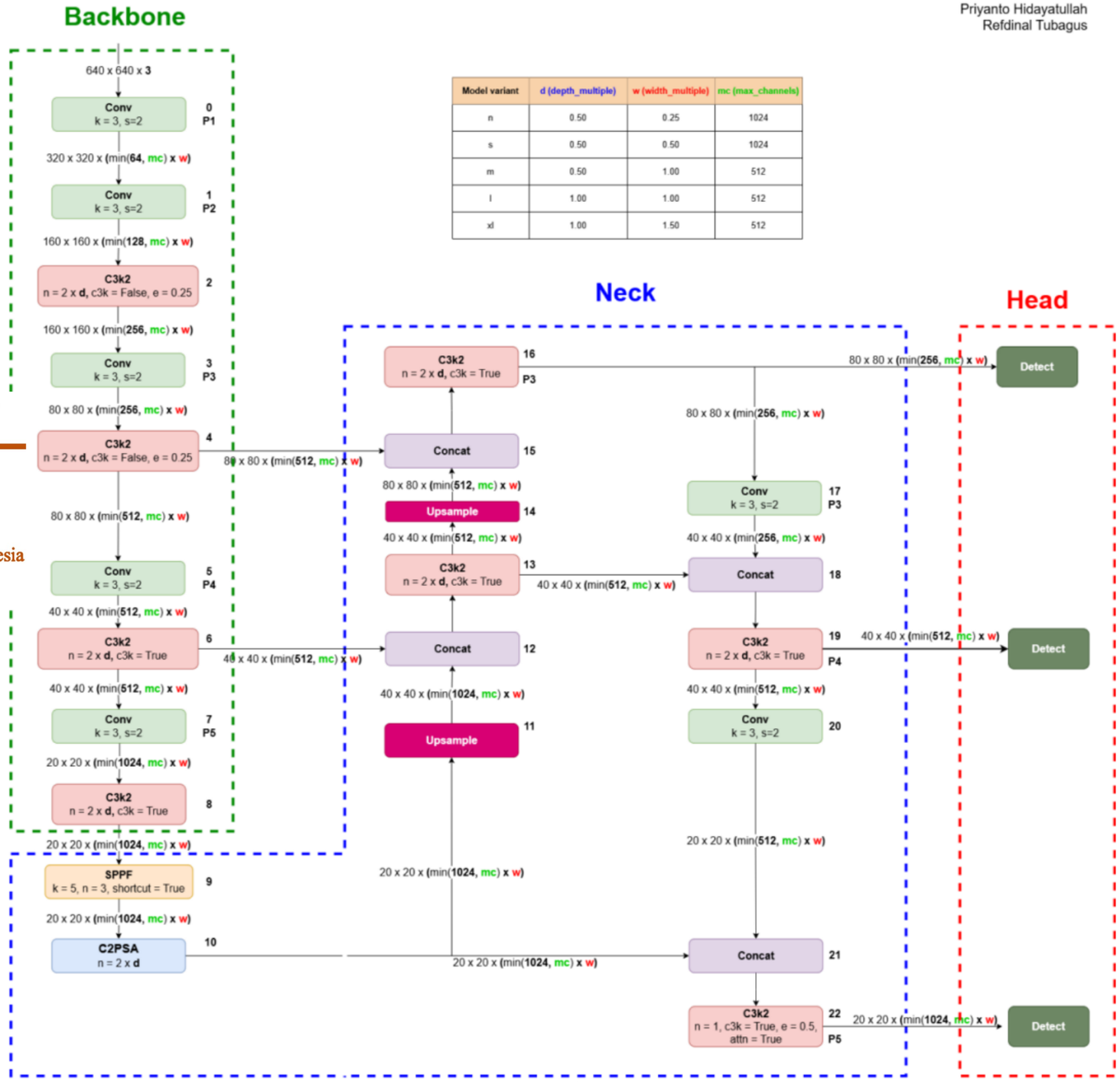
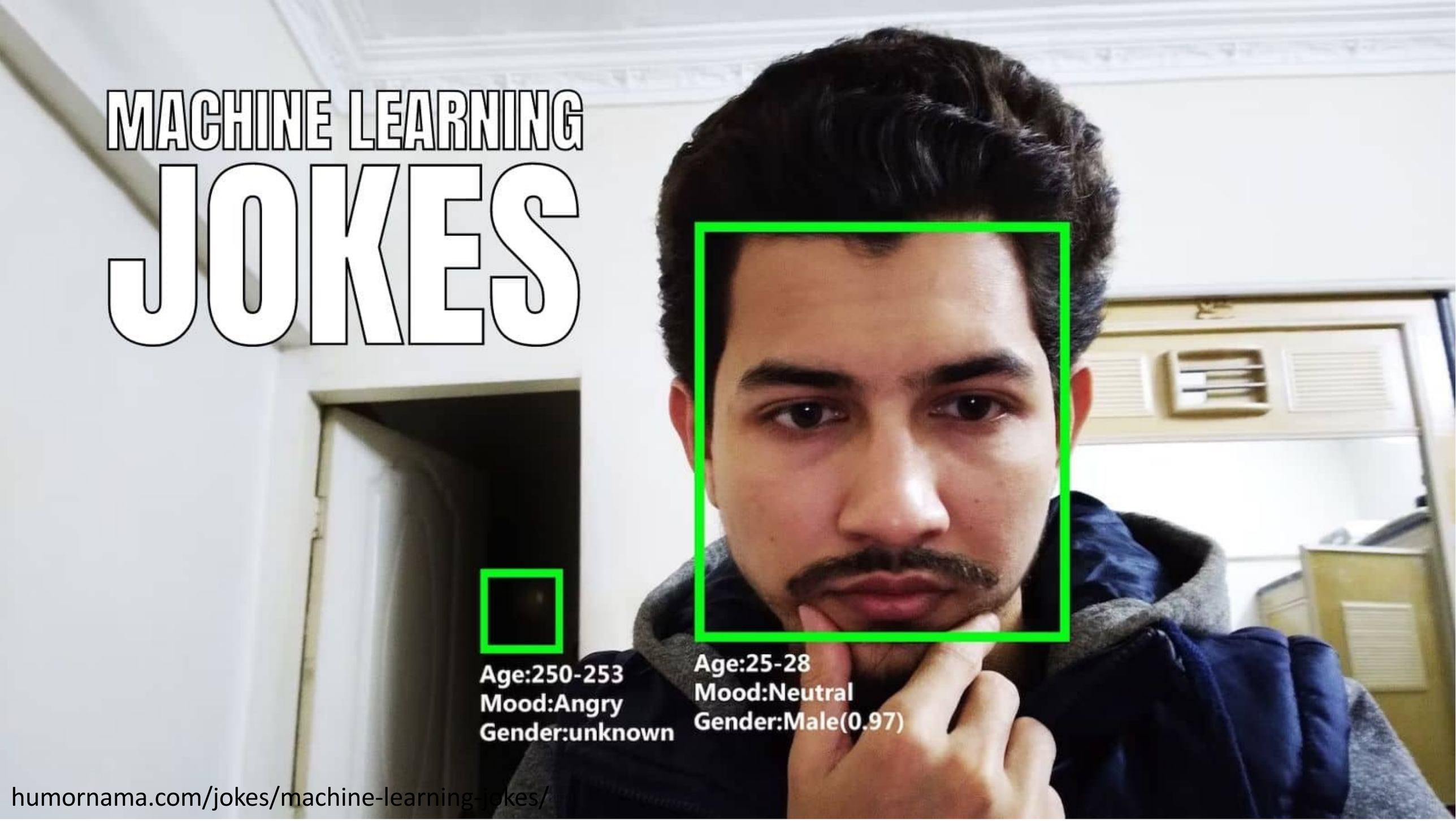


Figure 1: YOLO26 Architecture Diagram



humornama.com/jokes/machine-learning-jokes/

DNN: Image Segmentation

- Image classification → predict class-label probabilities for entire image
- Object detection in images → within an image,
  predict object locations (using bounding boxes) + object classes
- "Semantic" segmentation of image → for every pixel in image,
  predict its probability of being in a class

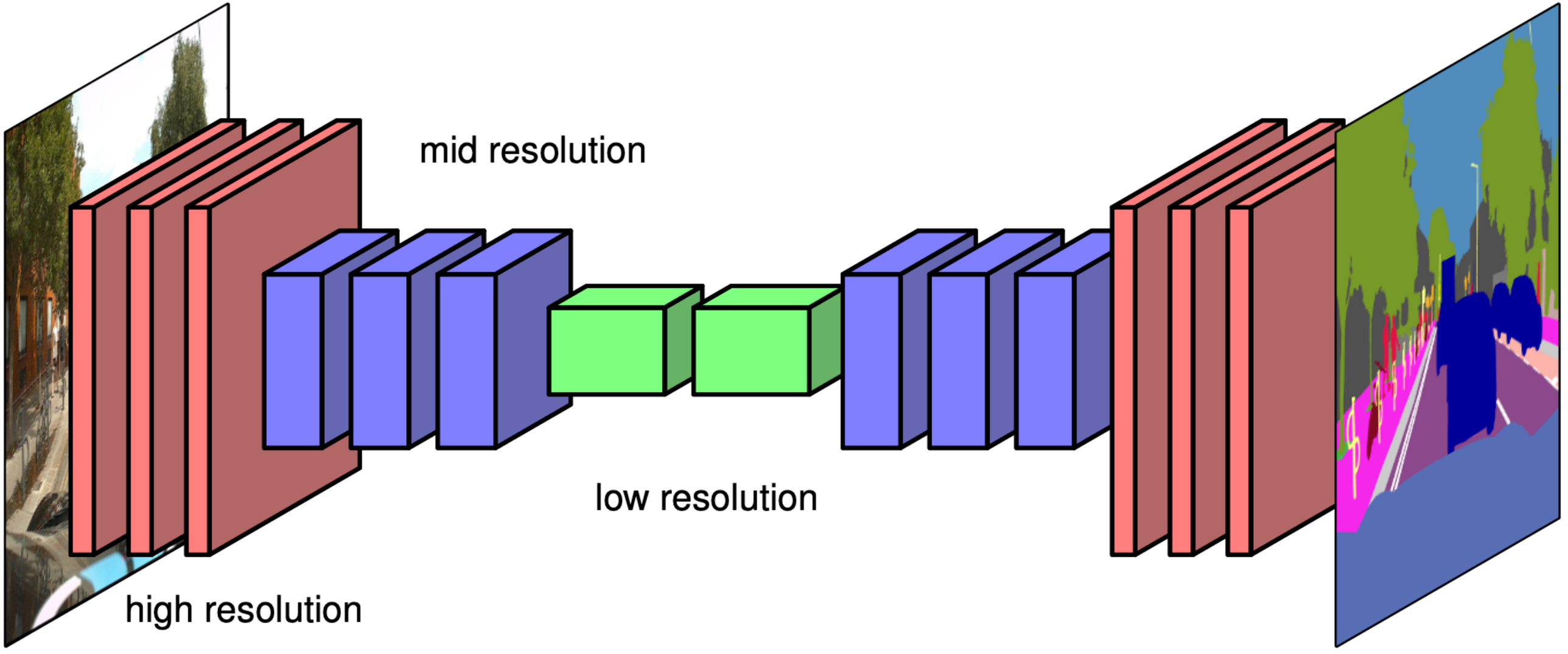
DNN: Image Segmentation

- Can use sliding-window approach to train CNN classifier for each patch
  - But this will limit size of receptive field to patch size
  - Too small patch size → too less context
  - Too large patch size → less number of training examples
- Typically, DNNs for segmentations use CNN having:

1) Encoder using **convolution** layers + pooling
2) Decoder using **convolution** layers + upsampling ("un-pooling")
   - Decoder maps encoding
     (smaller spatial extent than input image; with K channels)
     back to size of input image
     (with C channels)

DNN: Image Segmentation

- CNN having encoder-decoder architecture

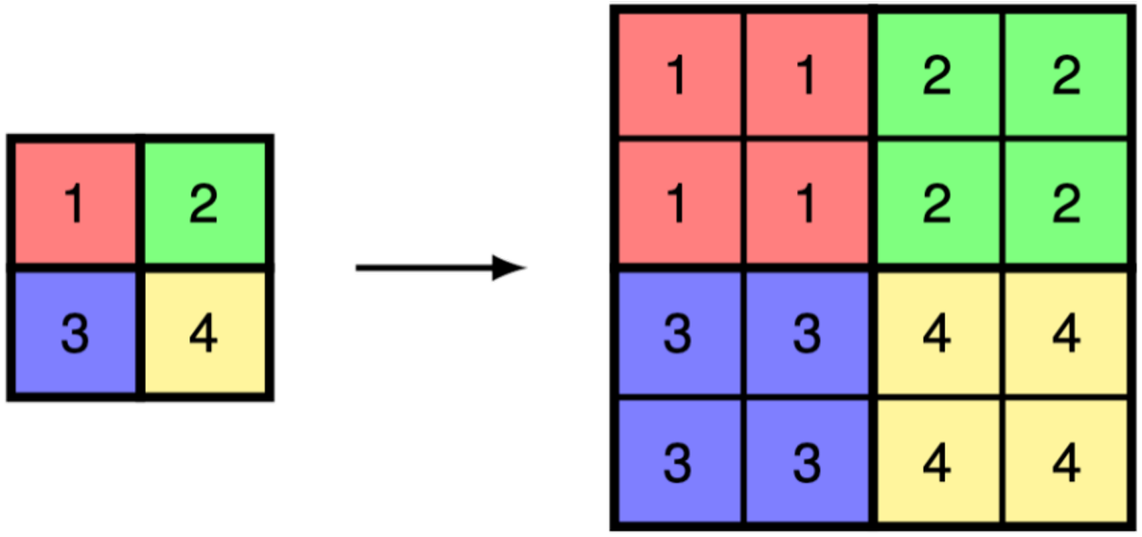


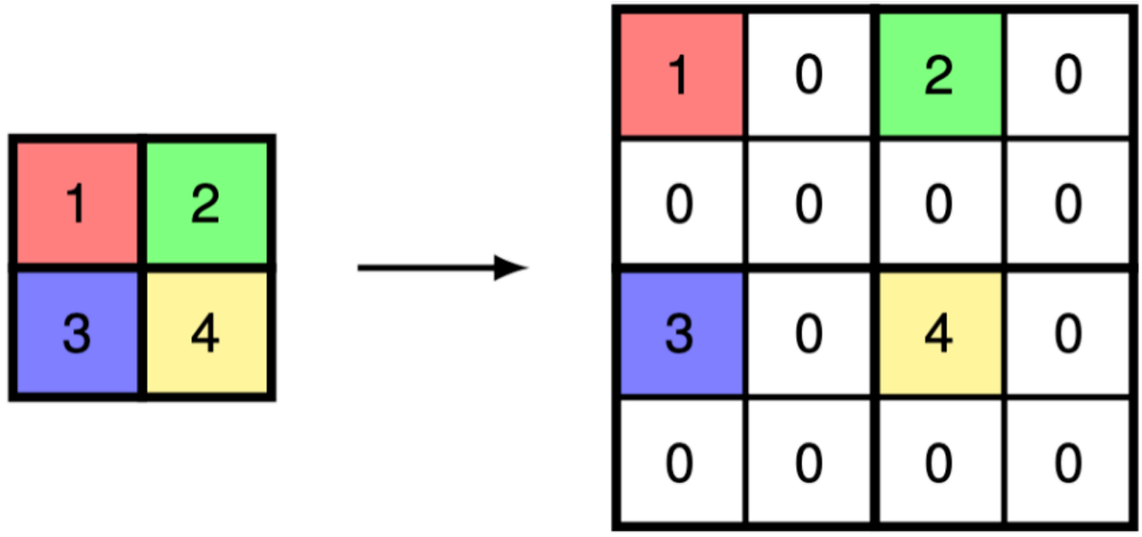
DNN: Image Segmentation

- Un-pooling: basic strategies

1) un-pooling average pooling
2) un-pooling max pooling

- These strategies can be modeled using a fixed linear/matrix mapping (or “convolution” kernel)

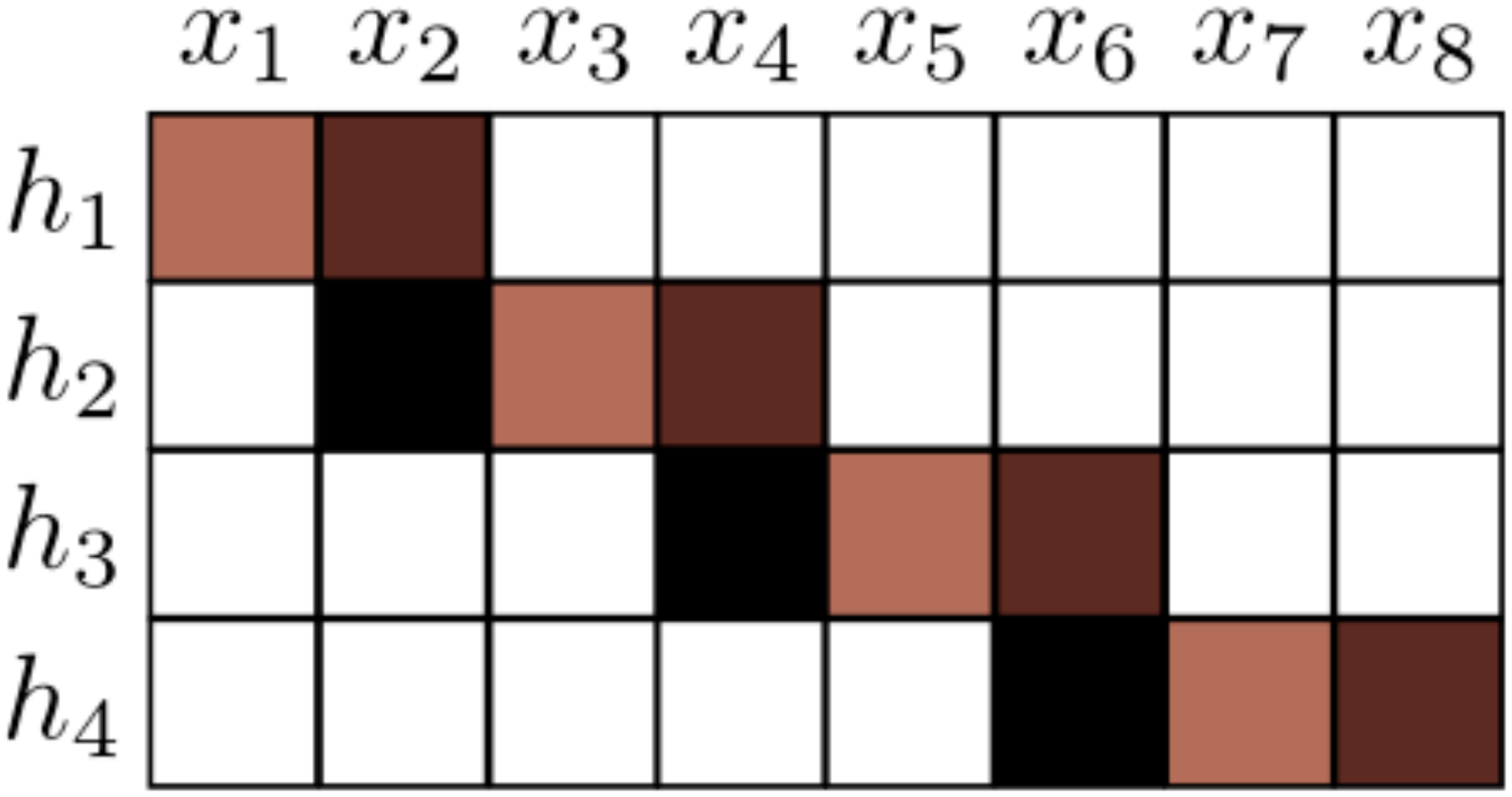


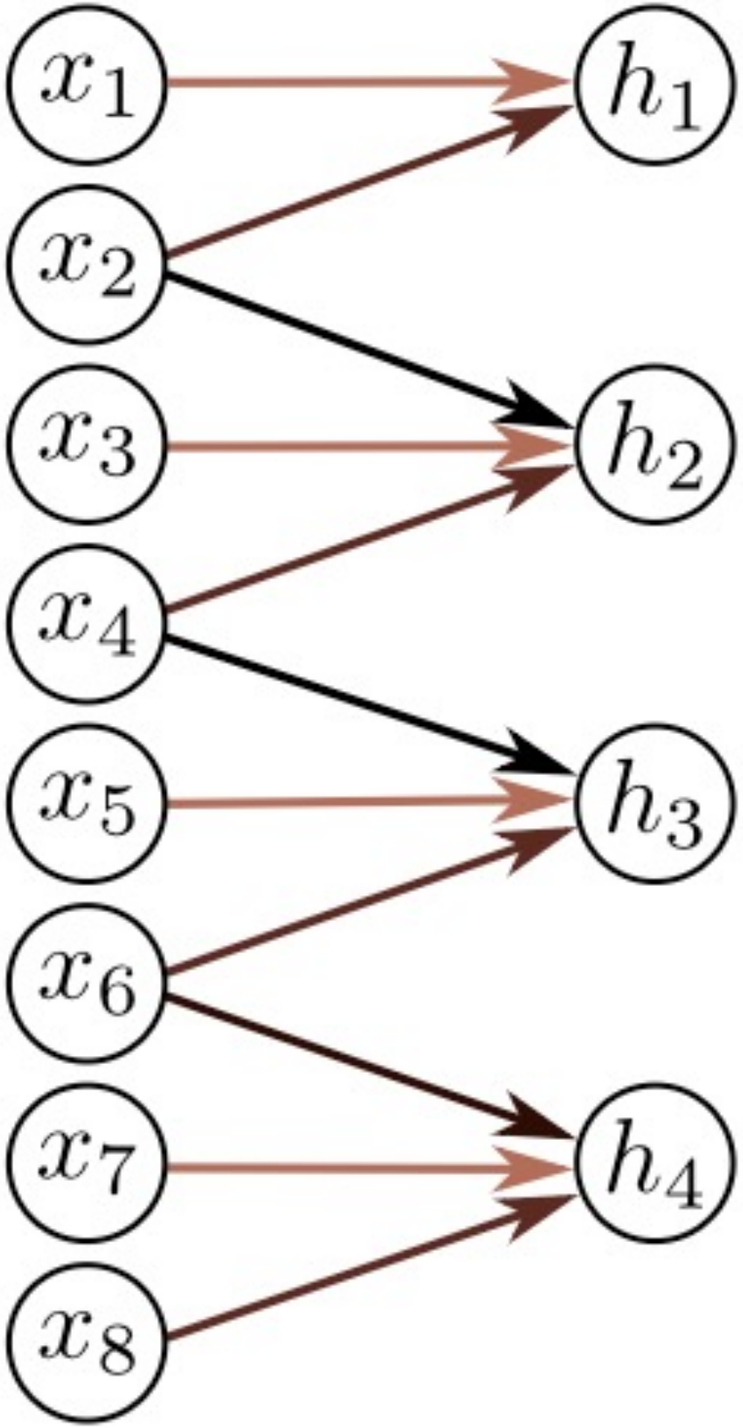


- Can also learn to perform “(bi)linear” interpolation

DNN: Image Segmentation

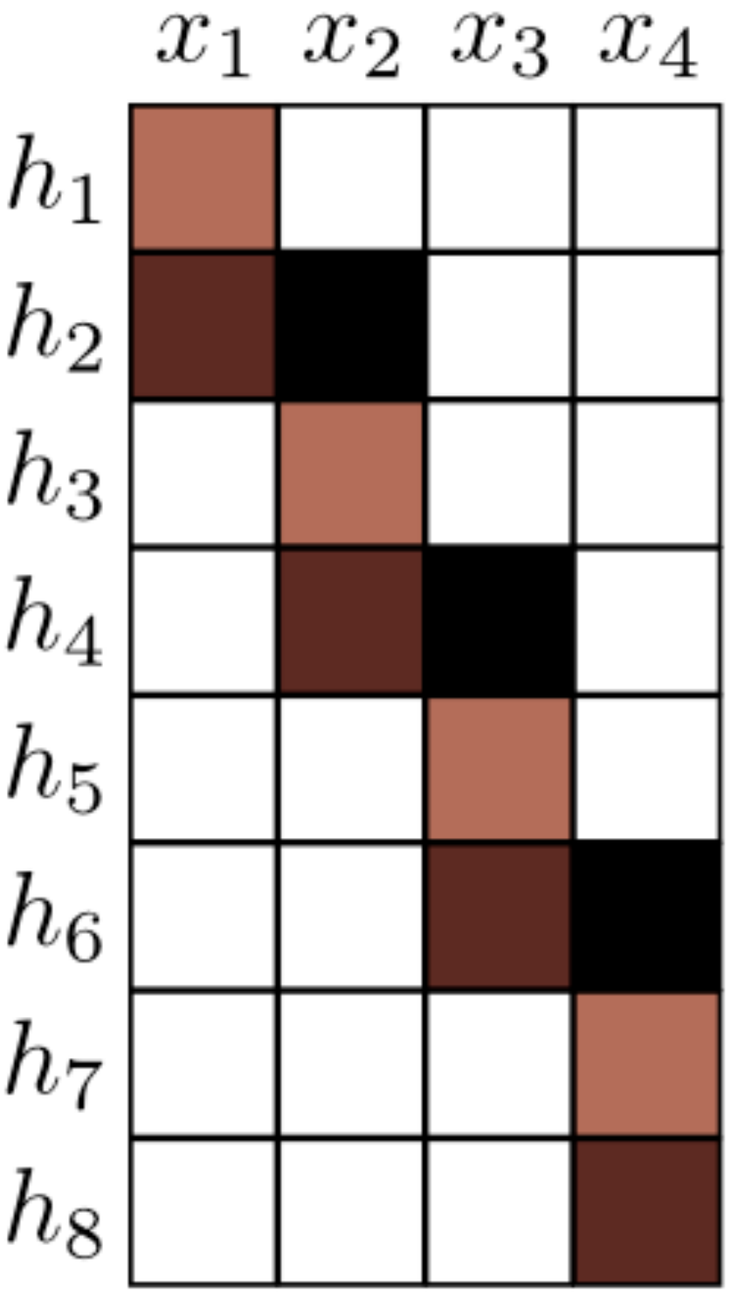
- An interpretation of pooling as linear transformation
- Pooling maps input vector to smaller vector

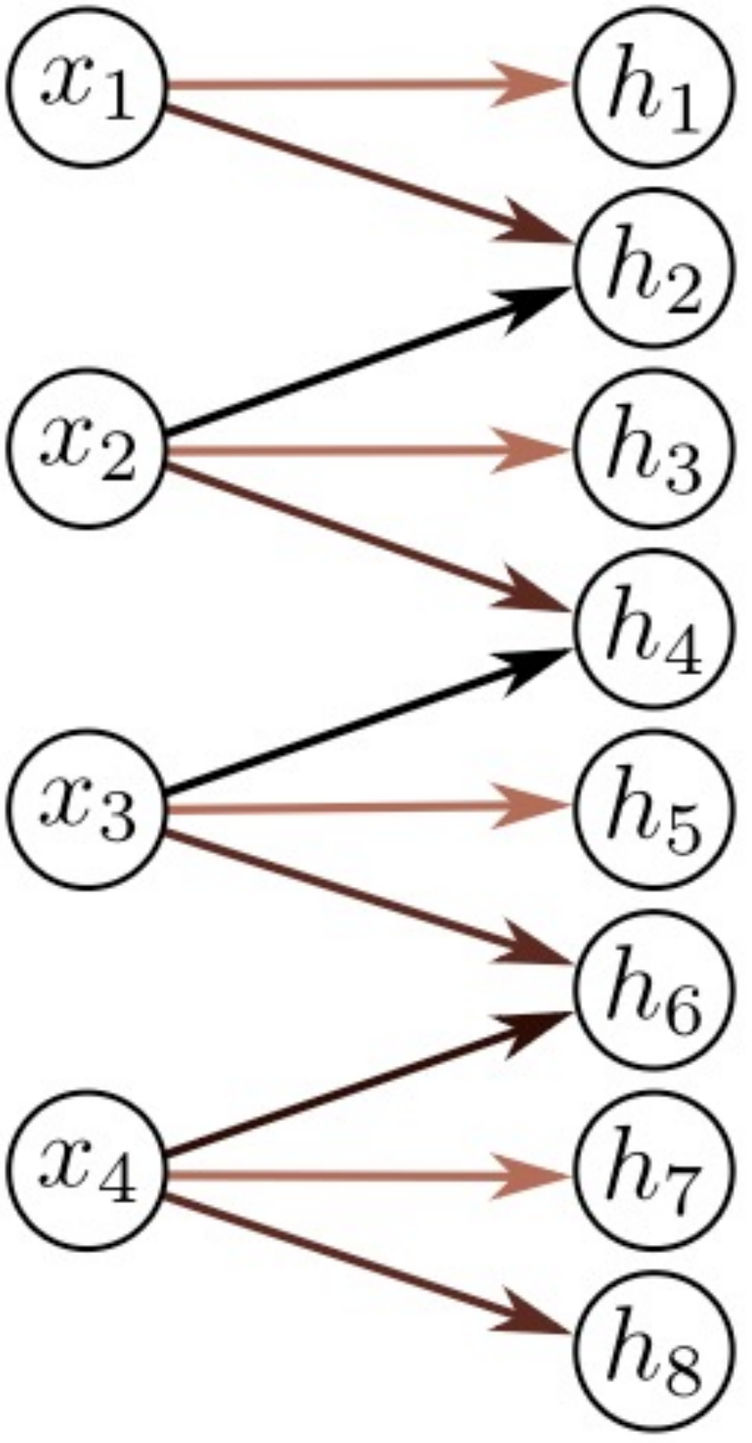




DNN: Image Segmentation

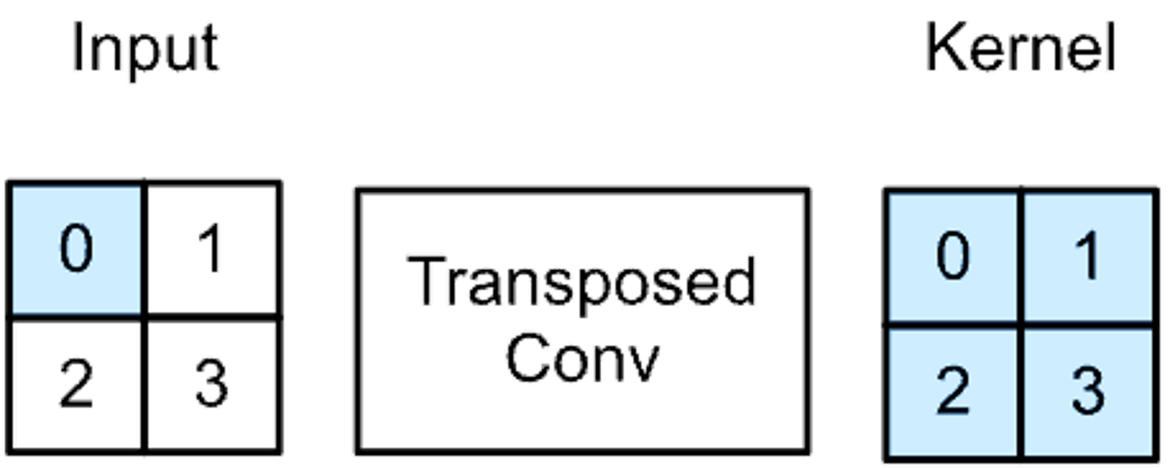
- Matrix size in un-pooling is transpose of that of matrix size in pooling
- Pictures show un-pooling as linear transformation; analogous to previous example



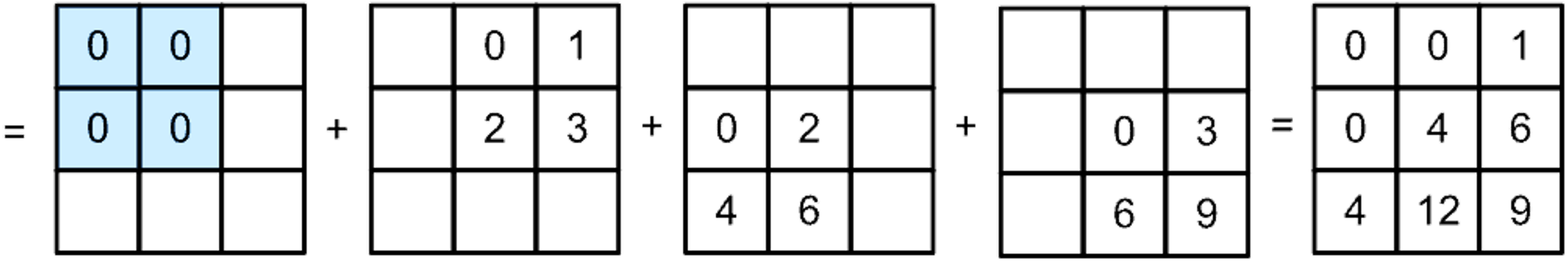


DNN: Image Segmentation

- Another interpretation: upsampling as "flipped"/"transpose" "convolution" where kernel is learned
- An example with "output stride" = 1



d2l.ai/chapter_computer-vision/transposed-conv.html



DNN: Image Segmentation

- Another example →
- Upsampling as "flipped"/"transpose" "convolution" with "output stride" = 2
- Kernel is learned

Input

0	1
2	3

Transposed Conv (stride 2)

Kernel

0	1
2	3

=

0	0		
0	0		

+

		0	1
		2	3

+

0	2		
4	6		

+

		0	3
		6	9

=

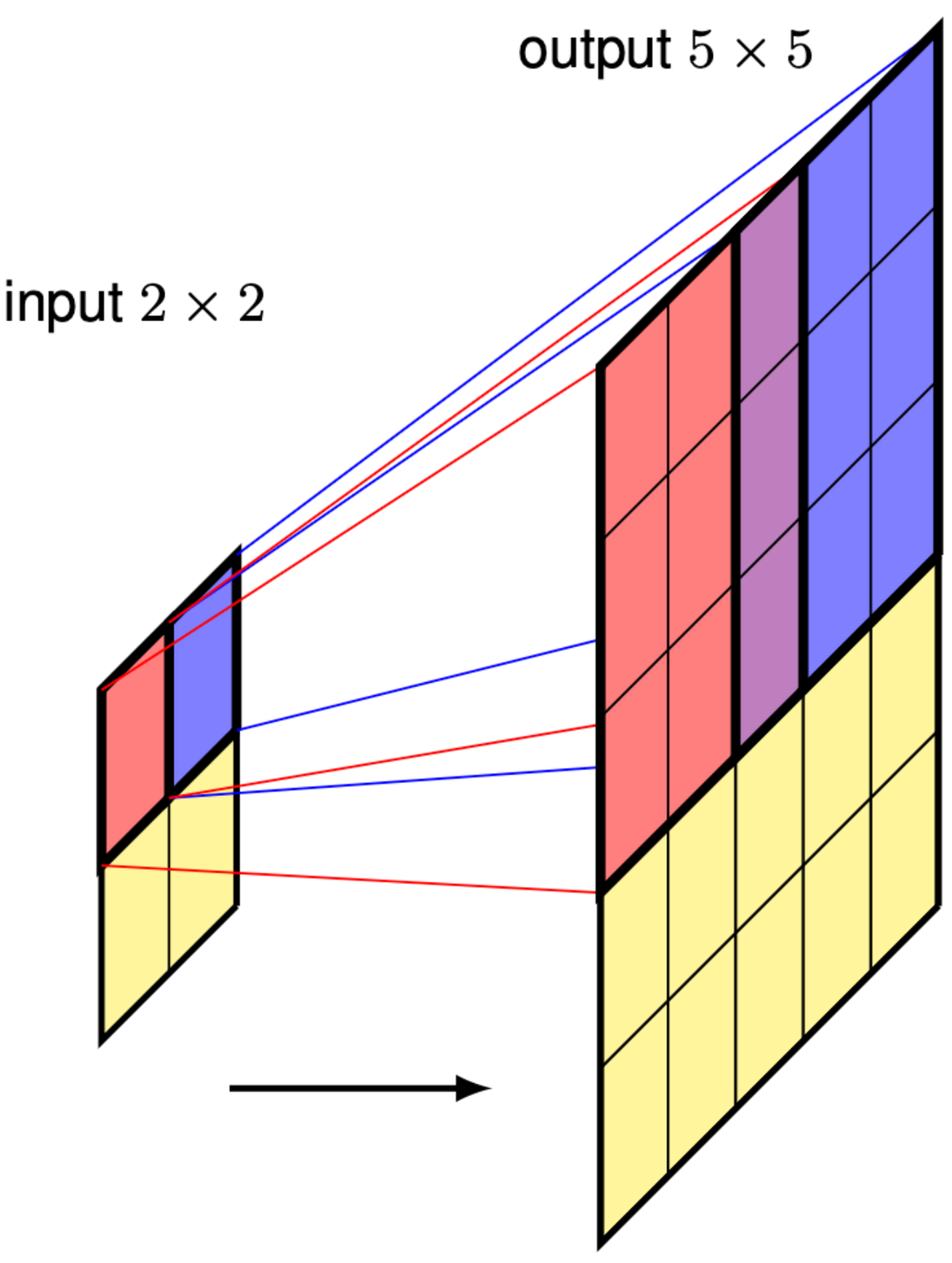
0	0	0	1
0	0	2	3
0	2	0	3
4	6	6	9

Output

d2l.ai/chapter_computer-vision/transposed-conv.html

DNN: Image Segmentation

- Another example →
- Upsampling as "flipped"/"transposed" convolution
- **3x3 filter**
- "Output stride" = 2



DNN: Image Segmentation

- Fully convolutional networks
- 2015 IEEE CVPR

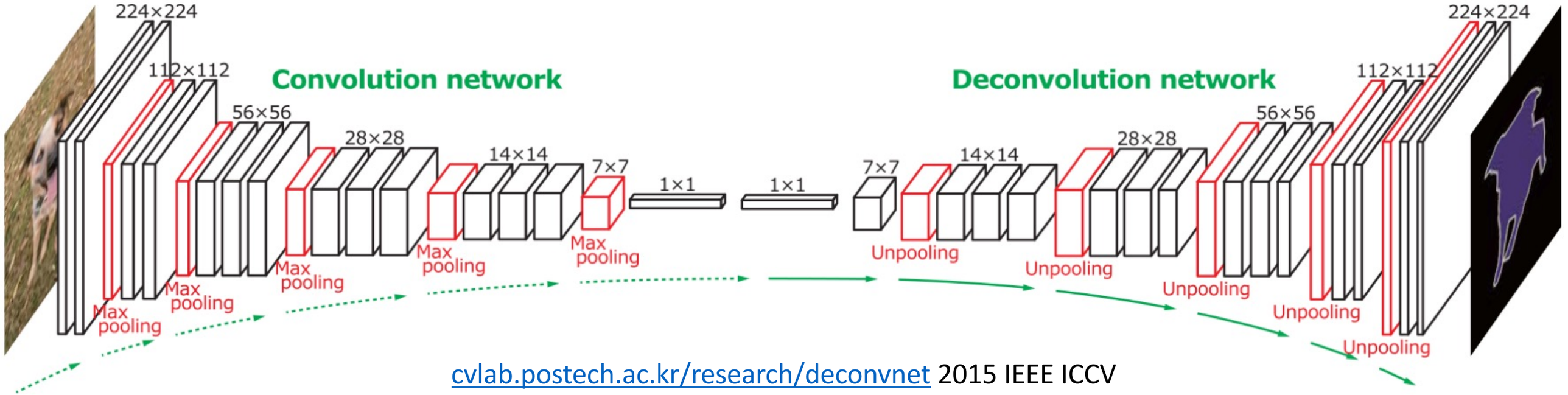
Fully Convolutional Networks for Semantic Segmentation

Jonathan Long* Evan Shelhamer* Trevor Darrell
UC Berkeley
{jonlong,shelhamer,trevor}@cs.berkeley.edu

DNN: Image Segmentation

Learning Deconvolution Network for Semantic Segmentation

Hyeonwoo Noh Seunghoon Hong Bohyung Han
POSTECH



cvlab.postech.ac.kr/research/deconvnet 2015 IEEE ICCV

DNN: Image Segmentation

- In a typical FCN, encoder uses multiple pooling layers that decrease spatial resolution in encoding
- Loss of spatial information is problem for semantic segmentation as we want to classify each pixel
- U-Net: 2015 international conference on Medical Image Computing & Computer-Assisted Intervention (MICCAI)

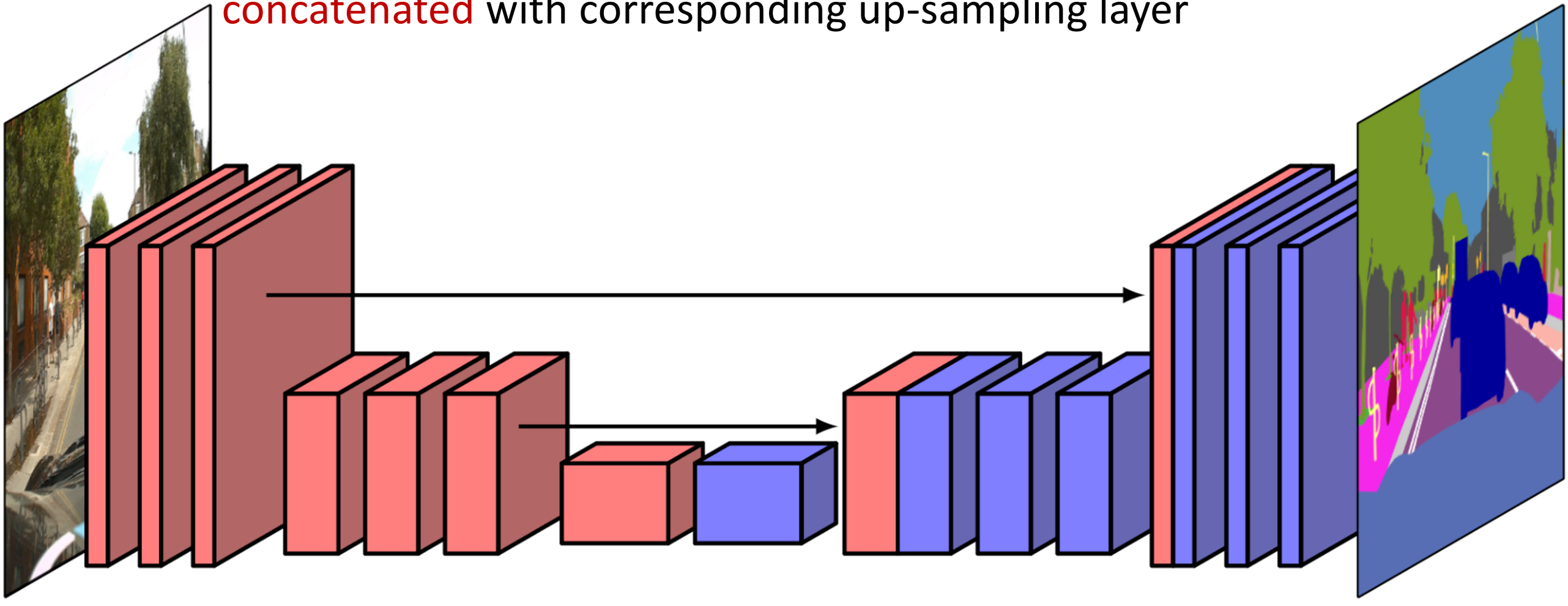
U-Net: Convolutional Networks for Biomedical Image Segmentation

Olaf Ronneberger, Philipp Fischer, and Thomas Brox

Computer Science Department and BIOSS Centre for Biological Signalling Studies,
University of Freiburg, Germany

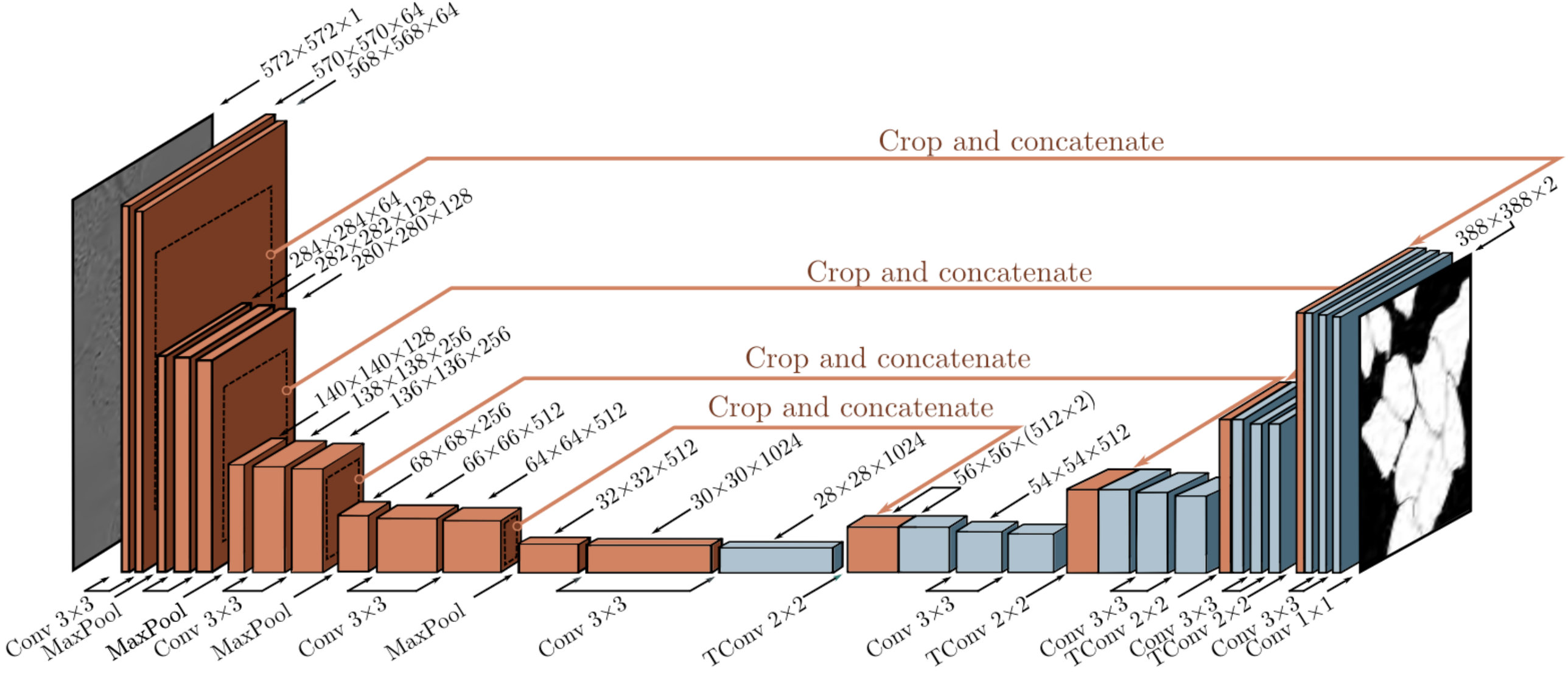
DNN: Image Segmentation

- U-Net [en.wikipedia.org/wiki/U-Net]
  - Symmetrical arrangement of down-sampling and up-sampling layers
  - Output from each down-sampling layer is concatenated with corresponding up-sampling layer



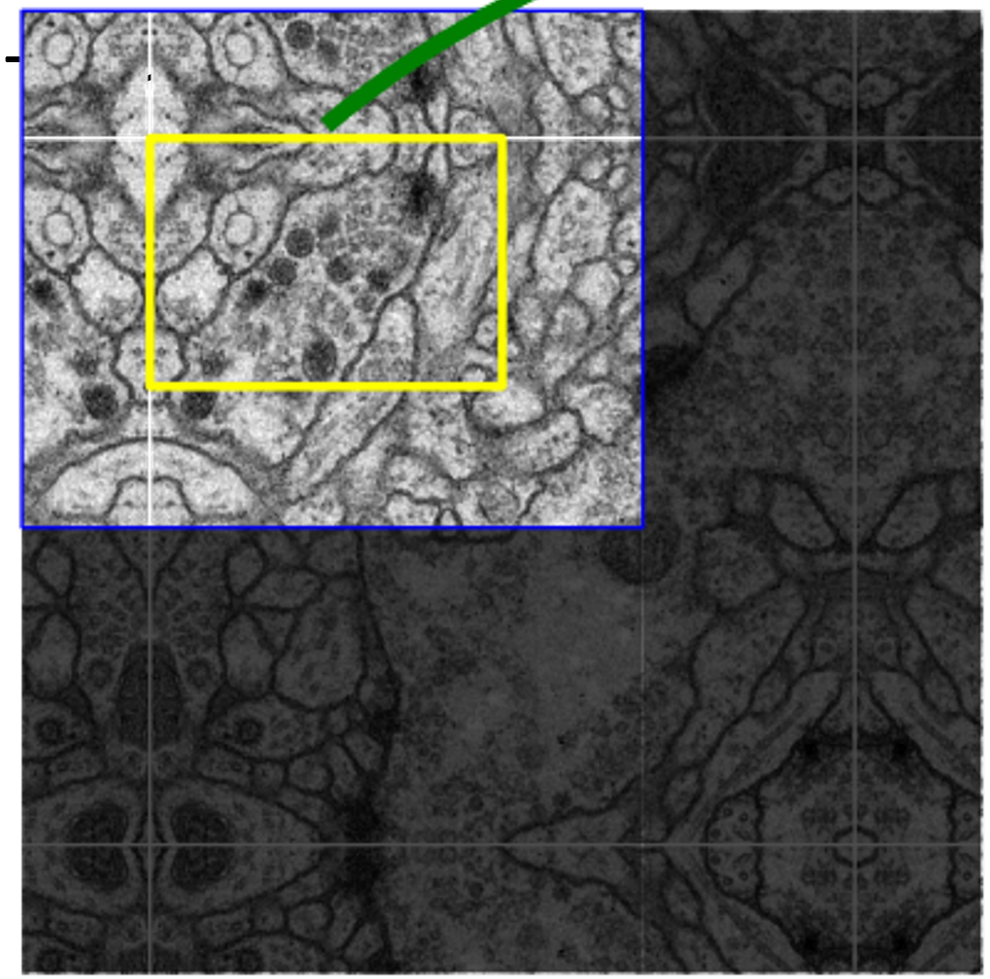
DNN: Image Segmentation

- Original U-Net [2015] used “valid” convolutions → size decreased a bit with each conv layer, even without downsampling



DNN: Image Segmentation

- U-



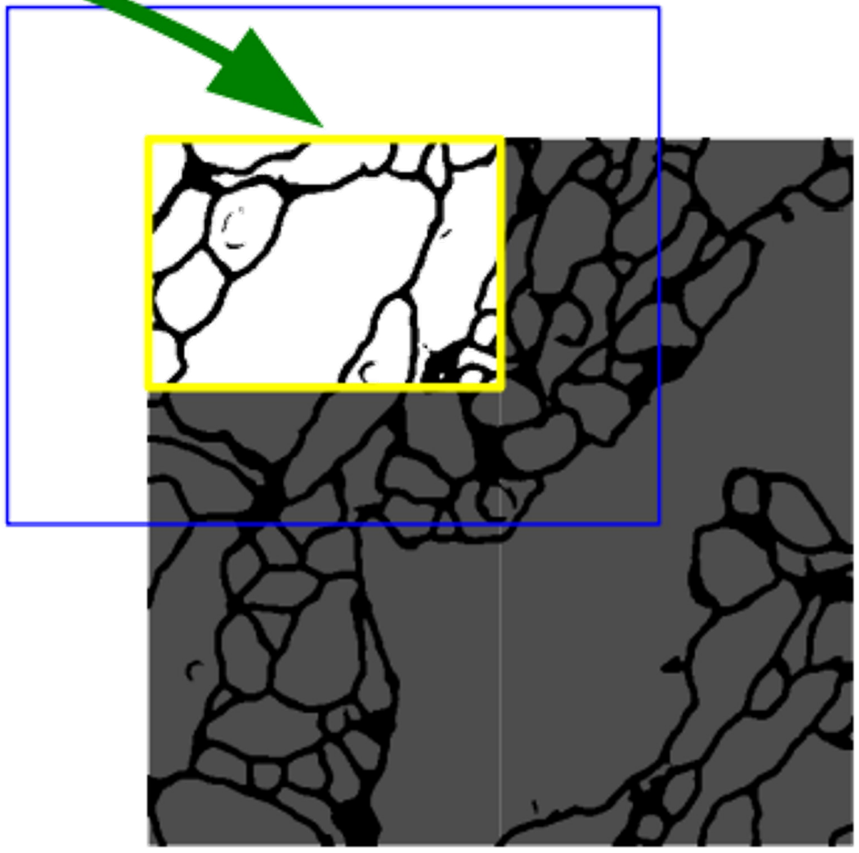


Fig. 2. Overlap-tile strategy for seamless segmentation of arbitrary large images (here segmentation of neuronal structures in EM stacks). Prediction of the segmentation in the yellow area, requires image data within the blue area as input. Missing input data is extrapolated by mirroring

DNN: Image Segmentation

- U-Net

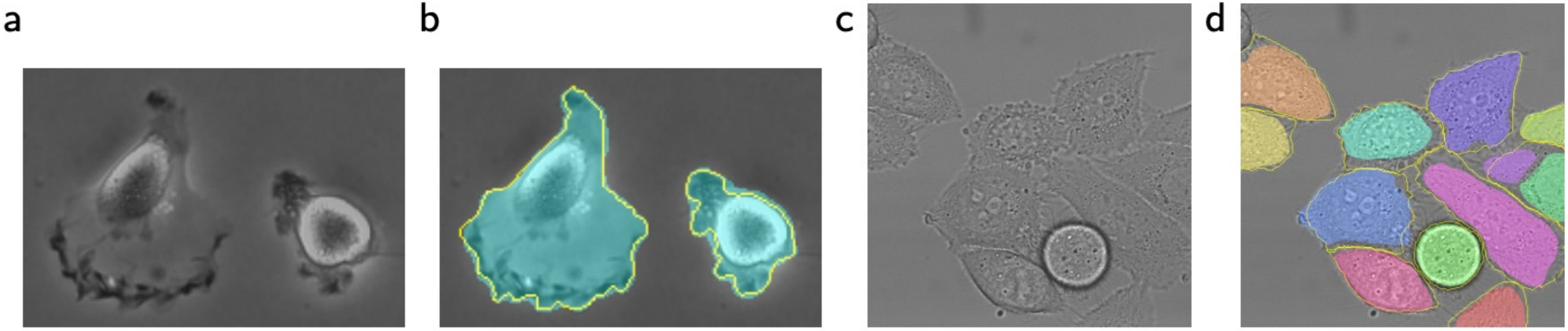


Fig. 4. Result on the ISBI cell tracking challenge. (**a**) part of an input image of the "PhC-U373" data set. (**b**) Segmentation result (cyan mask) with manual ground truth (yellow border) (**c**) input image of the "DIC-HeLa" data set. (**d**) Segmentation result (random colored masks) with manual ground truth (yellow border).

DNN: Image Segmentation

- U-Net today comes in various flavors of implementation
- Today, we also have various extensions of U-Net

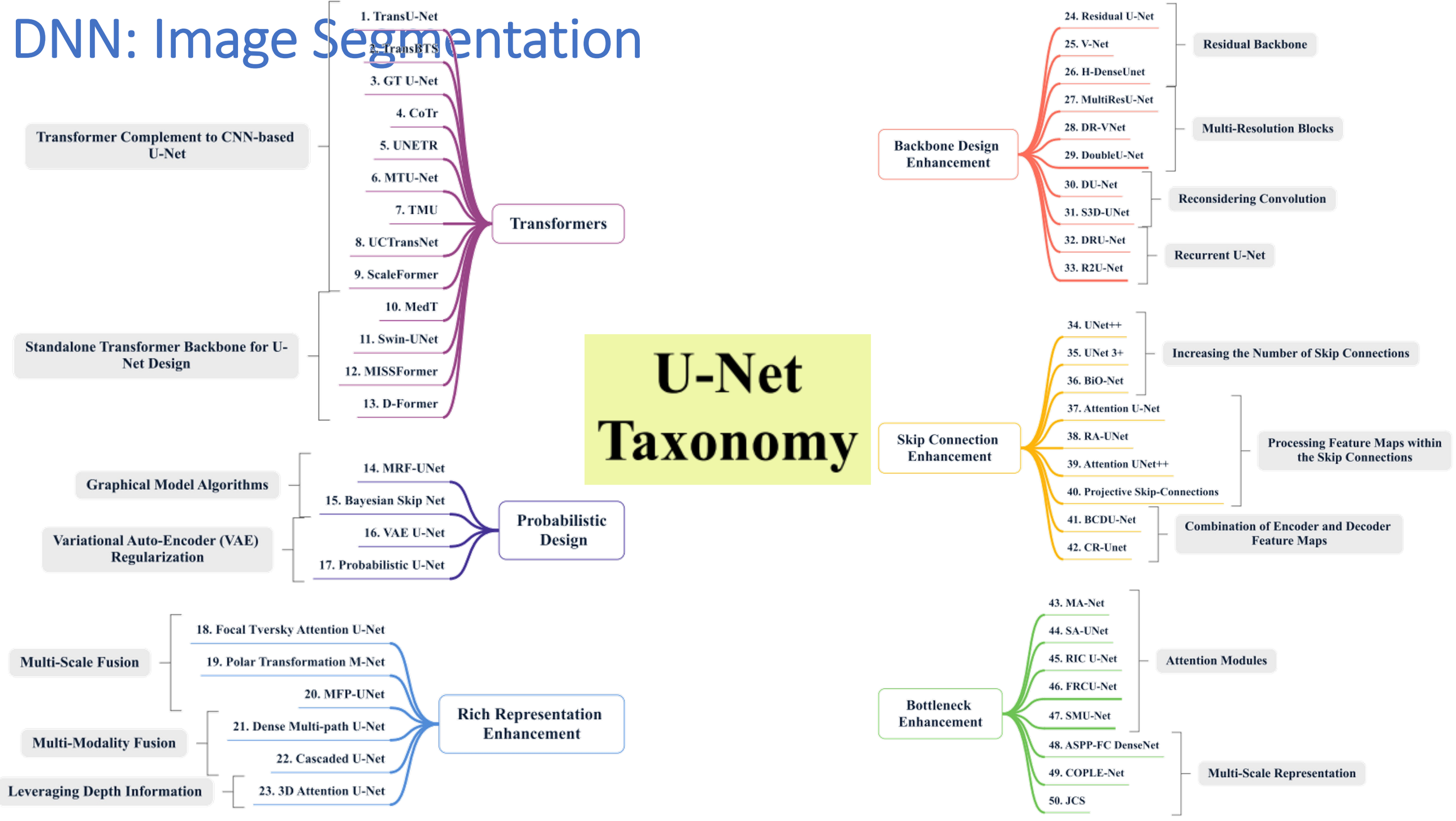
10076 IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, VOL. 46, NO. 12, DECEMBER 2024

Medical Image Segmentation Review: The Success of U-Net

Reza Azad, Ehsan Khodapanah Aghdam, Amelie Rauland, Yiwei Jia, Atlas Haddadi Avval, Afshin Bozorgpour, Sanaz Karimijafarbigloo, Joseph Paul Cohen, Ehsan Adeli, and Dorit Merhof

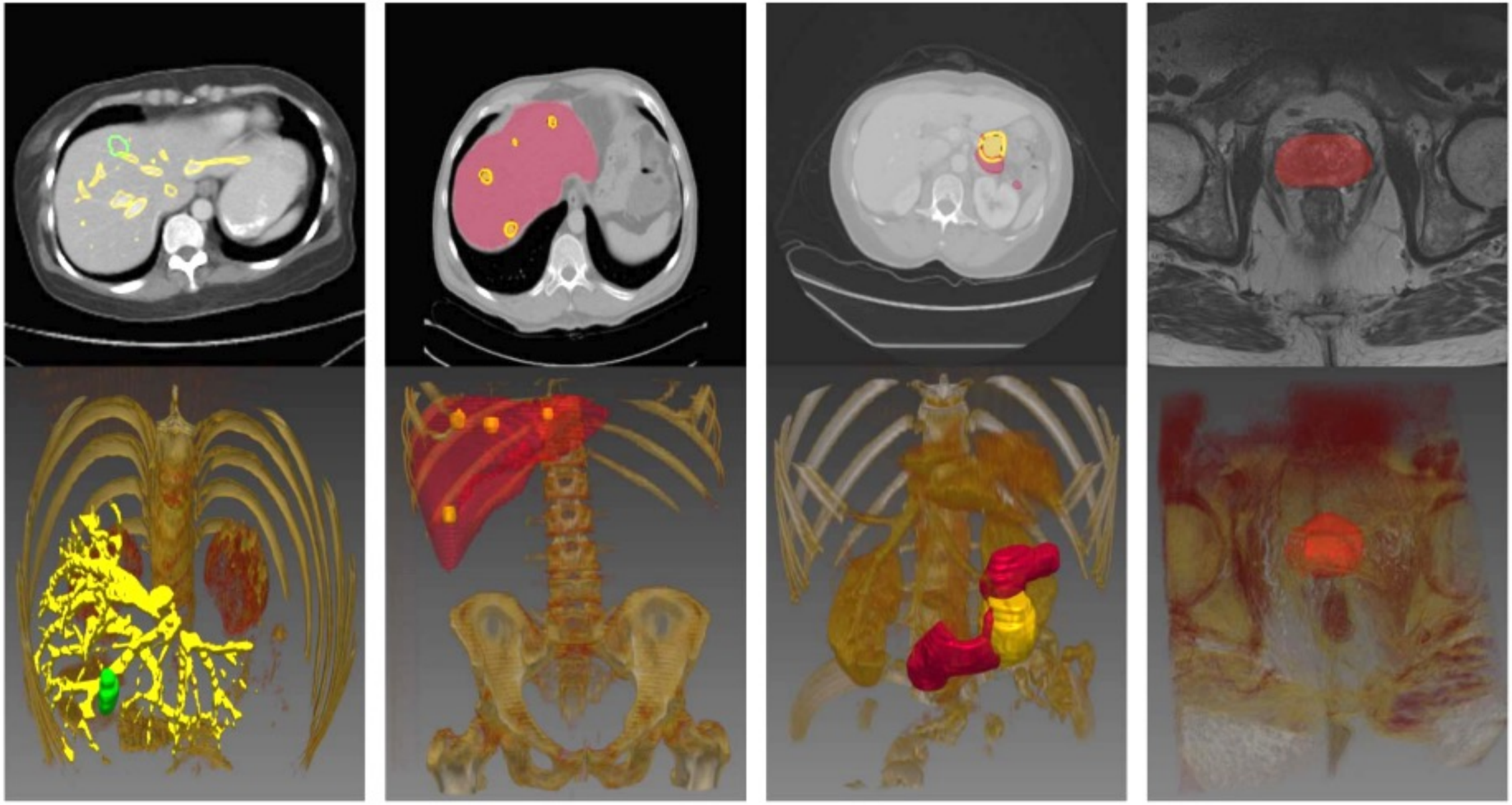
(Survey Paper)

DNN: Image Segmentation



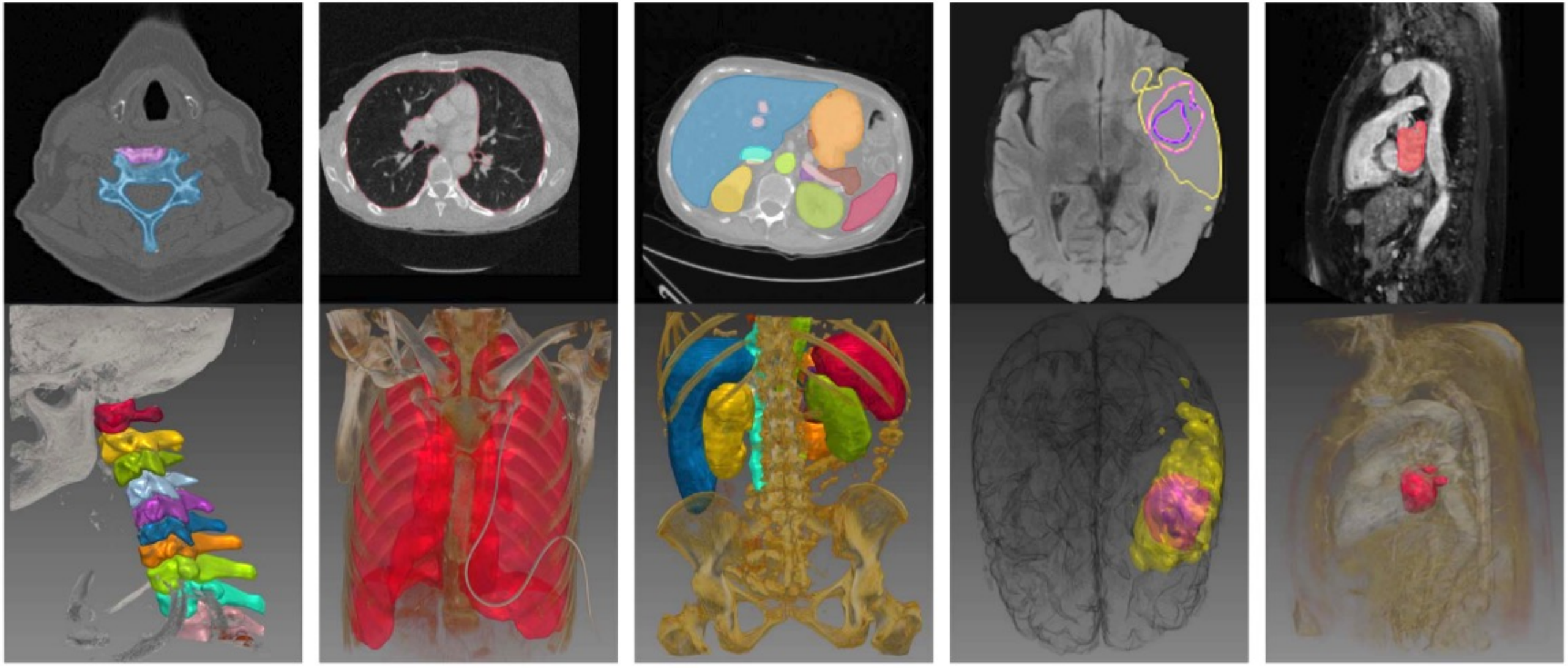
DNN: Image Segmentation

- Some U-Net applications in medical image analysis



DNN: Image Segmentation

- Some U-Net applications in medical image analysis



DNN: Image Segmentation

- U-Net is also being explored for text analysis in language models
- arxiv.org/pdf/2506.14761

From Bytes to Ideas: Language Modeling with Autoregressive U-Nets

Mathurin Videau[1,*], **Badr Youbi Idrissi**[1,*], **Alessandro Leite**[3], **Marc Schoenauer**[2], **Olivier Teytaud**[1], **David Lopez-Paz**[1]

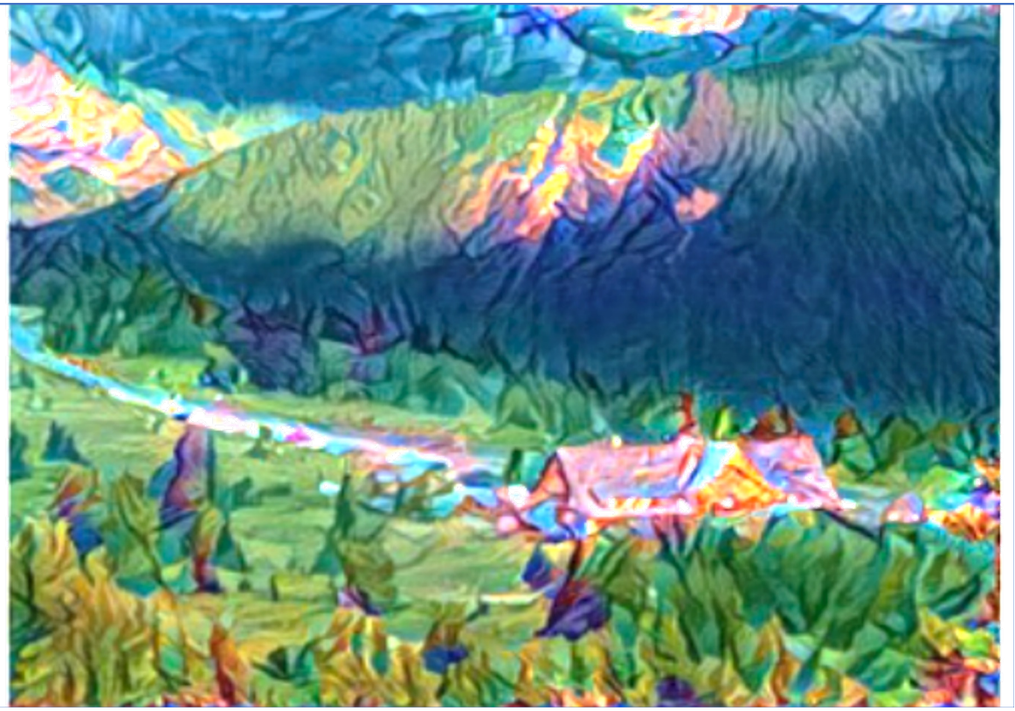
[1]FAIR at Meta, [2]TAU, INRIA and LISN, CNRS & Université Paris-Saclay, [3]INSA Rouen Normandy, LITIS, Rouen, France

CNN: Style Transfer

- Style Image

Content image

Synthesized image



CNN: Style Transfer

- CNN's early layers learn to detect simple features, e.g., edges, textures
- CNN's later layers learn to detect more complex, e.g., objects
- **Goal**: Use these to change image "style", while preserving "content"
- Generate image G whose content is from image C, style is from image S
- Train CNN to minimize loss function:
  $E(G) = E_{content}(G,C) + E_{style}(G,S)$
- Designing $E_{content}(G,C)$:
  At some chosen deeper convolutional layers within CNN,
  match activations in all channels k at all locations (i,j)

$$E_{\text{content}}(\mathbf{G}, \mathbf{C}) = \sum_{i,j,k} \{a_{ijk}(\mathbf{G}) - a_{ijk}(\mathbf{C})\}^2$$

  - Which layers to select is key

CNN: Style Transfer

- Designing $E_{style}(G,S)$
- Style is modeled by co-occurrence of features from different channels within a convolutional layer
  - e.g., within style image,
    horizontal lines co-occur with red dots, & vertical lines co-occur with blue dots
  - But our notion of style doesn't care about location of lines-dots
- Each channel models a feature; use correlation across channels
  - **Unlike** $E_{content}(G,C)$ that dictates that
    at layer k,
    if C gives higher $a_{ijk}(C)$ at location (i,j)
    then G should also give higher activation at that location

$$F_{kk'}(\mathbf{G}) = \sum_{i=1}^{I} \sum_{j=1}^{J} a_{ijk}(\mathbf{G}) a_{ijk'}(\mathbf{G})$$

- Style loss for a layer matches correlations

$$E_{\text{style}}(\mathbf{G},\mathbf{S}) = \frac{1}{(2IJK)^2} \sum_{k=1}^{K} \sum_{k'=1}^{K} \{F_{kk'}(\mathbf{G}) - F_{kk'}(\mathbf{S})\}^2$$

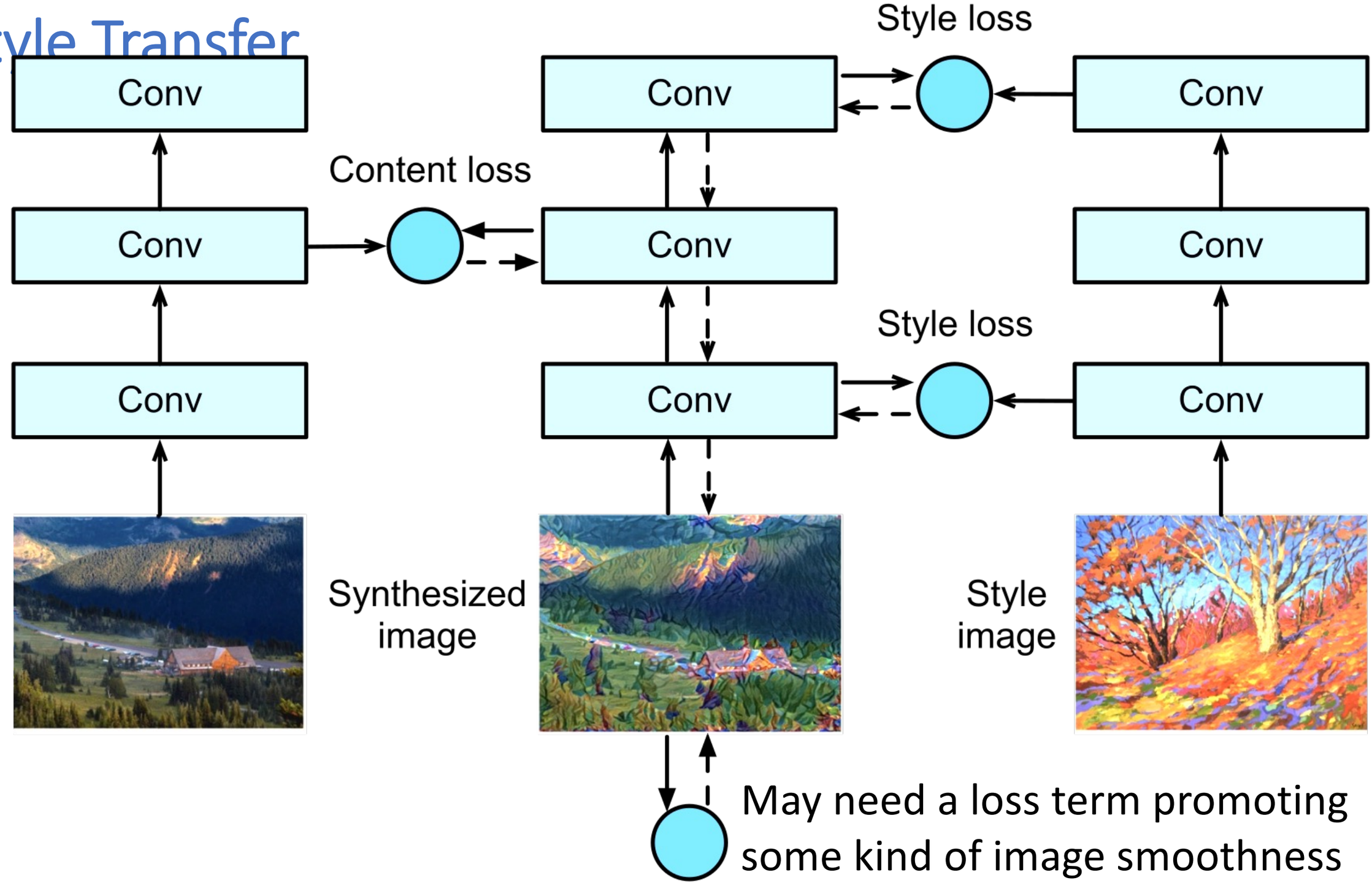
CNN: Style Transfer

- Designing $E_{style}(G,S)$
- In practice, apply style loss at a chosen subset of layers

$$E_{\text{style}}(\mathbf{G}, \mathbf{S}) = \sum_l \lambda_l E^{(l)}_{\text{style}}(\mathbf{G}, \mathbf{S})$$

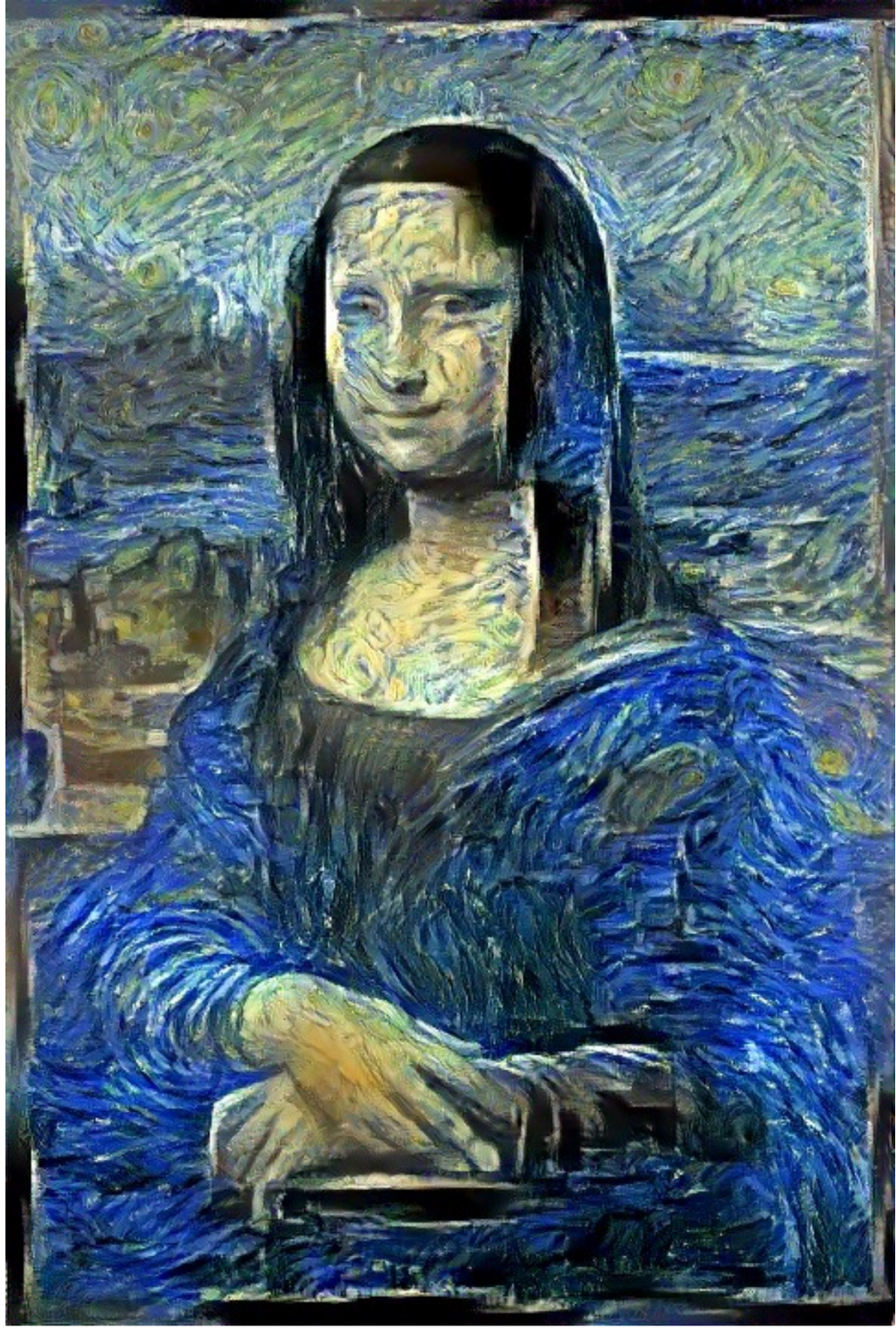
  - Which layers to select is key

CNN: Style Transfer



CNN: Style Transfer

- Style image



CNN: Style Transfer

- Style image

CNN: Style Transfer

- First proposed by [Gatys, Ecker, Bethge 2016 IEEE CVPR]

2016 IEEE Conference on Computer Vision and Pattern Recognition

Image Style Transfer Using Convolutional Neural Networks

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 26, NO. 11, NOVEMBER 2020

Neural Style Transfer: A Review

Yongcheng Jing, Yezhou Yang, *Member, IEEE*, Zunlei Feng, Jingwen Ye,
Yizhou Yu, *Fellow, IEEE*, and Mingli Song, *Senior Member, IEEE*